

The VehicleSim API

Running and Extending VS Math Models

NOTICE

This manual describes software that is furnished under a license agreement and may be used or copied only in accordance with the terms of such agreement. BikeSim, CarSim, SuspensionSim, TruckSim, and VehicleSim are registered trademarks of Mechanical Simulation Corporation.

© 1996 – 2024, Mechanical Simulation Corporation.

Last updated March 2024

Table of Contents

1. Introduction.....	5
Who Should Use this Manual.....	6
Contents of this Manual	6
Definitions	7
Location.....	7
Revision History.....	8
2. Run a VS Math Model	10
Running VS Math Models in Windows	10
Access SDK Examples.....	11
Locate and Load a VS Solver Library.....	12
Automate Runs with vs_run	15
Functions for More Detailed Control of the Run Process	17
Apply VS Commands and Statements Programmatically.....	22
3. Unit Conversions	25
4. Simulate Using Import and Export Arrays	26
Overview of Method	26
API Functions Involving Import and Export Arrays.....	27
Example: Extend a VS Math Model with MATLAB.....	30
5. Access Generic Simulation Information.....	34
Simulation Conditions.....	34
Access Messages	35
Pre- and Post- Processing Callback Functions	37
Get Simulation File Names	38
Write Documentation Files	39
6. Extend a Math Model with C Callbacks	41
7. Access VS Math Model Details.....	52
The VS Symbolic Database.....	52
Access and Extend the VS Math Model.....	52
Interact with Simulation Files	74
Define and Access Configurable Table Functions	78
8. Model Arrays, Save and Restore Options.....	81
Manage Model Arrays.....	81
Save and Restore the Model State	83
9. Steps in the Simulation Process	89
Stages Each Timestep.....	89
Initialize.....	91
Run	93
Terminate	97
Appendix: Legacy API Functions	98
Alternative Legacy Functions	98

1. Introduction

Mechanical Simulation Corporation produces and distributes software tools for simulating and analyzing the dynamic behavior of motor vehicles in response to inputs from steering, braking, throttle, road, and aerodynamics. The simulation packages are organized into families of products named BikeSim[®], CarSim[®], SuspensionSim[®], and TruckSim[®]. All are based on the simulation architecture named VehicleSim[®].

All the Windows versions of VehicleSim products include a main program that provides a GUI for managing a database of vehicle and test descriptions, running the simulation models, visualizing results through animation and plotting, and accessing documentation. This main program (e.g., `carsim.exe` in CarSim) is called the *VS Browser*.

A *VS Math Model* is an entity constructed to run a simulation in a VehicleSim product. It contains variables describing the model and equations and algorithms to calculate their values. It reads input files and write output files for viewing with plots and animation or exported to other software for additional analysis.

A *VS Solver* is a dynamic library with functions that are used to construct VS Math Models.

A *CPAR* (originally short for Consolidated Parsfile) is a compressed archive file (with extension `.cpar`) that can contain multiple files that are easily imported into a database using a VS Browser or other tools.

The *VS software development kit (SDK)* is a collection of documentation, examples, source code in several languages, and VS Solvers associated with BikeSim, CarSim, and TruckSim. The SDK files for each product are also included in a regular release package (Windows and Linux) in the form of CPAR archive files such that they can be easily accessed.

All the SDK documentation is available in the Help menu of a VS Brower and in a folder named `Help\VS_API`.

The VS Browser has several screens for running simulations. The home screen (**Run Control**) loads a VS Solver and constructs a VS Math Model automatically whenever you click a **Run Math Model** button. The VS Browser accesses the VS Solver using the VS application program interface (API) — a set of functions that exist in all VS Solver libraries.

At least four general methods can be used to extend or automate a VS Math Model:

1. The VS Library can be run from external simulation software such as Simulink that combines the VS Math Model with modeling in the external workspace.
2. Equations can be added at runtime using VS Commands and Embedded Python.
3. Variables can be exchanged with an external custom program written in any programming language that can load a dynamically linked library (Windows DLL or Linux SO) and access the major functions (e.g., MATLAB, Visual Basic).
4. Variables in the VS Math Model can be accessed directly by a custom program written in C/C++, or other programming languages that can access C API functions from a DLL/SO.

Calculations in the wrapper program can be tightly integrated with the calculations built the VS Math Model.

These methods can be applied independently or in combination. The second and third options involve interacting with the VS Math Model using API functions as described in this manual.

Who Should Use this Manual

The information in this document is intended for software engineers who wish to:

- run VS Math Models from other software, and/or
- extend the vehicle models with custom code.

This manual is a continuation of the material presented in the *VS Math Models Manual*. Besides being familiar with the material covered in the *VS Math Models Manual*, you should have basic knowledge about how your VehicleSim product is used. As a minimum, you should have gone through the Quick Start Guide for your VehicleSim product.

Many of the API functions support VS Commands that can be provided at runtime to extend the math model in the VS Math Model without using external software. These are documented separately in the *VS Commands Manual*.

Contents of this Manual

This manual is a reference that defines the API functions and provides details of how a simulation runs, combining the internal model equations with extensions made from external workspaces such as Simulink and LabVIEW, model extensions made with the VS Command scripting language, and model extensions made in external programs.

All API functions can be used from programs written with a language that support interfacing with C API.

The manual is organized to first cover the running of a VS Math Model with no model extensions, and to then cover the full capabilities of accessing details of the math model to extend it.

Chapter 2 describes how a VS Solver library is loaded and used to create and run a VS Math Model from other software. Examples are shown for C and MATLAB.

Chapter 3 describes the two unit-systems used in VS Math Models.

Chapter 4 explains how to extend a VS Math Model using Import and Export arrays to communicate between the calling program and the solver. An example is shown using MATLAB.

Chapter 5 describes some utility functions that are useful in most applications.

Chapter 6 explains how to extend a VS Math Model from C using callback functions.

Chapter 7 covers functions in the VS API that access details of the VS model in general, including Configurable Functions.

Chapter 8 covers functions for managing standard arrays that exist in most VS Solver libraries, including options to save and restore the state of the math model.

Chapter 9 goes through the simulation process, step by step, to fully document the interactions and priorities when considering built-in equations, Import and Export arrays, VS Commands, and external callback functions.

The appendix reviews legacy API functions that have been retired or which are no longer needed (but might be supported to provide backward compatibility).

Example source code for applying the API is provided in MATLAB and C as described in two tech memos in the VS SDK Help folder. The first — *Automating Runs with the VS API* — describes examples that simply load a VS Solver and run a VS Math Model with example programs written in MATLAB, C, VB, and Python. The second memo — *Extending VS Math Models with VS Commands and the VS API* — describes how a path follower is added to VS Math Models in CarSim, TruckSim, and BikeSim using VS Commands and the API, with example programs written in MATLAB and C.

The SDK tech memos also cover some of the details of how to set up the connection to an external program using the VS Browser and GUI. These details are not repeated in this reference manual, which is intended to serve as the definitive reference for core VS API functions.

Definitions

The terms *VS Math Model*, *VS Solver*, *VS Brower*, *CPAR*, and *SDK* were defined earlier. Following are a few other terms used in this manual.

DLL is a dynamically linked library. The acronym *DLL* as used in this documentation also applies for Linux, even though *DLL* files typically are called *shared objects* with the extension *.so*.

The word *variable* is used in many contexts to represent some quantity that is capable of change. With respect to a VS Math Model, a variable is a quantity that is expected to change during a simulation. With respect to program code, a variable is a quantity can vary at different times while the program code is being executed.

The word *parameter* refers to a variable in VS Math Model that is set at runtime (before the simulation starts) and remains constant until a new value is set.

The word *argument* refers to an argument passed to a function in a programming language.

A *wrapper* is a program module that loads a VS Solver and then builds and runs a VS Math Model.

When capitalized, *Import* and *Export* refer to arrays use to share variables between a wrapper and a VS Math Model. Import arrays have variables received by the VS Math Model from the wrapper; Export arrays have variables calculated by the VS Math Model and shared with the wrapper.

When showing the syntax for data, words shown in the Courier font indicate specific literal keywords or file names; words in *italics* indicate the name of the item whose value is variable (e.g., the argument to a function), and square brackets [like this] indicate that the content between the brackets is optional.

Location

The examples shown in this document are provided as CPAR archives in the regular Windows installations of CarSim, TruckSim, and BikeSim. The installation always includes three VS Solvers

libraries for the product, for running on Windows 32-bit, 64-bit, and Linux (64-bit). The installations also include tools such as a wrapper programs with a command-line interface and various utilities.

The VS software development kit (SDK) is also available for software engineers interested in just the tools and documentation needed to build custom wrappers for running VS models. The VS SDK can be downloaded from the Mechanical Simulation website.

Revision History

This manual applies to the solvers in CarSim, TruckSim, SuspensionSim, and BikeSim. The version of the manual is indicated by the date shown at the bottom of page ii. The manual is updated when new features are added to the VehicleSim architecture, when errors are found in the current version, or when improved descriptions have been written.

- The February 2024 revision describes how SDK examples are available in CPAR archives provided with VS products and updates the descriptions of existing examples. The example C and MATLAB files were updated to separate deprecated API functions that are no longer used.
- The October 2023 revision changed the timelines in Chapter 9 to show the effects of the new parameter `OPT_DELAY_INIT`.
- The April 2023 revision added the option to install a callback function whenever a log entry is generated by the VS Math Model during a run. More edits were made to cover changes made in versions 2023.0 and 2023.1.
- The November 2022 revision moved the Linux information from a separate chapter into Chapter 2. The function names in this document were checked against those installed with `vs_get_api`, and some missing descriptions were added. New functions were introduced for I/O variables to include the location in the VS Solver where that are used or calculated: `vs_define_imp_where` and `vs_define_out_where`. Descriptions of functions that had been discontinued prior to version 9.0 were removed. New callback locations were added to Chapter 7, and timeline descriptions were updated in Chapter 9. Many figures showing source code listings were updated.
- The April 2022 revision added a chapter describing typical usage of a VS Math Model in Linux. It updated the description of `vs_get_api.c` to match the current practice of providing an SDK package, and the options for importing CPAR mini databases with SDK examples. It also added detail to the descriptions of how import variables are handled.
- The April 2021 revision added more API functions that were introduced recently, updated figures for SDK examples, and updated references to VS Solver and VS Math Models.
- The October 2020 revision noted that the API for path and road functions have been moved to `vs_api_road.h`.

- The June 2020 revision moved API functions related to paths and roads to the *Paths and Road Surfaces* document. Moved ADAS functions to the *ADAS Sensors and Moving Objects* document. Updated the timeline for initialization.
- The October 2019 revision added a section on Location to clarify where the examples can be found.
- The February 2019 revision fixed a typo for the function `vs_path_yaw_id`.
- The November 2018 revision added a new API function: `vs_path_yaw_id`. Details of the initialization sequence were updated. Documentation for old API functions for working with a single path (used by the driver model) and road are limited given the current capability of solvers to support 200 paths and 100 roads, with different tires possible interacting with different road surfaces. The functions still exist, but the documentation has been moved to the appendix with other deprecated functions.
- The April 2018 revision added the `VS_EXT_EQ_DYN` location for callbacks and has a rewritten Chapter 9 that describes the timelines for the simulation sequences as revised in version 2018.1 of the products.
- The November 2017 revision added information about ID numbers associated with parameters and variables, removed obsolete information about multiple custom solvers, and mentioned the need to wait until after initialization before using the function `vs_get_tstep`.
- The April 2017 revision added documentation on string length restrictions and fixed a timeline figure.
- This document was created in February 2007 for the release of CarSim 7.0, with API functions to control the simulation from external custom software written in C, and access variables in the model. Over the years, content was added as many of the functions and tools used within Mechanical Simulation to create VS Solvers were made available for advanced users via the VS API. Access was also added for MATLAB and other languages. The `vs_statement` function was introduced to allow VS Commands to be applied from custom external software as an alternative to reading the commands from text Parsfile. Multiple revisions were made prior to the oldest supported versions of the VehicleSim tools (2017.0) in December 2016.

2. Run a VS Math Model

A VS Solver is a dynamically linked library file. The library is loaded by another program that builds the VS Math Model and runs it by using the VS API function.

The VS Solver has internal modules with “native” built-in equations that are assembled to create a VS Math Model. For VehicleSim products such as BikeSim, CarSim, and TruckSim, the math models are designed to predict the behavior of conventional vehicles in response to driver/rider controls. The VS Math Model can be extended to handle custom controllers and alternate component models using an external simulation environment such as Simulink and LabVIEW. The VS Solver can also be run under the control of other software for automation or to apply the math model equations for other applications, such as optimizations.

This chapter describes how to load a VS Solver and build and run a VS Math Model using a single API function `vs_run`, with examples in MATLAB and C/C++ (running under the Windows OS). Similar examples for VB and Python are also provided in the SDK Tech Memo *Automating Runs with the VS API*.

Running VS Math Models in Windows

From the user point of view, the simplest environment for running a VS Math Model under the Windows OS is to click a **Run Math Model** button from a VS Browser such as `carsim.exe`. When you do this, the VS Browser loads the appropriate VS Solver, builds a VS Math Model, and runs the simulation. No external software is involved in this basic interaction between the VS Browser and functions in the VS Solver library.

CarSim, TruckSim, and BikeSim each have two VS Solver DLLs: one to use with 32-bit Windows programs, and one for 64-bit programs. These solvers contain modules for parts of the vehicle (e.g., in CarSim this include a simple sprung mass, sprung mass with frame twist, independent suspension, solid-axle suspension, etc.). When run from the VS Browser, a 32-bit application, the 32-bit VS Solver is loaded, e.g., `carsim_32.dll`.

Note	The Windows installations also include a Linux shared object file with extension <code>.so</code> . It cannot be run directly under Windows but can be moved to Linux machines without requiring a separate download.
-------------	---

Sometimes users want to run a VS product together with third-party simulation environments, such as Simulink, LabView, ASCET, or custom software. Integration with third-party software is usually achieved with a wrapper module that connects the functions from the VS Solver DLL to the other environment (e.g., Simulink). The wrapper communicates with the calling environment in a way that is standard for that environment and communicates with the VS Solver DLL using the VS API.

For example, Figure 1 shows how a Simulink model with a brake controller is used with a VehicleSim product. In this case, the solver is `carsim_64.dll`.

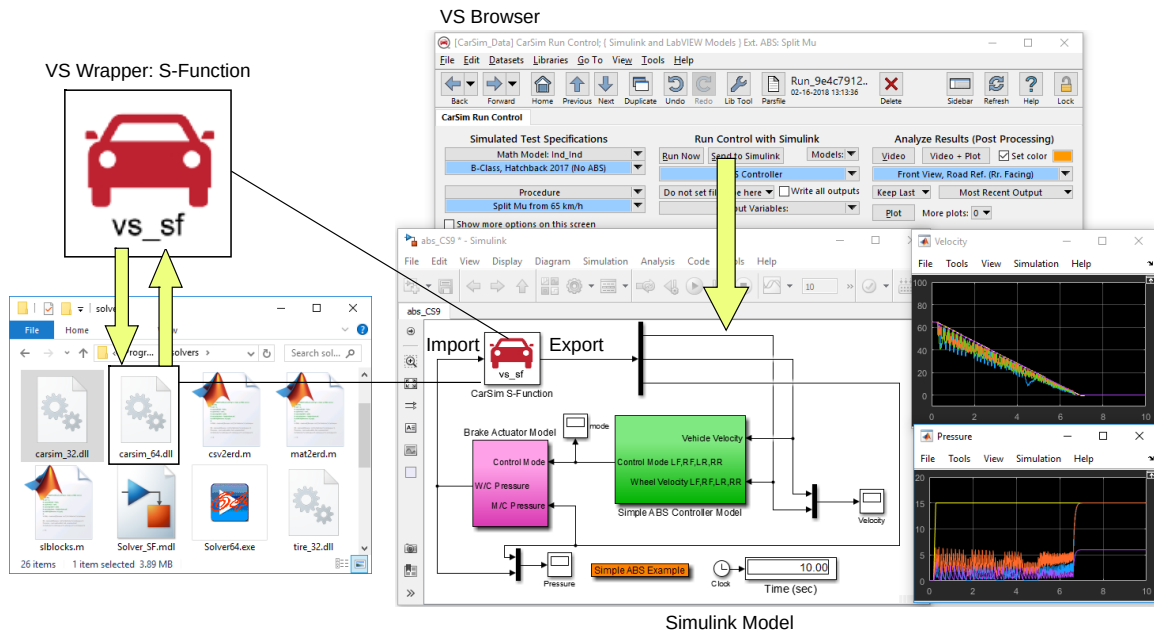


Figure 1. Using a VS Solver library with a wrapper for Simulink.

The wrapper for Simulink is an S-Function that appears as a block in the Simulink environment. The VS Browser launches a Simulink model, prepares input Parsfiles needed by the VS Solver and VS Math Model, and sends commands to Simulink using Windows COM. During the simulation, there is close communication between Simulink and the wrapper S-Function DLL, and between the wrapper and the VS Solver DLL.

Details for programming a wrapper like the S-Function provided with VS products are provided in Chapter 4: *Simulate Using Import and Export Arrays* (page 26).

Access SDK Examples

CarSim, TruckSim, and BikeSim include the examples described in this manual. They are included in two CPAR archive files that are available to be added to any new database or imported into an existing database: **SDK: External Control** and **SDK: Extended Models**.

This collection of datasets is available when building a database in the Database Builder in the VS Browser. It may also be imported into an existing database using the **File>Import Parsfile (Any Export Type)** menu item. The CPAR archives are in the folder `Resources\CPAR_Archives` in the product `_Prog` folder, e.g., `CarSim_Prog\Resources\CPAR_Archives`.

Each CPAR archive includes datasets needed to reproduce runs made with external wrapper programs, plus the external wrappers and the code used to build them. These are in a folder `Extensions` added to the database. It in turn has folders for building the example wrappers in different languages. For example, Figure 2 shows the contents of the folder for Custom C examples, and Figure 3 shows the contents of the `wrapper_simple` example. The code and project files were used to build two Windows programs: `wrapper_simple_Release_Win32.exe` and `wrapper_simple_Release_x64.exe`.

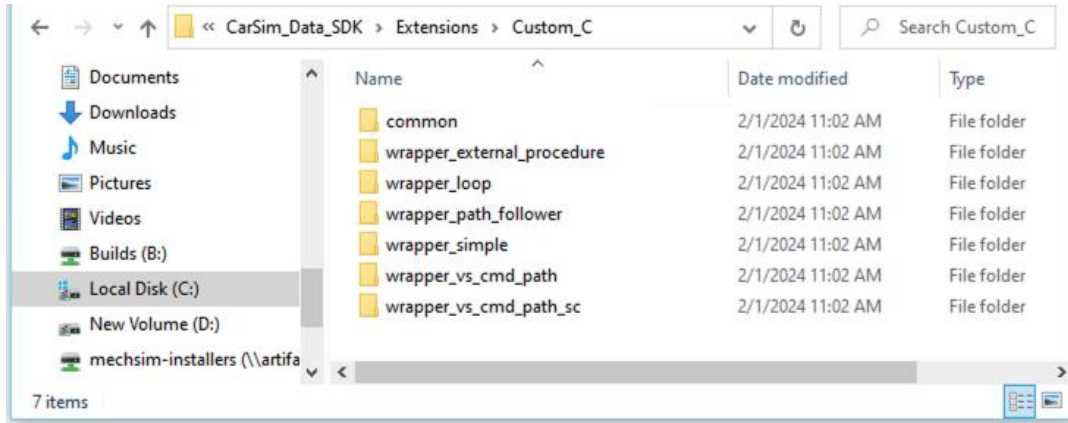


Figure 2. Folder with Custom C examples, located in the Extensions folder added with a CPAR.

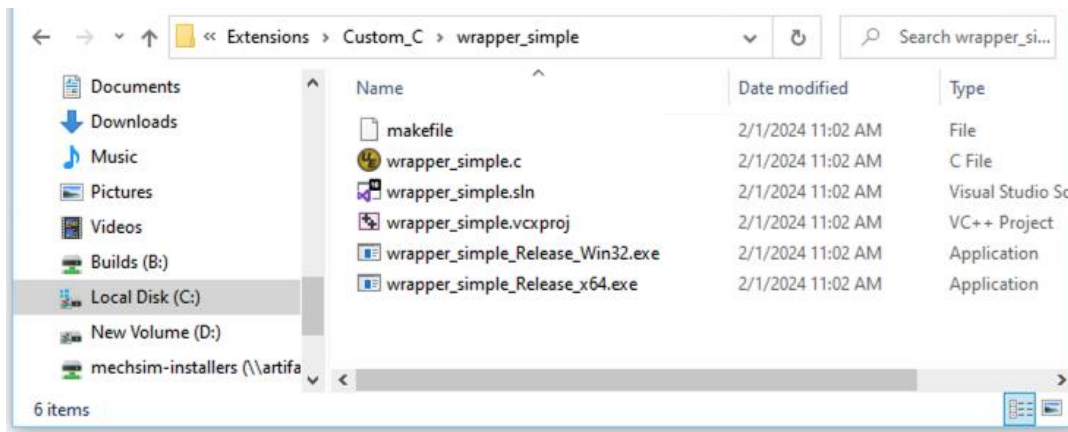


Figure 3. Source code, project files, and wrapper programs for a simple C example.

The SDK memo *Extending VS Math Models with VS Commands and the VS API* provides more detail about how to access the API examples.

Locate and Load a VS Solver Library

A VS wrapper typically starts with a text simulation control file (*Simfile*) that is used to locate the appropriate VS Solver library file. The wrapper performs these steps:

1. Locate the VS Solver library from information in the Simfile.
2. Load the VS Solver into memory.
3. Connect to some of the API functions in the VS Solver.
4. Use some of the API functions to run a simulation.
5. Unload the solver.

Table 1 lists the major functions in the file `vs_get_api.c` that is provided in the VehicleSim SDK (`vs_get_api.c` is in the folder `Extensions\Custom_C\common`).

Table 1. Major functions in *vs_get_api.c* to access API functions in a VS Solver.

Function	Description
<code>vs_free_library</code>	Free a loaded library
<code>vs_get_api</code>	Access sets of VS API functions from a VS Solver library. <code>vs_get_api</code> gets most functions; <code>vs_get_core</code> and <code>vs_get_api_road</code> get subsets of functions.
<code>vs_get_api_core</code>	
<code>vs_get_api_road</code>	
<code>vs_get_api_f</code>	Access a single named API function.
<code>vs_get_dll_path</code>	Read a Simfile and get a library pathname
<code>vs_load_library</code>	Load a library
<code>vs_set_error_message_function</code>	Assign function for handling error messages

These functions help to locate and load a VS Solver DLL from C. The API functions are declared in companion header files `vs_api.h` and `vs_api_road.h`.

vs_free_library

Declaration:

```
void vs_free_library (HMODULE dll);
```

This function frees the library using calls that are specific to the OS (Windows DLL or Linux shared object).

vs_get_api

Declaration:

```
int vs_get_api(HMODULE dll, char *dll_path);
```

Return:

This function returns a value of 0 if successful, -1 if not.

This function tries to load all the API functions that are included in the VS Solvers for CarSim, TruckSim, and BikeSim. It tries to load most of the functions declared in the companion header files `vs_api.h` and `vs_api_road.h`. With each new release of a VehicleSim product, it is possible that new API functions will be available. The API source files are typically updated to include the new functions. Names and properties of existing functions are rarely changed, so any existing C/C++ programs should continue to work without any modification or recompiling.

vs_get_api_core

Declaration:

```
int vs_get_api(HMODULE dll, char *dll_path);
```

Return:

This function returns a value of 0 if successful, -1 if not.

This function tries to load all the API core functions that exist in all VS Solver libraries, including those that are not full vehicle models, such as the SuspensionSim solver and tire tester solvers.

vs_get_api_road

Declaration:

```
int vs_get_api(HMODULE dll, char *dll_path);
```

Return:

This function returns a value of 0 if successful, -1 if not.

This function tries to load all the API functions that are declared in the companion header file `vs_api_road.h` and documented in the **Path and Road Surfaces** Help document (`roads.pdf`).

vs_get_api_f

Declaration:

```
int vs_get_api_f(void *api_func, HMODULE dll, const_char *func,
                 const_char *dll_name, const_char *get_func);
```

Return:

This function returns a value of 0 if successful, -2 if not.

This obtains a pointer to an API function named *func*, contained in *dll*, and assigns it to *api_func*. The local copy *api_func* must be declared to match the form of function in the *dll*.

The strings *dll_name* and *get_func* are used to generate text in an error message if the function fails to locate the requested function pointer.

vs_get_dll_path

Declaration:

```
int vs_get_dll_path(const char *simfile_path, char *dll_path);
```

Return:

This function returns a value of 0 if successful, -1 if not.

Call this function with a specified pathname for a Simfile *simfile_path*; the function scans the Simfile and finds the pathname *dll_path* for the DLL.

The pathname *dll_path* might be specified directly in the Simfile with the keyword `DLLFILE`. If the Simfile does not include the `DLLFILE` keyword, this function uses other information in the Simfile to find the pathname for the VS Solver that matches the product name, Prog folder location, and “bitness” of the wrapper (32-bit or 64-bit).

vs_load_library

Declaration:

```
HMODULE vs_load_library(char* pathDLL);
```

This function loads the library using calls that are specific to the OS (Windows or Linux).

vs_set_error_message_function

Declaration:

```
void vs_set_error_message_function (  
    int(*errorMsgFunc) (const char *format, ...))
```

This function specifies a new function to print error messages. By default, messages are shown in Windows OS with pop-up windows and in Linux by printing to the terminal.

If this function is used, the function *errorMsgFunc* should have the same calling convention as the C function *printf*.

Automate Runs with vs_run

The API function *vs_run* attempts to run a simulation.

vs_run

Declaration:

```
int vs_run(const char *simfile)
```

Return:

Error code: zero means everything was OK, not zero means there was a problem.

Note For consistency throughout this manual, all definitions and descriptions use C syntax.
--

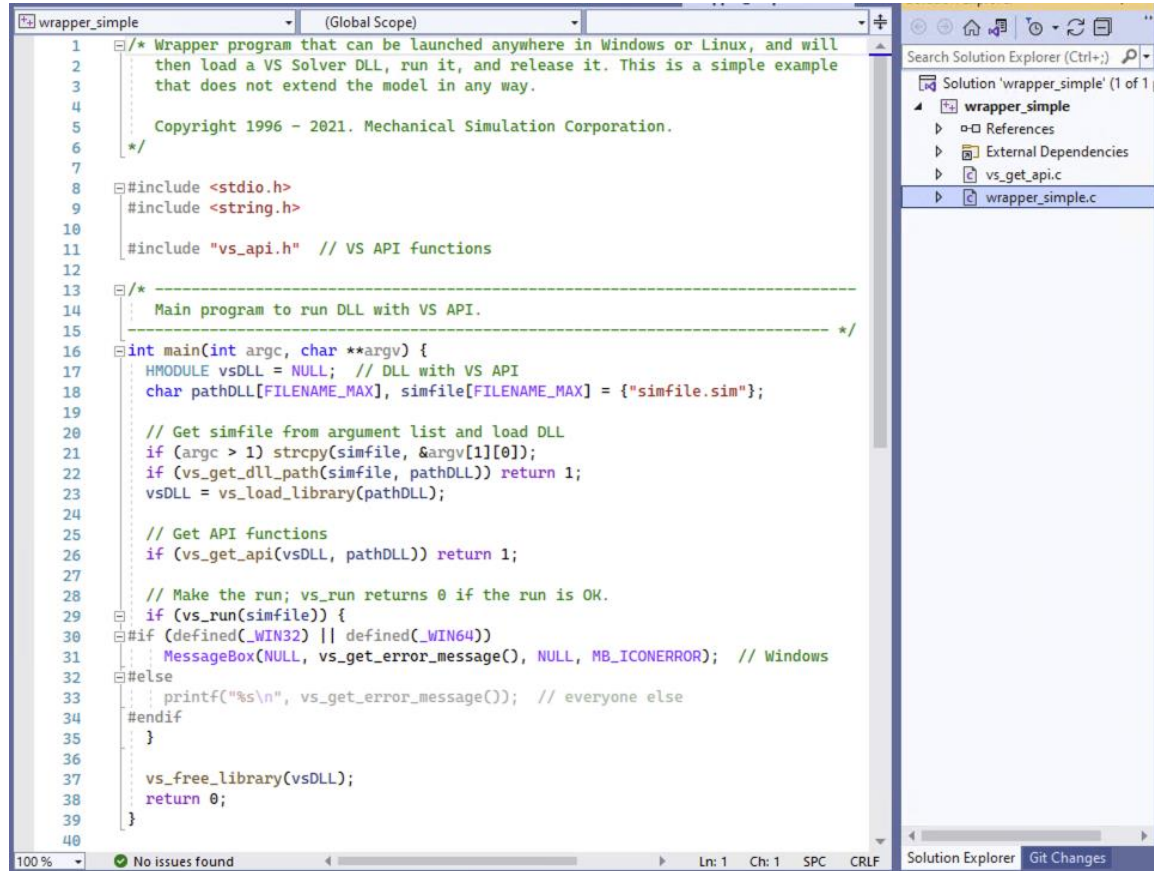
This function performs the standard steps performed by a VS Math Model:

1. **Initialize.** Read the Simfile (again) to obtain pathnames for input and output files that will be used for the run, along with other information that might be used to set up the simulation. Read input data and then create and initialize the VS Math Model.
2. **Loop through the run.** Run the simulation using the VS Math Model to repeatedly integrate equations of motion for the model over the specified timestep. The simulation includes the native built-in equations, equations added with VS Commands, and callback functions installed using the API functions (described later). It does not update Import and Export arrays that are used to pass variables back and forth between the VS Math Model and other programs.
3. **Terminate.** Finish any output files and close them. Free all memory that was allocated when the solver initialized the math model.

This function is intended for use in most ANSI C programs. For higher-level languages such as MATLAB and VB, *vs_run* is suitable when the intent is to simply run the VS Math Model with the math model “as is.”

Example: Run a Simulation from C

Figure 4 shows the source code for a C program that is provided with the `vs_sdk` and is described in the Tech Memo *Automating Runs with the VS API*. It simply loads a VS Solver DLL, makes a run, and quits.



```
1  /* Wrapper program that can be launched anywhere in Windows or Linux, and will
2     then load a VS Solver DLL, run it, and release it. This is a simple example
3     that does not extend the model in any way.
4
5     Copyright 1996 - 2021. Mechanical Simulation Corporation.
6  */
7
8  #include <stdio.h>
9  #include <string.h>
10
11 #include "vs_api.h" // VS API functions
12
13 /* -----
14    Main program to run DLL with VS API.
15    ----- */
16 int main(int argc, char **argv) {
17     HMODULE vsDLL = NULL; // DLL with VS API
18     char pathDLL[FILENAME_MAX], simfile[FILENAME_MAX] = {"simfile.sim"};
19
20     // Get simfile from argument list and load DLL
21     if (argc > 1) strcpy(simfile, &argv[1][0]);
22     if (vs_get_dll_path(simfile, pathDLL)) return 1;
23     vsDLL = vs_load_library(pathDLL);
24
25     // Get API functions
26     if (vs_get_api(vsDLL, pathDLL)) return 1;
27
28     // Make the run; vs_run returns 0 if the run is OK.
29     if (vs_run(simfile)) {
30         #if (defined(_WIN32) || defined(_WIN64))
31             MessageBox(NULL, vs_get_error_message(), NULL, MB_ICONERROR); // Windows
32         #else
33             printf("%s\n", vs_get_error_message()); // everyone else
34         #endif
35     }
36
37     vs_free_library(vsDLL);
38     return 0;
39 }
40
```

Figure 4. Simple C program to load a VS Solver and run a VS Math Model.

Supporting code for scanning the Simfile to locate the VS Solver is provided in the separate file `vs_get_api.c` that was described in the previous subsection. The API functions are declared in a header file named `vs_api.h`.

Two functions from `vs_get_api.c` are used for this example:

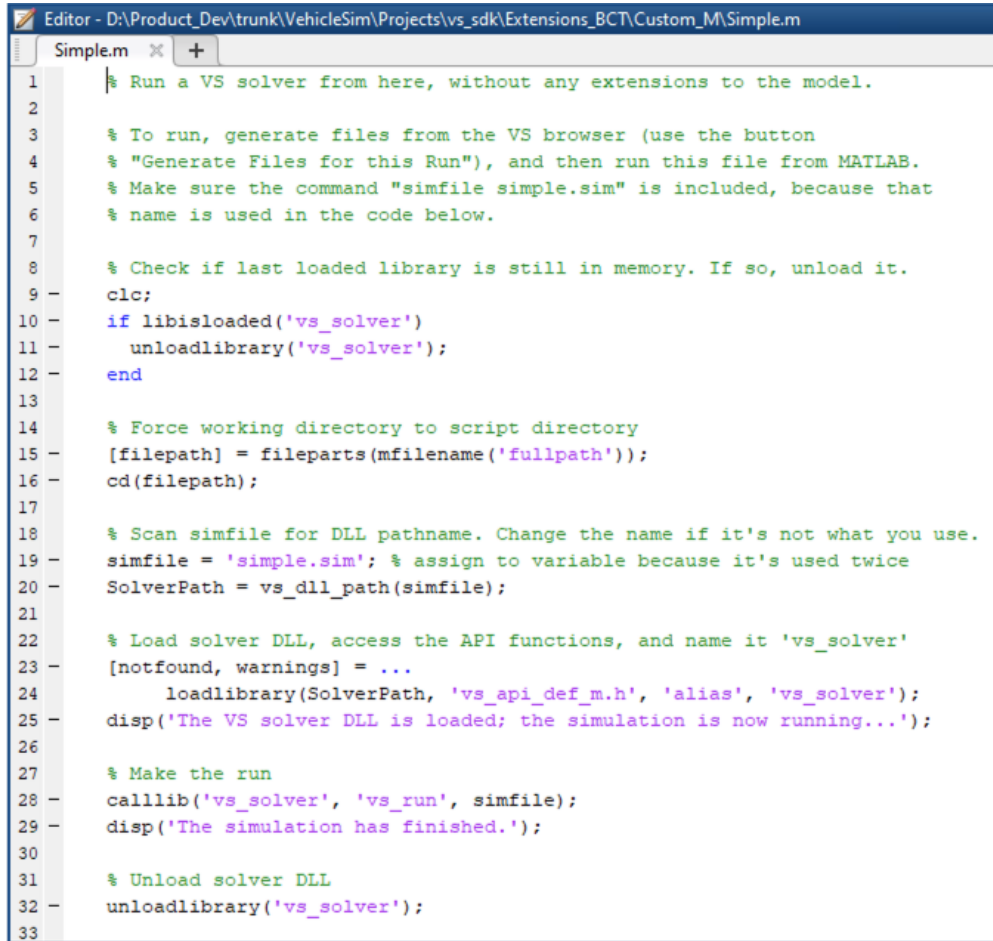
- `vs_get_dll_path` reads a specified Simfile and extracts the pathname for the DLL (line 22).
- `vs_get_api` gets access to all the API functions in the DLL (line 26).

In the example, the name of the Simfile is an optional argument (lines 20 and 21) that defaults to `"simfile.sim"` if not provided (line 18). The Simfile name is given to `vs_get_dll_path` (line 22) and, if successful, the library is loaded (line 23), and the API functions are accessed with `vs_get_api` (line 26).

Only two API functions are used: `vs_run` (line 29) and `vs_get_error_message` (line 31 or 33, described in Chapter 5, page 36).

Example: Run a Simulation from MATLAB

Figure 5 shows a MATLAB M file that is provided in `vs_sdk` and is described in the Tech Memo *Automating Runs with the VS API*. It loads a VS Solver DLL and makes a run.



```
1  % Run a VS solver from here, without any extensions to the model.
2
3  % To run, generate files from the VS browser (use the button
4  % "Generate Files for this Run"), and then run this file from MATLAB.
5  % Make sure the command "simfile simple.sim" is included, because that
6  % name is used in the code below.
7
8  % Check if last loaded library is still in memory. If so, unload it.
9  - clc;
10 - if libisloaded('vs_solver')
11 -     unloadlibrary('vs_solver');
12 - end
13
14 % Force working directory to script directory
15 - [filepath] = fileparts(mfilename('fullpath'));
16 - cd(filepath);
17
18 % Scan simfile for DLL pathname. Change the name if it's not what you use.
19 - simfile = 'simple.sim'; % assign to variable because it's used twice
20 - SolverPath = vs_dll_path(simfile);
21
22 % Load solver DLL, access the API functions, and name it 'vs_solver'
23 - [notfound, warnings] = ...
24     loadlibrary(SolverPath, 'vs_api_def.m.h', 'alias', 'vs_solver');
25 - disp('The VS solver DLL is loaded; the simulation is now running...');
26
27 % Make the run
28 - calllib('vs_solver', 'vs_run', simfile);
29 - disp('The simulation has finished.');
```

Figure 5. Loading a VS Solver and running a VS Math Model from MATLAB.

This makes use of an M-file function `vs_dll_path` (line 20) that loads the specified Simfile "simple.sim" and scans it for the location of the VS Solver DLL. That location is assigned to the string `SolverPath`.

VS API functions are declared in another file (`vs_api_def.m.h`) that is identified when loading the DLL library (line 24).

The API function `vs_run` is used to make the run (line 28). The DLL is unloaded when the run completes (line 32).

Functions for More Detailed Control of the Run Process

Some applications require more detailed control of the run process than is provided by `vs_run`. For example, it is sometimes helpful to provide VS Commands by program rather than writing to Parsfile. Table 2 lists the functions that are used to provide the functionality of `vs_run`, whose source code is in turn shown in Listing 1. Besides documenting the `vs_run` function, this listing

shows the role of the functions listed in Table 2 that are essential for a simulation. (The function names in the listing are shown in bold.)

Table 2. Functions that control the basic run process.

Function	Description
vs_bar_graph_update	Indicate if simple bar graph should be updated.
vs_error_occurred	Did an error occur since the last check?
vs_initialize	Calculate initial conditions and set up output files.
vs_integrate	Integrate one timestep and update all outputs.
vs_set_macro	Create and/or set the value of a Simfile macro.
vs_setdef_and_read	Scan the Simfile, create an internal keyword database, define parameters and variables, install units, scan all input Parsfiles.
vs_statement	Apply a specified VS Command or statement.
vs_stop_run	Determine if the run should be stopped.
vs_terminate	Finish writing files and free memory.
vs_terminate_run	

Note The **vs_run** listing makes use of a local function `sPrintMessage` (not shown) to print messages to a console output.

vs_bar_graph_update

Declaration:

```
int vs_bar_graph_update(int *ibar);
```

Return:

1 if a progress bar should be updated; 0 if not.

Support the dynamic display of a simple bar graph: if it is not time to update the bar graph then return 0. If it is time to update, then increment **ibar*, print '=' to console (in case the calling program has provided a standard output) and return 1. The variable **ibar* will cover a range of 0 to 50 as the run proceeds.

vs_error_occurred

Declaration:

```
int vs_error_occurred(void);
```

Return:

Error code: 0 means everything is OK; not zero means the run should be stopped.

If an error occurs, two internal variables are set to TRUE: one is the error condition returned by this function, and the other is the stop condition returned by the function **vs_stop_run**.

Listing 1. Source code for `vs_run`, using mostly functions from Table 2.

```
// API function to run. Model extensions come from installed external functions.
int vs_run (const char *simfile) {
    int ibarg = 0;
    vs_real t;
    char *printMsg = vs_get_output_message(); // pointer to text from DLL

    // Initialize VS model
    t = vs_setdef_and_read(simfile, NULL, NULL);
    if (vs_error_occurred()) {
        vsWriteEchoFile (vs_get_echofile_name(), false, NULL);
        vs_free_all ();
        return 1;
    }
    vs_initialize (t, NULL, NULL);

    if (vs_error_occurred()) {
        vs_free_all ();
        return 1;
    }
    sPrintMessage (printMsg);

    // Run. Each loop advances time one full step.
    while (!vs_stop_run()) {
        vs_integrate(&t, NULL);
        vs_bar_graph_update(&ibarg); // update bar graph?
    }

    // Terminate
    vs_terminate_run (t);
    sPrintMessage (printMsg);
    return vs_error_occurred();
}
```

The internal variable that is set TRUE when the run should be stopped is also triggered if an error occurs. Thus, if `vs_error_occurred` were to return a nonzero value, then so would `vs_stop_run`.

vs_initialize

Declaration:

```
void vs_initialize (vs_real t,
                  void (*external_calc) (vs_real, int),
                  void (*external_echo) (int));
```

This function causes the math model to perform calculations needed to prepare for the numerical integration of the model equations and set initial conditions. This is done after all input Parsfiles have been scanned using `vs_setdef_and_read`.

Some of the tasks performed by this function are described in sequence in the Initialization section in Chapter 9 (page 90).

The argument *t* is the simulated time at the start of the run; it is typically provided by `vs_setdef_and_read`.

The arguments *external_calc* and *external_echo* can be set to either `NULL` or pointers to callback functions. Mechanical Simulation recommends using `NULL` as shown in Listing 1, and specifying any callback functions with the installer functions that are described in Chapter 6.

vs_integrate

Declaration:

```
int vs_integrate(vs_real *t, void (*ext_calc) (vs_real, int));
```

Return:

0 if the run should continue; non-zero if the run should stop.

The API provides several functions for advancing a timestep during a run. The function `vs_integrate` is the most basic, without support for the Import and Export arrays used with external programs.

The first argument is a pointer to the current time *t*. The function updates this variable internally; when the function returns the value will be incremented by the timestep parameter `TSTEP`. This increment is the same regardless of the numerical integration method in use.

The argument *ext_calc* is either `NULL` or a pointer to a callback function. Mechanical Simulation recommends using `NULL` as shown in Listing 1, and specifying any callback functions with the installer function `vs_install_calc_function` that is described in Chapter 6.

vs_set_macro

Declaration:

```
void vs_set_macro (const char *macro_name,  
                  const char* macro_value);
```

Creates a macro that may be expanded during certain phases of reading the Simfile. Macros are only valid for certain Simfile entries (see the Simfile documentation for details).

vs_setdef_and_read

Declaration:

```
vs_real vs_setdef_and_read (const char *simfile,  
                            void (*external_setdef) (void),  
                            vs_bool (*external_scan) (char *key, char *buffer));
```

Return:

Start time. This is usually 0.0 unless an alternative start time is specified in the input Parsfile.

The argument *simfile* is a pathname for the Simfile (simulation control file) that is scanned to obtain the names of files of interest for the simulation (input Parsfile, output Echo files, VS/ERD files, etc.).

This function creates internal structures for a keyword dataset and sets of model parameters and variables. It creates required built-in parameters and variables, Configurable Functions, and units. Next, the input Parsfile specified by the Simfile is scanned to update values and units for existing parameters and variables, and to process VS Commands and other settings provided from the Parsfiles that might add features to the model.

The arguments *external_setdef* and *external_scan* are either `NULL` or pointers to callback functions. Mechanical Simulation recommends using `NULL` as shown in Listing 1, and specifying any callback functions with the installer functions that are described in Chapter 6.

vs_statement

Declaration:

```
int vs_statement(const char *keyword, const char *rest,
                int stopError);
```

Return:

0 if the statement was successfully processed; non-zero if an error occurred.

This function provides the same functionality as any line of input from an input Parsfile if the contents of the line constitute a self-contained statement. A statement is either an assignment for an existing variable identified in the VS Math Model by a *keyword*, or a VS Command.

The only types of input not supported are tables of numbers, which require multiple lines of text.

Keyword is a keyword recognized by the VS Math Model. The rest of the input line is provided with the string *rest*. If *keyword* is a VS Command with no additional information required, then *rest* may be specified as `NULL`.

The argument *stopError* determines the behavior if *keyword* is not recognized. If *stopError* is zero and *keyword* is not recognized, then the failure is written to the log file, but the run continues. If *stopError* is nonzero and *keyword* is not recognized, then an error is reported interactively and the run stops. If *keyword* is recognized, then *stopError* has no effect on the error handling involving the rest of the input *rest*.

Unlike most other API functions, *keyword* is not case sensitive (an internal copy is made and converted to uppercase). This is done to mimic the appearance of keywords provided in input Parsfiles, which are not case sensitive.

Within the VS Math Model, the processing applied to the arguments *keyword* and *rest* is the same as is applied to every line of text read from an input Parsfile. The string *rest* is copied internally and processed with the same routines used to process Parsfile inputs, including error handling.

If the `vs_statement` function is used to define new variables in the VS Math Model, it should be applied before the `vs_initialize` step in the simulation when the number of ODEs in the model is frozen. See the timeline for the initialization for details (Figure 14, page 91).

vs_stop_run

Declaration:

```
int vs_stop_run(void);
```

Return:

0 if the run should continue; not zero if the run should be stopped.

The condition for stopping might be a programmed condition such as reaching a specified stop time. It might also be a condition specified by the user at runtime.

The internal variable that is set TRUE when the run should be stopped is also triggered if an error occurs. Thus, if `vs_error_occurred` were to return a nonzero value, then so would `vs_stop_run`.

vs_terminate

Declaration:

```
void vs_terminate (vs_real t, void (*external_echo) (int));
```

Terminating the simulation performs these steps:

1. write outputs to file if they were buffered in memory,
2. perform a few calculations to support future continuations of the run,
3. write a final Echo file, and
4. free all memory that was allocated by the VS Math Model from the time it was loaded.

The first argument is the current value of time in the simulation.

The argument *external_echo* is either NULL or a pointer to a callback function that will be used in writing the final Echo file. Some languages do not support NULL pointers; if the intent is to skip the *external_echo* option, the function `vs_terminate_run` (below) is recommended.

vs_terminate_run

Declaration:

```
void vs_terminate_run (vs_real time);
```

This function simply applies the `vs_terminate` function with *external_echo* set to NULL. This works with wrappers written for some RT systems that do not support pointers or NULL. It is used in all SDK examples and is recommended as the function to terminate a run when there is no interest in inserting content in the final Echo file.

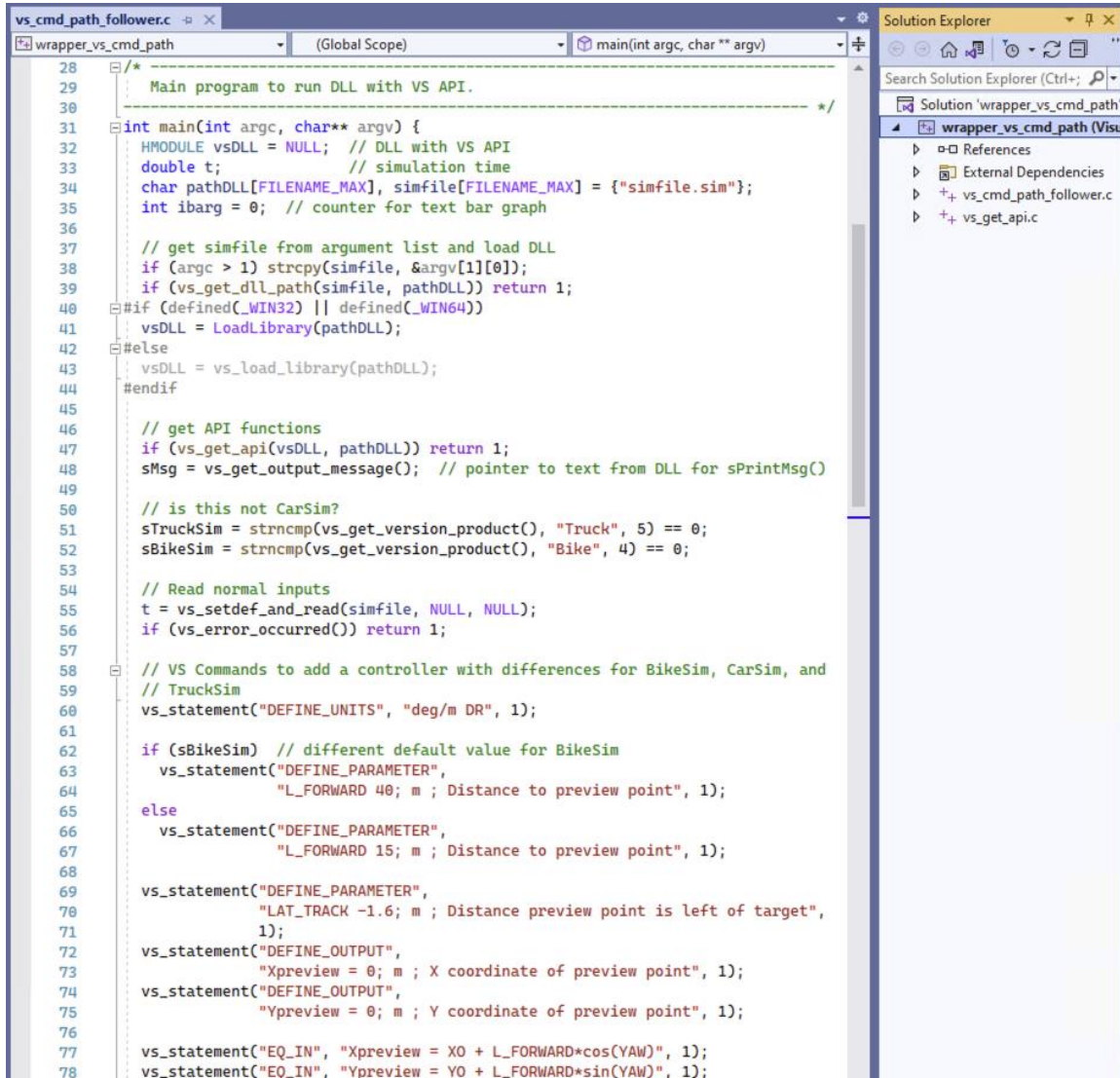
The argument *time* is the current value of time in the simulation.

Apply VS Commands and Statements Programmatically

There are situations where automation provided with an external program might be combined with standard inputs for the VS Math Model. The VS API function `vs_statement` can be used to provide almost any input to the VS Solver or VS Math Model that can be provided from a Parsfile input. The only exceptions are tables of numbers that involve multiple lines of text.

To use `vs_statement`, it is necessary to have access to the VS Math Model at different times in the simulation. For example, new commands might be inserted after the VS Math Model has been defined and has initialized the internal database, but before the differential equations in the math

model are initialized. The tech memo *Extending VS Math Models with VS Commands and the VS API* describes an example C program provided with the SDK that uses `vs_` statement to apply a set of VS Commands to define a path follower using VS Commands (Figure 6). This example works with CarSim, TruckSim, and BikeSim.



```

28  /*
29  Main program to run DLL with VS API.
30  */
31  int main(int argc, char** argv) {
32      HMODULE vsDLL = NULL; // DLL with VS API
33      double t; // simulation time
34      char pathDLL[FILENAME_MAX], simfile[FILENAME_MAX] = {"simfile.sim"};
35      int ibarg = 0; // counter for text bar graph
36
37      // get simfile from argument list and load DLL
38      if (argc > 1) strcpy(simfile, &argv[1][0]);
39      if (vs_get_dll_path(simfile, pathDLL)) return 1;
40      #if defined(_WIN32) || defined(_WIN64)
41      vsDLL = LoadLibrary(pathDLL);
42      #else
43      vsDLL = vs_load_library(pathDLL);
44      #endif
45
46      // get API functions
47      if (vs_get_api(vsDLL, pathDLL)) return 1;
48      sMsg = vs_get_output_message(); // pointer to text from DLL for sPrintMsg()
49
50      // is this not CarSim?
51      sTruckSim = strcmp(vs_get_version_product(), "Truck", 5) == 0;
52      sBikeSim = strcmp(vs_get_version_product(), "Bike", 4) == 0;
53
54      // Read normal inputs
55      t = vs_setdef_and_read(simfile, NULL, NULL);
56      if (vs_error_occurred()) return 1;
57
58      // VS Commands to add a controller with differences for BikeSim, CarSim, and
59      // TruckSim
60      vs_statement("DEFINE_UNITS", "deg/m DR", 1);
61
62      if (sBikeSim) // different default value for BikeSim
63      vs_statement("DEFINE_PARAMETER",
64                  "L_FORWARD 40; m ; Distance to preview point", 1);
65      else
66      vs_statement("DEFINE_PARAMETER",
67                  "L_FORWARD 15; m ; Distance to preview point", 1);
68
69      vs_statement("DEFINE_PARAMETER",
70                  "LAT_TRACK -1.6; m ; Distance preview point is left of target",
71                  1);
72      vs_statement("DEFINE_OUTPUT",
73                  "Xpreview = 0; m ; X coordinate of preview point", 1);
74      vs_statement("DEFINE_OUTPUT",
75                  "Ypreview = 0; m ; Y coordinate of preview point", 1);
76
77      vs_statement("EQ_IN", "Xpreview = XO + L_FORWARD*cos(YAW)", 1);
78      vs_statement("EQ_IN", "Ypreview = YO + L_FORWARD*sin(YAW)", 1);

```

Figure 6. C Wrapper Program that applies VS Commands.

The first few lines in `main` are like those shown earlier (Figure 4, page 16):

- The name of a Simfile is given to the function `vs_get_dll_path` (line 39) to get the pathname for the VS Solver DLL.
- The function `LoadLibrary` uses the DLL pathname to load the DLL (line 41).
- The function `vs_get_api` activates all the VS API functions (line 47).

Rather than using the `vs_run` API function, this example uses `vs_setdef_and_read` to get started (line 55). Before initializing, it applies the API function `vs_statement` to apply VS Commands (lines 60 – 78 show some of them).

This example works with CarSim, TruckSim, and BikeSim, even though the CarSim and TruckSim models are controlled using steer angle, whereas BikeSim is controlled by leaning the bike. In C, two variables are defined to determine whether the model is running under TruckSim (line 51) or BikeSim (line 52). Slightly different versions of the VS Commands are used for the different products (see the if-else statements in lines 62-67).

3. Unit Conversions

A VS Math Model maintains two sets of units:

1. All calculations during a run involve *internal dynamical units* that require no conversions. These are usually SI units; forces are Newtons, lengths are meters, angles are radians, accelerations are m/s^2 , etc.
2. *User units* are accepted for inputs and shown for outputs. Any scaling that is needed to convert to internal dynamical units is done automatically. For example, lengths with units of millimeters are divided by 1000 on input and multiplied by 1000 on output.

Unit conversions (from internal dynamical units to user units) are done automatically for output variables before they are exported or written to file for plotting and animation.

In basic use, with no VS Commands or model extensions, the VS Math Model communicates with the user completely in user units. User units are also used for communicating with external models in Simulink and other simulation environments.

When custom C code is used to access the internal variables, all values are associated with internal dynamical units, including parameters, Import variables, Export, output, and state variables. Therefore, any calculations made in custom C code are normally done only in internal dynamical units.

4. Simulate Using Import and Export Arrays

CarSim, TruckSim, and BikeSim work with Simulink, LabVIEW, and Functional Mockup Interface (FMI) programs by using Import and Export arrays to share information between the VS Math Model and the external modeling workspace (Figure 1, page 11). The internal functions that were used by Mechanical Simulation to create wrapper programs (e.g., the VS Wrapper S-Function for Simulink) are available as API functions to support the same methods when working with custom wrapper programs.

Connections that are more direct are possible when creating a wrapper program in C/C++ (see Chapter 6). However, when running a VS Math Model from a program written in a high-level language that does not fully support C pointers and/or strings (e.g., MATLAB) the model may be extended using the same array-based approach taken with Simulink, LabVIEW, and FMI.

This chapter describes the API functions that support this kind of operation and provides example code for running a VS Math Model from MATLAB.

If you are primarily interested in running a VS Math Model from programs written in C, then you can skip this chapter.

Overview of Method

Running a VS Math Model that is extended with additional equations from a high-level language involves switching back and forth between the wrapper program and the VS Math Model and using two shared arrays to exchange information.

- The *Export* array contains variables that are calculated by the VS Math Model and are made available for use in your wrapper program.
- The *Import* array contains variables that are calculated in your wrapper program and provided to the VS Math Model.

Once the simulation is running, calculations are made at discrete time increments. At each time, the following steps take place within the VS Math Model:

1. Contents of the Import array from the wrapper program are copied into a corresponding array within the VS Math Model and the units are converted by the VS Math Model to internal dynamical units.
2. The VS Math Model calculates derivatives of the state variables at the current time, it calculates output variables, and it copies specified output variables to an internal array and converts the units of each variable in the array to user units for export.
3. Contents of the internal Export array are copied to the Export array provided by the wrapper program.
4. The new values of the derivatives of the state variables of the VS Math Model are integrated numerically to calculate the state variables for the next timestep.

The full sequence has many more steps that consider VS Commands, C functions added with API calls, and VS Events; please see Chapter 9 for the description of the full sequence.

The main concept to appreciate when adding equations and variables from a wrapper program is the sequence in which calculations are made at each timestep. If the variables that are imported to the VS Math Model are based mainly on time and factors outside the scope of the math model in the VS Math Model (e.g., open-loop driver controls, locations of other vehicles, disturbances such as wind, etc.), then synchronization can be perfect. However, when there are interactions between variables in the VS Math Model and the wrapper, there might be a one-step delay because variables calculated in the wrapper for time T are based on values that were exported the previous timestep (at time $T - T_DT$, where T_DT is the time increment between calculations).

When the run starts, T is set to the specified start time $TSTART$. $TSTART$ is typically zero, but advanced users might specify arbitrary $TSTART$ values.

After the VS Solver DLL is loaded, a VS Math Model is created with default values for all internal variables and tables. It then reads one or more Parsfiles to change the vehicle properties, controls, and environmental variables to the runtime values specified by the user. The input files also specify the variables that will be shared using the Import and Export arrays. Until the Parsfiles are read, the dimensions of the Import and Export arrays are not known by the VS Math Model.

After input Parsfiles are read, the VS Math Model calculates all the variables in the model for the time $T = TSTART$. However, at this time, the Import array has not been received from the external program. When operating with external workspaces such as Simulink, advanced users can specify initial values for the Import variables to achieve the correct initial condition of the vehicle. The same method also works for custom wrapper programs that use the Import array.

At the end of the initialization, the VS Math Model calculates all the specified Export variables and copies them to the shared Export array, where they are available to the wrapper program.

Note	The 2023.0 release of CarSim, TruckSim, and BikeSim added support for multiple Simulink S-Function wrappers, as described in the <i>VS Math Models</i> reference manual, <i>VS Commands</i> reference manual, and Tech Memo <i>Extending VS Math Models with VS Commands and the VS API</i> . To do so, the sequence of calculations in the VS Math Models was re-organized, and multiple Import and Export arrays were added to support up to four wrapper programs interacting with the VS Math Model.
	This manual was updated to describe the new sequence of calculation and supporting callback locations (Chapter 7) but does not have details yet about the management of multiple Import and Export arrays.

API Functions Involving Import and Export Arrays

Table 3 lists API functions that are typically used to run the VS Math Model when it will share arrays of Import and Export variables during the run.

Table 3. VS API Functions used to access a VS Math Model using Import and export arrays.

Function	Description
<code>vs_copy_export_vars</code>	Copy Export variables from the VS Math Model to an array.
<code>vs_copy_io</code>	Copy scaled data to and from import and export arrays.
<code>vs_get_tstep</code>	Get timestep covered by a call to <code>vs_integrate_io</code> .
<code>vs_integrate_io</code>	Advance one timestep and transfer i/o data to arrays.
<code>vs_read_configuration</code>	Set defaults, read inputs, initialize, and start files.

`vs_copy_export_vars`

Declaration:

```
void vs_copy_export_vars (vs_real exports[]);
```

This function copies Export variables from the VS Math Model into the array provided as an argument. The variables in the array have already been converted to user units.

This function is applied automatically as needed by `vs_copy_io` and `vs_integrate_io`. It is used in some wrappers that communicate with a VS Math Model between stages in a timestep.

`vs_copy_io`

Declaration:

```
void vs_copy_io (vs_real imports[], vs_real exports[]);
```

Copy Import and Export data using the two arrays provided as arguments. The variables in the import array are scaled as needed to convert from user units to internal dynamical units, and variables for the export array are converted from internal units to user units.

This function is used by wrappers after initialization but before the simulation starts in case the VS Math Model calculates variables for export that are needed when the external software is starting.

Once the simulation is running, the import and export arrays are copied and scaled automatically by `vs_integrate_io`.

`vs_get_tstep`

Declaration:

```
vs_real vs_get_tstep(void);
```

Return:

Timestep seen by external software when interacting with the VS model using any of the `vs_integrate...` functions.

The value returned matches the system parameter that is shown in Echo files with the keyword `T_DT`. Depending on the value of the VS parameters `OPT_INT_METHOD` and `OPT_IO_UPDATE` set by the user at runtime, this is either the value of the parameter `TSTEP` or half of that value. (See the *VS Math Models Manual* for details.)

The internal variable whose value is returned by `vs_get_tstep` is set during initialization, after all inputs are read. The initialization action can be made with the function `vs_read_configuration` or the function `vs_initialize` (see page 19).

`vs_integrate_io`

Declaration:

```
int vs_integrate_io(vs_real t, vs_real imports[], vs_real
exports[]);
```

Return:

Zero if the run should continue; non-zero if the run should stop.

The function `vs_integrate_io` uses two arrays to communicate between the internal math model and the calling program.

The first argument is the current time t . The value of t is managed by the calling program and must be incremented by the timestep (obtained with `vs_read_configuration` or `vs_get_tstep`) before `vs_integrate_io` is called again.

The arrays *imports* and *exports* must be dimensioned to include at least the number of elements that are returned from the function `vs_read_configuration`. When `vs_integrate_io` solves the equations of motion, it starts by copying the values from the *imports* array to make use of all variables that were provided by the calling program. The contents of *imports* array itself are left intact.

At the end of the calculations for this time, the VS Math Model copies new values of exported variables into the *exports* array for use in the calling program. More information about the sequence of calculations is provided in the **Running** subsection on page 93 in Chapter 9.

`vs_read_configuration`

Declaration:

```
void vs_read_configuration (const char *simfile, int *n_import,
int *n_export, vs_real *tstart
vs_real *tstop, *t_dt);
```

This function performs several actions:

1. read the simulation control file *simfile* to get the name of the input Parsfile,
2. read the Parsfile(s) with all parameters for the math model,
3. initialize the math model, and
4. provide values for the number of Import variables (*n_import*), Export variables (*n_export*), start time (*t_start*), stop time (*t_stop*), and timestep (*t_dt*).

The string *simfile* is the name of the Simfile used to start the run, as described in the *VS Math Models Manual*. All other arguments are pointers to variables that are set after the VS Math Model reads the Parsfile identified in the simulation control file. The arguments *n_import* and *n_export*

are pointers to the sizes of the Import and Export arrays that are used to exchange data with an external environment such as Simulink with the function `vs_integrate_io`.

The arguments `tstart` and `tstop` are pointers to the start and stop times specified at runtime. The argument `t_dt` is a pointer to time interval needed between calls to the function `vs_integrate_io`. This is the same as the system parameter shown in Echo files as `T_DT`. Depending on other settings, this is either value of the input parameter `TSTEP` or half of that value.

This function does not work well if the external program will perform actions after the time that the VS Math Model has started and defined built-in variables, but before the VS Math Model calculates initial conditions and sets up for numerical integration. In these conditions, the API functions `vs_setdef_and_read` and `vs_initialize` can be used, as will be shown in the next section for an example in MATLAB. In this case, the timestep is obtained using the `vs_get_tstep` function.

Example: Extend a VS Math Model with MATLAB

To run a VS Math Model from MATLAB, a MATLAB M file will typically use commands to scan a Simfile to get the pathname of the VS Solver DLL, load the DLL, and access the VS API functions using a header file provided with VehicleSim products named `vs_api_def_m.h` (see lines 9 – 25, Figure 5, page 17). At that time, the major API functions are accessible and can be applied using the MATLAB `calllib` function.

The tech memo *Extending VS Math Models with VS Commands and the VS API* describes an example model extension made in MATLAB using Import and Export arrays. The model extension is a path follower that uses the lateral position of a preview point to calculate a steering wheel angle. (The example also works in BikeSim, where it sets a target lean angle in response to lateral error of the preview point.) The following discussion does not go into detail about the extension (see the tech memo for the full description). Instead, it focuses on the overall method for using MATLAB to communicate with a VS Math Model with arrays, to apply modeling from the MATLAB environment to the VS Math Model.

Figure 7 shows part of the example M-File described in the tech memo. The listing is scrolled to start with the `calllib` function (line 35) that applies the `vs_setdef_and_read` API function. After this line, the contents of this example M file differ from the example shown earlier (Figure 5, page 17).

Setting up a model extension involving Import and Export arrays requires that both the VS Solver and the MATLAB code be prepared for the communication. The VS Math Model can be configured using datasets from the input Parsfiles, as is done when working with Simulink. Alternatively, it can be configured from MATLAB with the API function `vs_statement`, as is shown in Figure 7 (lines 38 – 58). An advantage of using `vs_statement` is that the requirements of the VS Math Model and the MATLAB extensions are all contained in the same file (the M file) and can perhaps be coordinated and maintained more reliably.

```

34 % Start and read normal inputs
35 t = calllib('vs_solver', 'vs_setdef_and_read', simfile, 0, 0);
36
37 % activate 3 export variables and 1 import (different for BikeSim)
38 vs_statement('EXPORT', 'XO');
39 vs_statement('EXPORT', 'YO');
40 vs_statement('EXPORT', 'YAW');
41
42 if IsBikeSim
43     vs_statement('IMPORT', 'IMP_GAMMA_TARG REPLACE 0'); % target lean angle
44 else
45     vs_statement('IMPORT', 'IMP_STEER_SW REPLACE 0'); % steering wheel angle
46 end
47
48 % define and activate 2 new import variables
49 vs_statement('DEFINE_IMPORT', 'IMP_X_PREV = 0; m ; Preview point from MATLAB');
50 vs_statement('DEFINE_IMPORT', 'IMP_Y_PREV = 0; m ; Preview point from MATLAB');
51 vs_statement('IMPORT', 'IMP_X_PREV');
52 vs_statement('IMPORT', 'IMP_Y_PREV');
53
54 % define two new output variables for plotting and video
55 vs_statement('DEFINE_OUTPUT', ...
56     'Xpreview = IMP_X_PREV; m ; X coordinate of preview point');
57 vs_statement('DEFINE_OUTPUT', ...
58     'Ypreview = IMP_Y_PREV; m ; Y coordinate of preview point');
59
60 calllib('vs_solver', 'vs_initialize', t, 0, 0);
61 disp(calllib('vs_solver', 'vs_get_output_message'));
62

```

Figure 7. Setting up Import and Export variables for the VS Math Model.

The `vs_statement` command replaces inputs that might otherwise be provided from the input Parsfiles. Within the simulation process, it should be used after the VS Math Model has been started and has defined its internal database with all built-in parameters and output variables (that is, after the `vs_setdef_and_read` step has been taken). These commands must also be applied before the model calculates initial conditions and prepares for the simulation (that is, before the `vs_initialize` step). In this example, the `vs_setdef_and_read` function (line 35) is used to start up the VS Math Model and have it scan all input Parsfiles. After the multiple `vs_statement` commands, the `vs_initialize` function is used in preparation for starting the simulation (line 60).

The VS Commands applied in Figure 7 set up the VS Math Model to export three variables each timestep (e.g., `XO`, `YO`, and `YAW`, lines 38, 39 and 40) and to import three variables each timestep (`IMP_STEER_SW`, `IMP_X_PREV`, and `IMP_Y_PREV`, lines 45, 51, 52 for CarSim and TruckSim). Two new output variables are defined (`Xpreview` and `Ypreview`, lines 55 – 58). A formula for each is simply the symbol for a corresponding import variable (`IMP_X_PREV` and `IMP_Y_PREV`, respectively).

Figure 8 shows the rest of this example M file.

To match the setup of in the VS Math Model for an Import array with three variables and an Export array with three variables, two arrays are defined in MATLAB with three elements each (lines 78 and 79). Arrays are accessed with VS API functions using pointers, so a pointer to each array is defined (lines 80 and 81).

```

63 % Define local parameters for the controller, with values set for CarSim
64 DR = 180/pi; % constant for units conversion: degrees per radians
65 Lfwd = 15; % preview distance
66 GainCtrl = 10; % control gain for steering or lean angle (deg/m)
67 LatTrack = -1.6; % projected lateral position relative to path
68
69 % adjust a few parameter values for BikeSim and TruckSim
70 if IsBikeSim
71     Lfwd = 40.0;
72     GainCtrl = -6;
73 elseif IsTruckSim
74     GainCtrl = 100;
75 end
76
77 % Define import/export arrays (both with length 3) and pointers to them
78 imports = zeros(1, 3);
79 exports = zeros(1, 3);
80 p_imp = libpointer('doublePtr', imports);
81 p_exp = libpointer('doublePtr', exports);
82
83 % get time step and export variables from the initialization
84 t_dt = calllib('vs_solver', 'vs_get_tstep');
85 calllib('vs_solver', 'vs_copy_export_vars', p_exp);
86 stop = calllib('vs_solver', 'vs_error_occurred');
87 disp('The simulation is running...');
88
89 % This is the main simulation loop. Continue as long as stop is 0.
90 while ~stop
91     t = t + t_dt; % increment time
92
93     % Update the array of exports using the pointer p_exp
94     exports = get(p_exp, 'Value');
95     Yaw = exports(3)/DR; % convert from deg to rad
96
97     % calculate values for MATLAB-based controller
98     Xpreview = exports(1) + Lfwd*cos(Yaw);
99     Ypreview = exports(2) + Lfwd*sin(Yaw);
100     road_l = calllib('vs_solver', 'vs_road_l', Xpreview, Ypreview);
101
102     % copy values into import array and set pointer for the VS solver
103     imports(1) = GainCtrl*(LatTrack - road_l);
104     imports(2) = Xpreview;
105     imports(3) = Ypreview;
106     set(p_imp, 'Value', imports); %set pointer for array of imports
107
108     % Call VS API integration function and exchange import and export arrays
109     stop = calllib('vs_solver', 'vs_integrate_io', t, p_imp, p_exp);
110 end
111
112 % Terminate
113 calllib('vs_solver', 'vs_terminate_run', t);
114 disp('Simulation has finished.');
```

Figure 8. The MATLAB side of the interface after the VS Math Model initializes.

The main simulation loop starts with the `While` statement (line 90) and continues to the matching `end` statement (line 110). The VS Math Model updates the math model equations via the function `vs_integrate_io` (line 109) and will continue if the function returns a value of 0 (assigned to the variable `stop`).

Each timestep, two actions are needed in MATLAB to connect the values within the arrays to the pointer variables used to communicate with the VS Math Model.

1. Values from the VS Math Model are copied to MATLAB using the `get` function (line 94).
2. The pointer sent to the VS Math Model for the Import array is updated each timestep using the `set` function (line 106).

Besides the two pointers, the API function `vs_integrate_io` needs the current simulation time `t`. A statement at the top of the loop (line 91) updates `t` using the timestep `t_dt`, which was obtained from the VS Math Model after the initialization using the API function `vs_get_tstep` (line 84).

Each time through the loop, MATLAB needs the values of the three output variables exported by the VS Math Model (94 - 99). Before starting the loop, the pointer to the Export array from the VS Math Model is updated using the API function `vs_copy_export_vars` (line 85). Every other timestep, the pointer is updated by `vs_integrate_io` (line 109).

5. Access Generic Simulation Information

This chapter describes some utility functions that can be useful in getting information from any VS Math Model that uses numerical integration to solve model-specific equations of motion.

Simulation Conditions

Table 4 shows functions that provide information about the simulation that can be used from any language. Some of the functions have already been described in previous chapters; in those cases, the page is noted where the function is described.

Table 4. Utility functions to obtain information about the simulation.

Function	Description
<code>vs_during_event</code>	Determine if the solver has processed any events (used to distinguish startup initialization vs. Event condition).
<code>vs_get_time</code>	Get current time in the solver.
<code>vs_get_tstep</code>	Get timestep between <code>vs_integrate</code> function calls. (See p. 28)
<code>vs_error_occurred</code>	Did an error occur since the last check? (See p. 18)
<code>vs_opt_pause</code>	Determine if the run display should pause after the simulation.
<code>vs_stop_run</code>	Determine if the run should be stopped. (See p. 21)

`vs_during_event`

Declaration:

```
int vs_during_event(void);
```

Return:

0 if no events have occurred in the run, non-zero otherwise.

This can be used for equations that are applied during initialization, to distinguish whether the solver is doing the first initialization (value = 0), or an initialization due to the processing of an Event condition. (Events are a feature of the VS Commands scripting language, described in the *VS Commands Manual*.)

`vs_get_time`

Declaration:

```
vs_real vs_get_time(void);
```

Return:

Current simulation time within the VS Math Model.

vs_opt_pause

Declaration:

```
int vs_opt_pause(void);
```

Return:

Value of the internal integer parameter with the name `OPT_PAUSE`. A non-zero value means the user requested that the display should remain at the end of the run until the user acts to move on.

The internal integer is static; it remains valid after the simulation has terminated and memory has been freed.

Access Messages

Table 5 lists functions that access messages that are available from the VS Math Model.

Table 5. Utility functions involving VS Solver string buffers.

Function	Description
<code>vs_clear_error_message</code>	Clear the internal error message string.
<code>vs_clear_output_message</code>	Clear the internal output message string.
<code>vs_get_error_message</code>	Access text from the internal VS message buffer.
<code>vs_get_output_message</code>	Access text from the internal VS message buffer.
<code>vs_get_version_model</code>	Get a string with more details about the model.
<code>vs_get_version_product</code>	Get a string with the name of the VS product.
<code>vs_get_version_vs</code>	Get a string with the version VS Lisp code generator.
<code>vs_printf</code>	Print formatted string to buffer for later display.
<code>vs_printf_error</code>	Print formatted string to error buffer and set error flag.
<code>vs_printf_message</code>	Print formatted string to buffer and wait for user.

- These functions all work in C/C++ and other languages that can follow C calling conventions.
- Optional arguments in `vs_printf` and `vs_printf_error` are not supported in MATLAB; these functions are declared in MATLAB for use with a single string argument.
- None of these functions should be used from VB programs, because VB and C use different internal conventions for handling strings.

vs_clear_error_message

Declaration:

```
void vs_clear_error_message (void);
```

The VS Solver has an internal string buffer with pending error messages that will be written into the log file and possibly displayed on screen. The buffer is accessed with the function `vs_get_error_message`.

The function `vs_clear_error_message` clears this internal string by setting the first element of the character array to zero. This can be helpful if an external program is running a series of simulations without reloading the DLL.

vs_clear_output_message

Declaration:

```
void vs_clear_output_message (void);
```

The VS Solver has an internal string buffer with pending messages that might appear as a run proceeds. The function `vs_clear_output_message` clears this internal string by setting the first element of the character array to zero. This is typically done after displaying the message.

vs_get_error_message

vs_get_output_message

Declarations:

```
char *vs_get_error_message(void);  
char *vs_get_output_message(void);
```

Return:

String with current message from an internal VS buffer.

Messages generated by the VS Math Model are written to internal text buffers. Informative messages go to the output buffer; error messages go to the error buffer. These functions provide strings (technically, pointers to character arrays) so they can be accessed in the calling program.

Both internal character arrays are static in memory. When a message is cleared, the first element of the array is set to zero. However, the full array remains in memory, even after dynamic memory is released at the end of the simulation with `vs_terminate_run`.

vs_get_version_model

vs_get_version_product

vs_get_version_vs

Declarations:

```
char *vs_get_version_model(void);  
char *vs_get_version_product(void);  
char *vs_get_version_vs(void);
```

Return:

String that identifies a property of the current solver.

The function `vs_get_version_model` is no longer used. In older versions of the VS products, it would specify a particular model within a product.

The function `vs_get_version_product` identifies the product associated with the current solver, e.g., CarSim 2021.1.

The function `vs_get_version_vs` identifies the version of the VS Lisp code generator used to generate the source code for the currently loaded VS Solver. For example:

Generated by VehicleSim Lisp 1.0 on December 18, 2007

vs_printf

Declaration:

```
void vs_printf (const char *format, ...);
```

This function is used to generate formatted printed output. The calling convention is identical to the C function `printf`. However, instead of printing to a default output, it prints to an internal buffer in the DLL. The buffer can be accessed with the VS API function `vs_get_output_message`.

The optional arguments after *format* are not declared in the header file used with MATLAB; in this case, *format* is treated as a simple string.

vs_printf_error

vs_printf_message

Declaration:

```
void vs_printf_error (const char *format, ...);  
void vs_printf_message (const char *format, ...);
```

These functions are used to generate a formatted error message and also to set an internal flag indicating that an error occurred. The calling convention is identical to the C function `printf`. However, instead of printing to a default output, it prints to an internal buffer in the DLL. The buffer can be accessed with the VS API function `vs_get_error_message`.

The status of the internal error flag is obtained with the function `vs_error_occurred` (see Table 3 on page 28).

The optional arguments after *format* are not declared in the header file used with MATLAB; in this case, *format* is treated as a simple string.

The function `vs_printf_message` is similar to `vs_printf_error` except no error flag is set and the message appears as a pop-up screen. The user must click on the screen to resume a simulation.

Pre- and Post- Processing Callback Functions

Table 6 lists functions that set up callback functions that can be executed before the solver runs or after it has completed execution.

The function template for both the pre-processing and post-processing functions is:

```
int user_function(void *userData);
```

Table 7. Utility functions to set up Pre- and Post- Processing callbacks.

Function	Description
<code>vs_pre_proc_function</code>	Sets up function to be called before solver runs.
<code>vs_post_proc_function</code>	Sets up function to be called after solver runs.

The pre-processing function is executed after the `run_all.par` file has been scanned, but before any solver execution has occurred. The post-processing function executes after the solver has completed execution. The optional Message string is used to hold information which is printed to the log file.

`vs_pre_proc_function` **`vs_post_proc_function`**

Declarations:

```
void vs_pre_proc_function (const char *Message,
                          int (*pre_proc)(void *userData), void *userData);
void vs_post_proc_function (const char *Message,
                           int (*post_proc)(void *userData), void *userData);
```

Return:

None.

Get Simulation File Names

Table 8 lists functions that get file names that were specified in the Simfile that was used to set up the run.

Table 8. Utility functions to get simulation file names.

Function	Description of File
<code>vs_get_database_dir</code>	Directory that contains i/o files for VS Math Model.
<code>vs_get_echofile_name</code>	Echo file written by the VS Math Model after initialization.
<code>vs_get_endfile_name</code>	Echo file written by the VS Math Model at termination.
<code>vs_get_erdfilename</code>	ERD or VS header file written by the VS Math Model.
<code>vs_get_filebase_name</code>	Root name for i/o files used in normal Windows operation.
<code>vs_get_infile_name</code>	Input Parsfile that provides all input data for the run.
<code>vs_get_logfile_name</code>	Log file written by the VS Math Model as the run proceeds.
<code>vs_get_simfile_name</code>	Simulation control file that specifies the other file names.

The Simfile name is passed as an argument to `vs_run` (page 15), `vs_read_configuration` (page 29), and `vs_setdef_and_read` (page 20). For more information about the Simfile and the input and output files listed in the table, please see the *VS Math Models Manual*.

```

vs_get_database_dir
vs_get_echofile_name
vs_get_endfile_name
vs_get_erdfile_name
vs_get_filebase_name
vs_get_infile_name
vs_get_logfile_name
vs_get_simfile_name

```

Declarations:

```

char *vs_get_database_dir(void);
char *vs_get_echofile_name(void);
char *vs_get_endfile_name(void);
char *vs_get_erdfile_name(void);
char *vs_get_filebase_name(void);
char *vs_get_infile_name(void);
char *vs_get_logfile_name(void);
char *vs_get_simfile_name(void);

```

Return:

String with pathname, as obtained from the current Simfile.

Write Documentation Files

Several screens in the VS Browser in a VehicleSim product make use of documentation files that are generated by the VS Math Model. These include **Run Control**, **Plot Setup**, **I/O Channels: Export**, **I/O Channels: Import**, and **I/O Channels: Write**. In each case, the VS Browser loads the VS Solver associated with a selected **Run Control** dataset and reads the associated **Run Control** Parsfile, along with all linked Parsfiles, and writes up to seven types of files with information about the VS Math Model as set up with the linked data (Table 9).

Table 9. Documentation files that can be written by a VS Math Model.

File Type	File Suffix	Description
State variables in math model	_svs.txt	Text file listing all state variables for the model as configured, including those added with VS Commands
Imports into math model (text)	_imp.txt	File listing all Import variables that exist in the model as configured, including those added with VS Commands
Imports into math model (tabbed text)	_imp_tab.txt	
Imports into math model (Excel)	_imp.xls	
Outputs from math model (text)	_out.txt	File listing all output variables that exist in the model as configured, including those added with VS Commands
Outputs from math model (tabbed text)	_out_tab.txt	
Outputs from math model (Excel)	_out.xls	

The files are all generated using the function `vs_write_doc_files`.

vs_write_doc_files

Declaration:

```
void vs_write_doc_files (const char *simfile, const char *root,  
                        int model, int import, int output);
```

This function provides information about the VS Math Model as configured by the input Parsfiles identified with the specified Simfile *simfile*. Any output files have the same *root* name, with the suffix specified in Table 9. For example, if *root* is `file_123_456`, then the text file listing all state variables is `file_123_456_svs.txt`.

The VS Math Model can provide documentation files for three kinds of information: the state variables in the model, the available Import variables, and the available output variables. In all three cases, the lists of variables include those added with VS Commands in the input Parsfiles.

The valid options for *model* are 0 and not zero. The value indicates whether the VS Math Model should write a text file describing all the state variables: 0 means no, anything else means yes.

The valid options for *import* are integers between 0 and 7, inclusive. The code for writing a simple text file is 1, the code for writing a spreadsheet file is 2, and the code for writing a tabbed text file is 4. *Import* should be the sum of the three codes. For example, 0 means no file is written with import information, 1 means write only a simple text file, 1+2 means write a text file plus a spreadsheet file, etc.

The valid options for *output* are integers between 0 and 7, inclusive. As with the *import* argument, the code for writing a simple text file is 1, the code for writing a spreadsheet file is 2, the code for writing a tabbed text file is 4, and the value of *output* should be the sum of the three codes.

6. Extend a Math Model with C Callbacks

Methods for extending VS Math Models used in high-level languages such as MATLAB can be also used in C. However, when running a VS Math Model from a wrapper program written in C/C++, several options are available that are not possible with MATLAB, because the wrapper can use C pointers to internal variables and functions:

1. The wrapper program can directly access and possibly change any variable within the VS Math Model that has an identifying keyword. (Import and Export arrays are not needed.)
2. The wrapper program can install I/O support for new variables that exist in the wrapper, but which may be handled by the VS Math Model as parameters, output variables, state variables, etc.
3. The wrapper program can provide callback functions that are applied automatically by the VS Math Model at specified times within the simulation process. The callback functions can in turn access existing variables, add new ones, perform calculations, etc.

A *callback function* is an externally defined function that is made available to the VS Math Model program to enable the execution of externally defined procedures at specific points in the simulation. For example, one of the functions is applied whenever the equations of motion for the math model are updated. It is invoked before the built-in equations are processed, to provide forces or controls that might affect the internal calculations. It is also invoked after the built-in equations are processed, to provide new output variables that might depend on built-in variables that were just updated.

Table 10 lists types of callback functions that can be provided for use within the VS Math Model. It also lists the related installer functions. These allow the VS Math Model to set up additional variables and perform calculations using compiled functions from external code.

Table 10. External callback functions and installers.

Callback Type	Installer Functions	Purpose
calc	vs_install_calc_function vs_install_calc_function2	Perform calculations as needed to extend the model
echo	vs_install_echo_function vs_install_echo_function2	Write text into the output echo file
free	vs_install_free_function vs_install_free_function2	Free memory and clean up if necessary at the end of a run
scan	vs_install_scan_function vs_install_scan_function2	Scan text read from an input Parsfile
setdef	vs_install_setdef_function vs_install_setdef_function2	Set up new variables and define links to the solver program
status	vs_install_status_function	Report status of long-running operations and allow user to cancel them.
log	vs_install_vlog_function	Report log messages.

The name of the callback function is specified when using an installer function and can be any legal function name.

The installer functions listed in Table 10 support multiple callback functions as might be needed to provide several model extensions. For example, one set of callback functions might be installed to support an alternative driver model, and another set might be installed to install alternative models of moving objects and sensors.

An example C program is described in the tech memo *Extending VS Math Models with VS Commands and the VS API*. The same example will be used in this chapter to illustrate some callback functions; Figure 9 shows the source code for a wrapper that installs callbacks for the types `setdef`, `echo`, and `calc` (lines 38, 39, 40).

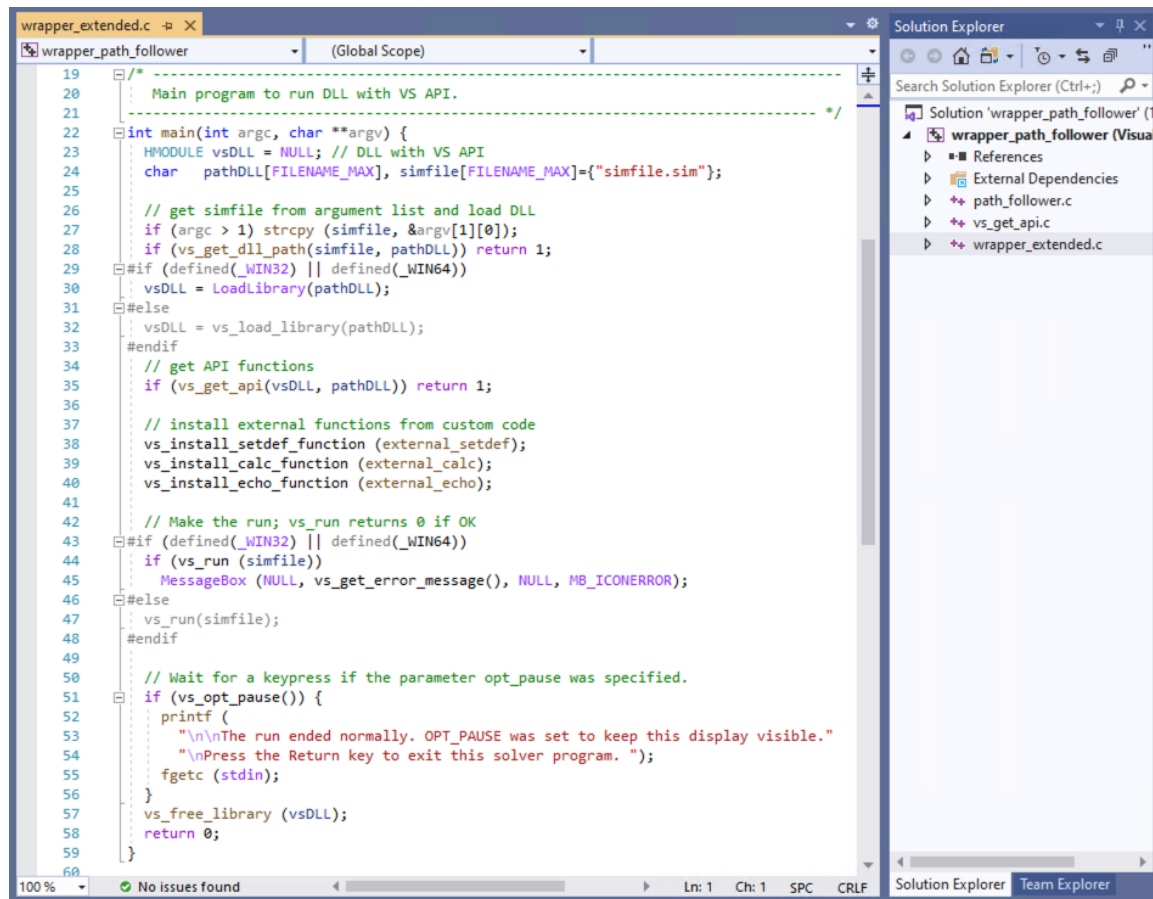


Figure 9. C wrapper that installs callback functions to extend the model.

The following descriptions include references to equivalent VS Commands, which are in turn documented in the *VS Commands Manual*.

Two of the callback functions are applied from several places in the solver (these are `calc` and `echo`). They each have an argument *where* that specifies the location. The location is identified with an enumeration constant of type `vs_ext_loc` that is defined in the header file `vs_deftypes.h` along with constants that are described later in the relevant function descriptions.

calc callback

vs_install_calc_function

Declarations:

```
void calc (vs_real t, vs_ext_loc where);

void vs_install_calc_function (void (*calc) (vs_real time,
                                             vs_ext_loc where));
```

Perform calculations involving the model extensions. The function is called from twelve places as indicated with the argument *where*, which can have the following values (these are defined in `vs_deftypes.h`).

VS_EXT_AFTER_READ: Perform calculations after all the data have been read from Parsfiles at the start of a run, but before state variables, output variables, and import variables have been setup and locked. This is where calls to the `vs_statement` function are typically made to override built-in parameter values, add new state variables, and add VS Commands programmatically.

VS_EXT_EQ_PRE_INIT: Perform calculations after all the data have been read from Parsfiles at the start of a run and the number derivatives is locked, but before the internal initialization is performed. Calculations made in this location are similar in concept to the calculations done with equations added with the `EQ_PRE_INIT` VS Command.

VS_EXT_EQ_INIT: Perform calculations after the internal initialization is done, but before the native calculations are made of derivatives and output variables. These calculations are also made when processing an Event, after the internal initialization is made, but before the run continues. Calculations made in this location are similar in concept to the calculations done with equations added with the `EQ_INIT` VS Command.

VS_EXT_EQ_INIT2: Perform calculations after the internal initialization is done, including calculations of all derivatives and output variables. These calculations are also made when processing an Event, just before the run continues. Calculations made in this location are similar in concept to the calculations done with equations added with the `EQ_INIT2` VS Command.

VS_EXT_EQ_STATE: Perform calculations at each timestep during a run at the start of the **State** stage. When this callback occurs, State variables for the model have values, but any variables calculated from those state variables still have their values from the previous time step. After the callback and equations added with the VS Command `EQ_STATE`, the VS Math Model calculates variables related to the environment of the vehicle model, such as locations and speeds of moving object, detections from built-in ADAS sensors, and some system-level variables used by the driver model.

VS_EXT_EQ_IN: Perform calculations at the start of the **Control** stage of the model. This is like the calculations done with equations added with the `EQ_IN` VS Command. When this callback occurs, all built-in state information is known, but built-in driver model controls have not yet been calculated.

VS_EXT_EQ_KIN: Perform calculations at the start of the **Kinematics** stage of the model. This is like the calculations done with equations added with the `EQ_KIN` VS Command. It

is useful for calculated variables with units of length, angle, and speed (translational and rotation). This is where built-in kinematical calculations are made for suspension, tire, and steering inputs to models that generate forces and moments. When this callback occurs, controls for the vehicle are known, but built-in kinematical equations have not yet been made.

VS_EXT_EQ_DYN: Perform calculations at the start of the **Dynamics** stage of the model. This is like the calculations done with equations added with the `EQ_DYN` VS Command. It is useful for adding forces and moments based on kinematical variables (e.g., spring force as a function of calculated jounce). When this callback occurs, kinematical variables for the vehicle model are known, but forces, moments, and accelerations have not yet been calculated.

VS_EXT_EQ_OUT: Perform calculations at the just after the **Dynamics** stage of the built-in model calculations. This is like the calculations done with equations added with the `EQ_OUT` and `EQ_DIFFERENTIAL` VS Commands. Because all the native variables have been updated, this is useful for calculating new output variables that depend on other variables in the model.

VS_EXT_EQ_SAVE: Save values for use at the next timestep. This is done after all the calculations described above have been applied, along with calculations added with VS Commands `EQ_OUT` and `EQ_DIFFERENTIAL`. Variables saved in this way can be compared with current values, e.g., to determine reversals. This is like the equations added with the `EQ_SAVE` VS Command.

As with the equations added with `EQ_SAVE`, the external function is not applied at the last timestep of the simulation. This way, conditions recorded in the End Parsfile written at the end of the simulation have values based on the next-to-last timestep, as is typically intended.

VS_EXT_EQ_FULL_STEP: Perform calculations at the end of each full timestep.

Equations in this group are applied only at the full timestep; they are not used at the half step. They can be used to reset state variables in the VS Math Model that are calculated by numerical integration of differential equations.

The function is applied after all externally defined equations are updated, except for those added with the VS Command `EQ_FULL_STEP`. Therefore, calculations here can make use of updated values of nearly all variables added at runtime.

VS_EXT_EQ_END: Perform calculations at the end of a run, after the simulation has stopped but before generating the echo file at the end. Calculations made in this location are similar in concept to the calculations done with equations added with the `EQ_END` command.

Values for all variables should be calculated with internal dynamical units.

Chapter 9 includes several figures showing the calculation timeline.

Figure 10 shows the source code for `external_calc`, a `calc` callback function that calculates CarSim/TruckSim steering wheel angle or BikeSim lean target for the example program. Although `where` has 12 valid values, only two are used for this example:

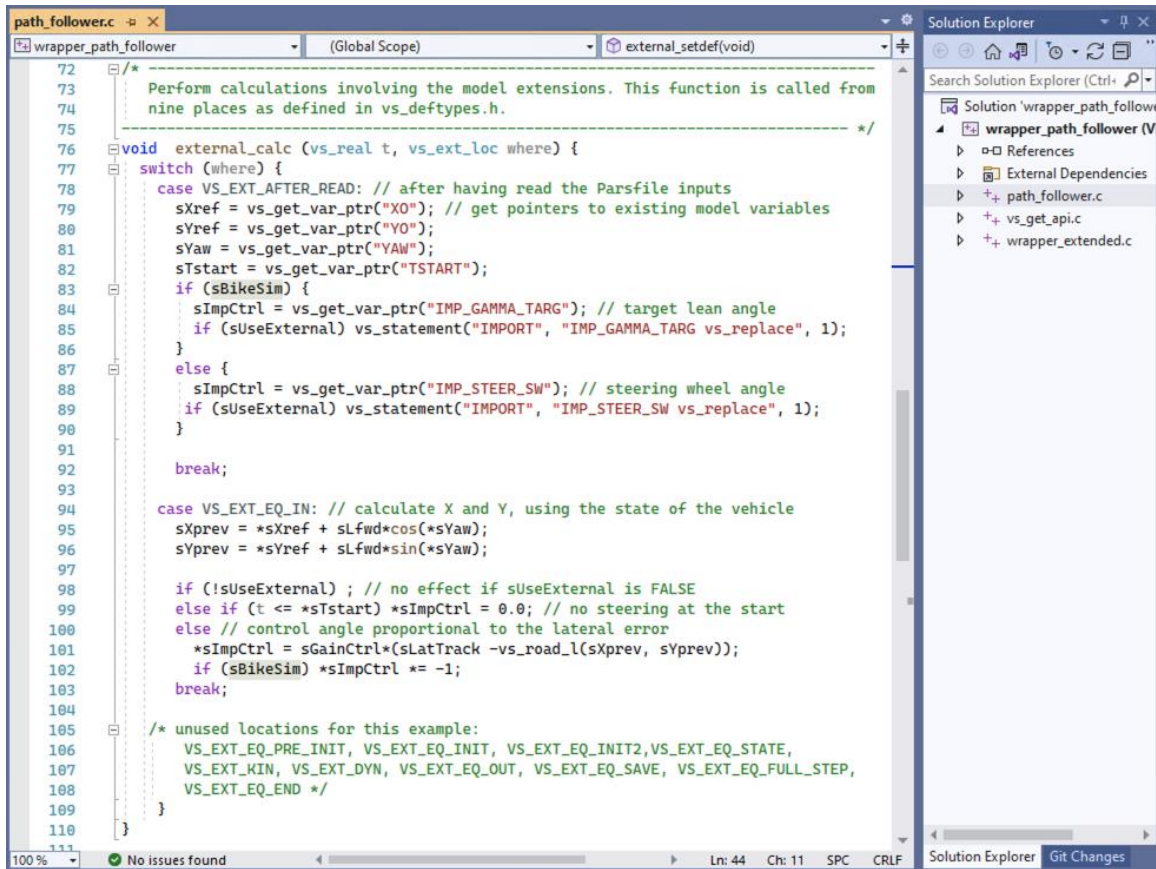


Figure 10. Example calc callback function.

1. VS_EXT_AFTER_READ (after the input Parsfiles have been read but before the initialization is performed), and
2. VS_EXT_EQ_IN (the beginning of each timestep, when calculated values will affect other variables in the model), and

Because the existing Import variable IMP_STEER_SW (CarSim/TruckSim) or IMP_GAMMA_TARG (BikeSim) is used for this example, it is automatically activated with the IMPORT VS Command after all Parsfiles have been read (VS_EXT_AFTER_READ) if a new parameter sUseExternal is not zero (lines 85 and 89).

Given that this extension involves vehicle control, the calculations are made in the Control stage with at the VS_EXT_EQ_IN location (line 94). The two coordinates of the preview point are calculated (VS_EXT_EQ_OUT, lines 95 and 96). The control angle (*sImpCtrl) is then calculated if sUseExternal is not zero (line 101, and if this is BikeSim, also line 102).

calc2 callback

vs_install_calc_function2

Declarations:

```
void calc2 (vs_real t, vs_ext_loc where, void *userData);
```

```
void vs_install_calc_function2 (
```

```
void (*calc2) (vs_real time, vs_ext_loc where, void *userData),
void *userData);
```

These functions are equivalent to `calc` and `vs_install_calc_function`, each with an additional parameter. The value that you pass in the `userData` parameter of the “install” function will be passed back to the callback function. This value is not used or modified in any way by the solver. See `vs_install_calc_function` for more information.

echo callback

vs_install_echo_function

Declarations:

```
void echo (vs_ext_loc where);

void vs_install_echo_function (void (*echo) (vs_ext_loc where));
```

Write information into the current output echo file using the function `vs_write_to_echo_file` (see Table 14Table 11 on page 76). This function is called four times when generating the echo file as indicated with the argument *where*, which can have the following values (defined in `vs_deftypes.h`).

`VS_EXTERNAL_ECHO_TOP`: write information at the top of the echo file. For example, describe the extensions made to the model.

`VS_EXTERNAL_ECHO_SYPARS`: write information at the end of the section of system parameters. For example, indicate any model flags or options here that enable or disable model features.

`VS_EXTERNAL_ECHO_PARS`: write information at the end of the section listing model parameters.

`VS_EXTERNAL_ECHO_END`: write information at the end of the echo file.

This function is used at the end of initialization and at termination.

Note	An echo callback is not necessary for many custom extensions. Variables added using the VS API functions <code>vs_define_imp_where</code> , <code>vs_define_out_where</code> , <code>vs_define_par</code> , and others (see Table 11, page 53) are echoed the same as variables that are native in the VS Math Models. The function <code>external_echo</code> is provided to support extensions that involve different kinds of data formats, e.g., for third-party tire models.
-------------	---

Figure 11 shows the source code for `external_echo`, an echo callback function that extends the information written into an echo file for the example. The code in this example uses just one of the locations: additional text is written at the top of the echo file to tell users that the run was made with an extended model (lines 124 – 126).

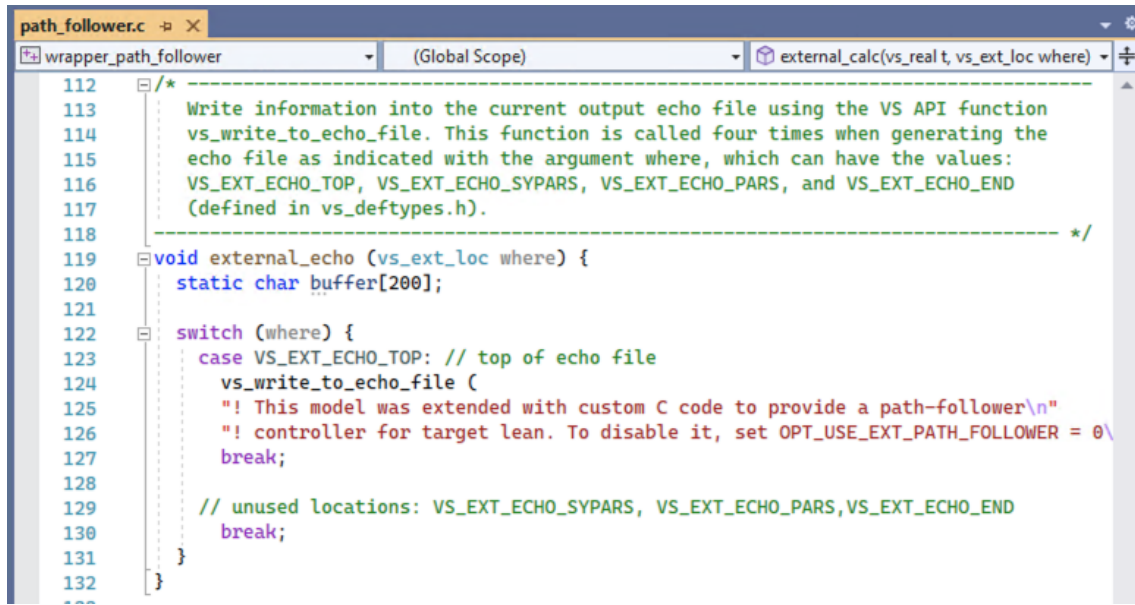


Figure 11. Example echo callback function.

echo2 callback vs_install_echo_function2

Declarations:

```

void echo2 (vs_ext_loc where, void *userData);

void vs_install_echo_function2 (
    void (*echo2) (vs_ext_loc where, void *userData),
    void *userData );

```

These functions are equivalent to echo and vs_install_echo_function, each with an additional parameter. The value that you pass in the *userData* parameter of the “install” function will be passed back to the callback function. This value is not used or modified in any way by the solver. See vs_install_echo_function for more information.

free_ext callback vs_install_free_function

Declarations:

```

void free_ext (void);

void vs_install_free_function (void (*free_ext) (void));

```

This type of function can be provided in case the external code allocates memory or performs initializations that require cleanup at the end of the run.

free_ext2 callback vs_install_free_function2

Declarations:

```
void free_ext2 (void *userData);

void vs_install_free_function2 (
    void (*free_ext2) (void *userData), void *userData);
```

These functions are equivalent to `free_ext` and `vs_install_free_function`, each with an additional parameter. The value that you pass in the *userData* parameter of the “install” function will be passed back to the callback function. This value is not used or modified in any way by the solver. See `vs_install_free_function` for more information.

scan callback **vs_install_scan_function**

Declarations:

```
vs_bool scan(char *keyword, char *buffer);

void vs_install_scan_function (
    void (*scan) (char *keyword, char *buffer));
```

Return:

The `scan` callback function should return 0 if *keyword* was not recognized. A non-zero return value indicates to the solver that the keyword was recognized.

Scan a line read from an input text Parsfile, and possibly continue reading from the currently open file. Each line in the Parsfile is scanned for a beginning keyword. If the keyword is recognized in the native VS Math Model, then the line is processed and that line of text is never seen by `scan`. However, if the keyword is not recognized, then it is passed to `scan` for possible processing. The first part of the line of text that potentially defines a keyword is converted to ALL CAPS and passed to `scan` as the string *keyword*. The rest of the line is passed as the string *buffer*. (Any tabs or spaces between the keyword and the next character in the line of text are removed.)

This function is not necessary for all custom extensions. Variables added using the VS API functions `vs_define_imp_where`, `vs_define_out_where`, `vs_define_par`, and others are added to the internal keyword database of the VS Math Model and are handled automatically. However, this function is useful for specifying information for variables that were not added to the model using the `vs_define_` functions.

If you need to read more from the currently open Parsfile, use the VS API function `vs_read_next_line`, described in Chapter 7.

scan2 callback **vs_install_scan_function2**

Declarations:

```
vs_bool scan2(char *keyword, char *buffer, void *userData);

void vs_install_scan_function2 (
    void (*scan2) (char *keyword, char *buffer, void *userData),
    void *userData);
```


These functions are equivalent to `scan` and `vs_install_scan_function`, each with an additional parameter. The value that you pass in the *userData* parameter of the “install” function will be passed back to the callback function. This value is not used or modified in any way by the solver. See `vs_install_scan_function` for more information.

setdef callback **vs_install_setdef_function**

Declarations:

```
void setdef (void);

void vs_install_setdef_function (void (*setdef ) (void));
```

The `setdef` callback function is critical for almost any form of custom extension to a VS model. This is where you add variables to the existing math model and obtain pointers to most of the variables in the model by using VS API functions described in Chapter 6. This is where you set default values of any internal parameters, and variables that are used in the external custom code.

This function is called before any data files are read at the start of a run.

The initial values for all variables should be made using internal dynamical units.

Not all internal variables can be accessed in the `setdef` callback. Variables associated with optional vehicle components (e.g., powertrain) are set up in a dynamic fashion after the `setdef` phase. These variables can be accessed after the `vs_setdef_and_read` VS API function is explicitly called.

Figure 13 (page 59) shows the source code for `external_setdef`, an example `setdef` callback function. (The example code is in the next chapter because it illustrates the use of some of the main API functions described in that chapter for interacting with the math model.)

setdef2 callback **vs_install_setdef_function2**

Declarations:

```
void setdef2 (void *userData);

void vs_install_setdef_function2 (
    void (*setdef2) (void *userData), *userData);
```

These functions are equivalent to `setdef` and `vs_install_setdef_function`, each with an additional parameter. The value which you pass in the *userData* parameter of the “install” function will be passed back to the callback function. This value is not used or modified in any way by the solver. See `vs_install_setdef_function` for more information.

status callback **vs_install_status_function**

Declarations:

```
int status (vs_ext_loc where, int statusCode,
            vs_real statusValue, void* userData);
```

```
void vs_install_status_function (
    int (*status) (vs_ext_loc where, int statusCode,
        void* userData),
    void* userData)
```

Callback Function arguments:

where – Location from which the callback is being called.

statusCode – Reserved for future use.

statusValue – Reserved for future use.

userData – Value supplied when callback was registered.

Install Function arguments:

status – Pointer to the external callback function.

userData – Value to be passed back when callback is called.

The status callback function is called from within the solver during some long-running tasks. This is mainly used for lengthy initializations.

The callback function should return zero (0) to indicate that the Run should continue, and return a non-zero value to request that the simulation be terminated.

The value which you pass in the *userData* parameter of the `install` function will be passed back to the callback function whenever it is called. This value is not used or modified in any way by the solver.

log callback

vs_install_vlog_function

Declarations:

```
int log (int logLevel, const char *format,
    va_list vlist, void* userData);

void vs_install_vlog_function (int (*log) (int logLevel,
    const char *format, va_list vlist, void* userData),
    void* userData);
```

Callback Function arguments:

logLevel – The log level of this log message.

format – The format string. See the C function `vprintf` for details.

vlist – The argument list for the format string. See the C function `vprintf` for details.

userData – Value supplied when callback was registered.

Install Function arguments:

log – Pointer to the external callback function.

userData – Value to be passed back when callback is called.

The log callback function is called from within the solver whenever a log entry is generated.

The callback function should return the number of characters that were output.

The value which you pass in the *userData* parameter of the `install` function will be passed back to the callback function whenever it is called. This value is not used or modified in any way by the solver.

7. Access VS Math Model Details

VS Math Models can be extended with custom code in C and other programming languages. The VS API provides functions to support the same capabilities as VS Commands, with the additional capability and efficiency of a full programming language. The API also gives access to the main arrays in the math model, allowing the state of the model to be saved and restored in support of advanced applications such as optimizations.

API functions also exist for the path and road models used in the CarSim, TruckSim, and BikeSim vehicle models. Documentation for these functions has been moved from this reference manual to the *Paths and Road Surfaces* document and the header file `vs_api_road.h`.

The VS Symbolic Database

Each VS Math Model includes an internal database that relates keywords to the parameters, variables, tables, and function names that are part of the model. The VS convention is that when using a VS API function to access a variable, you must always provide the keyword as a string with all letters in uppercase. When reading from a Parsfile, names are always converted to uppercase automatically.

Items in the internal database are organized into several internal arrays. One array is for system parameters, another for model parameters, another for Import variables, another for output variables, etc. The VS API includes a few low-level functions that can access and set symbolic properties such as names and units. These functions identify the symbolic entity with two integers: an ID number and an attribute type that identifies the array and property of the symbolic structure. The ID has a value between 1 and the number of symbols installed for the given type (parameters, state variables, outputs, etc.). The attribute types are defined in the header file `vs_deftypes.h`. This header file also defines a generic floating-point data type `vs_real` that is set to `double` and Boolean data type `vs_bool`.

Access and Extend the VS Math Model

Table 11 lists functions in the VS API that can be used to access and extend the math model. These can involve variables and algorithms defined in external code as part of a calling program or the callback functions listed in Table 10 on page 41.

Accessing the variables in the internal database is fully supported when programming in ANSI C. Support for other languages is limited if the other language does not support C pointers.

Some of these functions can be applied with VS Commands that are documented in the *VS Commands Manual*. In these cases, the corresponding VS Commands are noted.

Example source code for applying the API is provided in ANSI C and other languages in the tech memo, *Extending VS Math Models with VS Commands and the VS API*. Some of the source code from this example was shown in is shown in the previous chapter to illustrate the use of the callback functions, and more is shown in this chapter to show some of the functions from Table 11.

Table 11. Functions used to access and extend the math model.

Function	Description
vs_check_import	Boolean: is a specified keyword an import?
vs_define_imp_where	Define a new Import variable.
vs_define_ipar_1d vs_define_ipar_2d vs_define_ipar_3d vs_define_ipar_4d vs_define_ipar_5d	Add internal structure for indexed parameter array and set its properties
vs_define_out_where	Define a new output variable.
vs_define_par	Define a new model parameter.
vs_define_sypar	Define a new system parameter.
vs_define_sv	Define a new state variable.
vs_define_sv_ode	Define a new state variable that has a derivative.
vs_define_units	Install new units.
vs_get_numeric_value	Get the numerical value of a symbolic expression.
vs_get_int_value	Get the integer value of a symbolic expression.
vs_get_sym_attribute	Get an attribute of an internal symbolic structure.
vs_get_var_id	Get the ID for an internal symbolic array element.
vs_get_var_ptr	Get the pointer to a variable (real) in the solver.
vs_get_var_ptr2	Get the pointer to a variable (real) in the solver.
vs_get_var_ptr_int	Get the pointer to a variable (int) in the solver.
vs_get_var_ptr_int2	Get the pointer to a variable (int) in the solver.
vs_get_var_output_gain	Get the gain applied when this variable is output.
vs_have_keyword_in_database	Boolean: is a specified keyword in the database?
vs_get_import_id	Get an Import ID from its name.
vs_get_import_count	Get the total number of available Imports.
vs_get_active_import_count	Get the number of Imports that are currently active.
vs_get_import_active_index vs_get_active_import_id	Convert between Import ID and Active Import Index.
vs_get_import_keyword vs_get_active_import_keyword	Get the keyword (name) of the specified Import.
vs_import_type vs_get_active_import_mode	Get the mode (type) of Import, if active.
vs_get_import_units vs_get_active_import_units	Get the units of the specified Import
vs_get_import_value vs_get_active_import_value	Get the current value of the specified Import.

Table 11. Functions used to access and extend the math model.

Function	Description
<code>vs_get_import_scale</code>	Get the scale factor of the specified Import's units.
<code>vs_import_result</code>	Get the value of an Import combined with another variable.
<code>vs_get_active_import_result</code>	
<code>vs_install_import_callback</code>	Add a callback function for an Import.
<code>vs_remove_import_callback</code>	Remove a callback function for an Import.
<code>vs_install_symbolic_func</code>	Install a new function for use in symbolic expressions.
<code>vs_install_keyword_alias</code>	Install an alias for an existing keyword.
<code>vs_put_on_free_list</code>	Put pointer on list of objects automatically freed.
<code>vs_set_stop_run</code>	Set a condition for stopping the run with a message.
<code>vs_set_sym_attribute</code>	Set an attribute of an internal symbolic structure.
<code>vs_set_sym_int</code>	Set an integer attribute of an internal symbolic structure.
<code>vs_string_copy_internal</code>	Copy a string using internal VS memory management.

Keywords must be static, global, or literal strings in uppercase, such that pointers to the keywords stored in the internal database will remain valid. Once a keyword is defined with a VS API function, it must be treated as a constant. If it is later changed, the VS symbolic database might not work.

Notes These functions are used in thousands of lines of machine-generated code from VS Lisp, with the keywords and descriptions being provided with literal strings. (VS Lisp is the symbolic multibody code generator used at Mechanical Simulation to create source code for the VS Solvers.) The requirement that strings be static helps provide the high efficiency associated with VS Math Models.

The function `vs_string_copy_internal` is provided to create static strings that will persist until memory is cleared with `vs_terminate` or `vs_terminate_run`.

When you add program variables (parameters, state variables, import variables, etc.), the variables must be defined as static or global variables, to assure that pointers remain valid throughout the duration of the simulation. For example, see the beginning of the file `path_follower.c` from the example program (Figure 12). Along with the required `#include` statements, it defines some static variables that are shared by callback functions whose source code is shown in other figures.

Figure 13 (page 59) shows the next section of the same source code file. This part has code for `external_setdef`, a `setdef` callback function that defines new variables and links to the variables for the steering controller example. It shows how some of the major functions listed in Table 11 are used. (Details about this example are provided in the tech memo *Extending VS Math Models with VS Commands and the VS API*.)

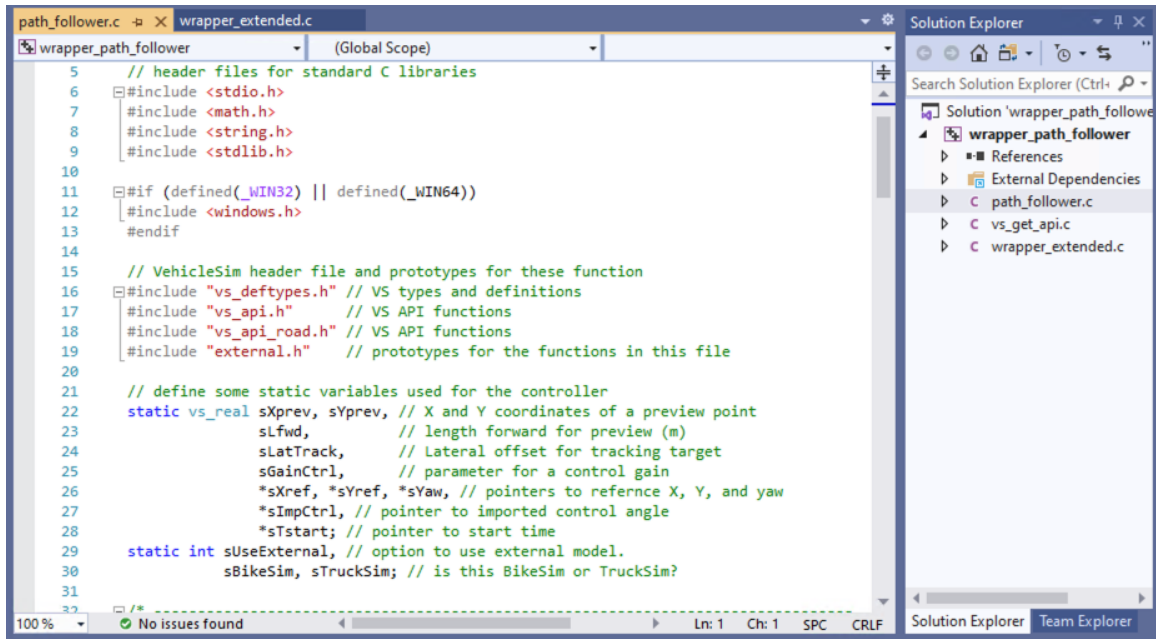


Figure 12. Required #include statements and some shared static variables.

vs_check_import

Declaration:

```
int vs_check_import(char *keyword, int stopError);
```

Return:

1 if *keyword* is an import in the database; 0 if it is not.

This function is used to determine whether a keyword is in the VS Math Model database and is an import.

The string *keyword* is case sensitive and must be ALL CAPS.

If *stopError* equals 1, then an error message is printed, and the execution stops if the keyword is not an import. If *stopError* is 0, then no error message is printed, and execution continues.

vs_define_imp_where

Declaration:

```
int vs_define_imp_where(char *keyword, char *units, int native,
    char *component, char *desc, vs_stage where);
```

Return:

ID of the new Import if successful; it returns -2 and generates an error if the keyword is already associated with an existing variable. ID numbers start at one and increase with each Import. Thus, the return value is also the total number of Imports that are now installed.

This function adds an Import variable to the math model. It is intended for use in the calling program or in a *setdef* callback function.

The argument *keyword* is a string in ALL CAPS that is used to identify the Import variable in subsequent API function calls and in regular input files for the VS Math Model.

All strings must remain constant from the time this function is invoked until the completion of the run.

The argument *units* should be the keyword (ALL CAPS) for the units for the new variable.

The argument *native* is a flag that be zero if there is no native variable automatically linked to the import (needed to support options to replace a native variable or add or multiply the import with the native variable). A non-zero value means there is a native variable. Documentation files generated with the function `vs_write_doc_files` include a suffix “[V]” for Imports created with this flag non-zero. The general rule-of-thumb is that if the `vs_import_result` function is used with the second argument being 0, then a zero value should be given for *native*; if a calculated value (a variable or an expression) is used for the second argument in applications of the `vs_import_result` function, then *native* should be set to 1.

The argument *component* is a string used to describe a component in the model associated with the variable. It is written into documentation files generated with the function `vs_write_doc_files`.

The argument *desc* is a string used to describe the Import variable in echo files generated by the solver and in documentation files.

The argument *where* is a `vs_stage` (integer) that indicates the stage in a timestep where the import variable is used. The valid stage values and enum macros are 1 (State, `BLK_STATE`), 2 (Control `BLK_CTL`), 3 (Kinematics `BLK_KIN`), and 4 (Dynamics `BLK_DYN`). If the stage is not known, set the value to 0.

`vs_define_ipar_1d`
`vs_define_ipar_2d`
`vs_define_ipar_3d`
`vs_define_ipar_4d`
`vs_define_ipar_5d`

Declarations:

```
int vs_define_ipar_1d(char *key, int n, char *index);

int vs_define_ipar_2d(char *key, int n1, char *index1,
    int n2, char *index2);

int vs_define_ipar_3d(char *key, int n1, char *index1,
    int n2, char *index2, int n3, char *index3);

int vs_define_ipar_4d(char *key, int n1, char *index1,
    int n2, *index2, int n3, char *index3,
    int n4, char *index4);

int vs_define_ipar_5d(char *key, int n1, char *index1,
    int n2, char *index2, int n3, char *index3,
    int n4, char *index4, int n5, char *index5);
```


Return:

The integer value returned by any of these functions is an ID needed for other VS functions such as `vs_set_sym_attribute` and `vs_set_sym_int`.

VS Math Models have indexed parameters that appear in Echo files as array elements, e.g., the spin inertia of the right-rear tire in a CarSim model has the keyword `IT(2,2)`. The VS Math Model also identifies the same parameter with the keyword `IT`, with context provided by two systems parameters `IAXLE` and `ISIDE`. If `IAXLE = 2` and `ISIDE = 2`, then `IT` is interpreted the same as `IT(2,2)`. If `ISIDE` is instead 1, then `IT` is interpreted the same as `IT(2,1)`.

The behavior is installed using the `vs_define_ipar_nd` functions.

For the `IT` example, the behavior is installed with a function call:

```
vs_define_ipar_2d ("IT", 2, "IAXLE", 2, "ISIDE");
```

This function is intended for use in the calling program or in a `setdef` callback function.

The argument *key* is the root keyword (e.g., `"IT"`). Each integer *n* argument is a dimension, followed by the keyword associated with a system index parameter. For the example, there are two axes and two sides. If the similar behavior were specified for a 3-axle TruckSim single-unit vehicle with dual tires, the function call would be:

```
vs_define_ipar_3d ("IT", 3, "IAXLE", 2, "ISIDE", 2, "ITIRE");
```

As another example, the function call for a CarSim vehicle with a 1-axle trailer would be:

```
vs_define_ipar_3d ("IT", 2, "IUNIT", 2, "IAXLE", 2, "ISIDE");
```

`vs_define_out_where`

Declaration:

```
int vs_define_out_where (char *shortname, char *ername,  
                        char *longname, vs_real *real, char *units, char *genname,  
                        char *bodyname, char *component, vs_stage where);
```

Return:

This function returns the ID of the new output if successful; it returns -2 and generates an error if the keyword is already associated with an existing variable. ID numbers start at one and increase with each variable added. Thus, the return value is also the total number of outputs that are now installed.

This function adds an output variable to the math model. It is intended for use in the calling program or in a `setdef` callback function.

All strings must remain constant from the time the function is invoked until the completion of the run.

The string *shortname* is copied internally and converted to uppercase to define four keywords for the new variable. (The four keywords are made by using the root keyword, plus three keywords made with the prefixes `ANI_`, `EXP_`, and `WRT_`, to allow users to specify that the new variable be used for live animation, Export, and writing to VS and ERD files, respectively.)

The argument *erdname* is used to provide a shorter name (8 characters or less) that can be used for ERD outputs, which have length limitations on their output names. Alternatively, specify `NULL` and a name will be generated automatically if *shortname* exceeds 8 characters.

The *longname* string defines a possible plot label written into VS and ERD files. The string is limited to 64 characters (such a long name is not recommended). If a longer string is provided, the function will truncate it.

The argument *real* is a pointer to a variable that is declared and calculated in your external code.

The argument *units* should be the keyword (ALL CAPS) for the units for the new variable.

The value for the variable should be calculated using internal dynamical units. Conversion to the specified *units* is done when the variable is exported or written to file.

The *genname*, *bodyname*, and *component* strings define additional labels written into documentation files generated with the with the function `vs_write_doc_files`. The strings are also used as labels written into VS and ERD files to support automated labeling of plots. Labels written into ERD file headers are limited to 64 characters; longer strings are truncated.

The argument *where* is a `vs_stage` (integer) that indicates the stage in a timestep where the output variable is calculated. The valid stage values and enum macros are 1 (State, `BLK_STATE`), 2 (Control `BLK_CTL`), 3 (Kinematics `BLK_KIN`), and 4 (Dynamics `BLK_DYN`). If the stage is not known, set the value to 0.

Lines 64 – 69 (Figure 13) show how `vs_define_out_where` is used to define two new outputs for the example steering controller, using pointers to external variables `sXprev`, `sYprev`, and `sLprefErr` that were declared in line 26 (Figure 12, page 55).

`vs_define_par`

`vs_define_sypar`

Declarations:

```
int vs_define_par (char *keyword, void *par, vs_real value,
                  char *units, int visible, int init, char *desc);

int vs_define_sypar (char *keyword, void *par, vs_real value,
                    char *units, int visible, int init, char *desc);
```

Return:

Each of these functions returns the ID of the new parameter if successful; they return -2 and generate an error if the keyword is already associated with an existing variable. ID numbers start at one and increase with each variable added. Thus the return value is also the total number of parameters that are now installed.

Each function adds a parameter to the math model. They are intended for use in the calling program or in a `setdef` callback function.

`vs_define_sypar` is almost the same as `vs_define_par`, except the new parameter is installed internally on a list of system parameters rather than model parameters.

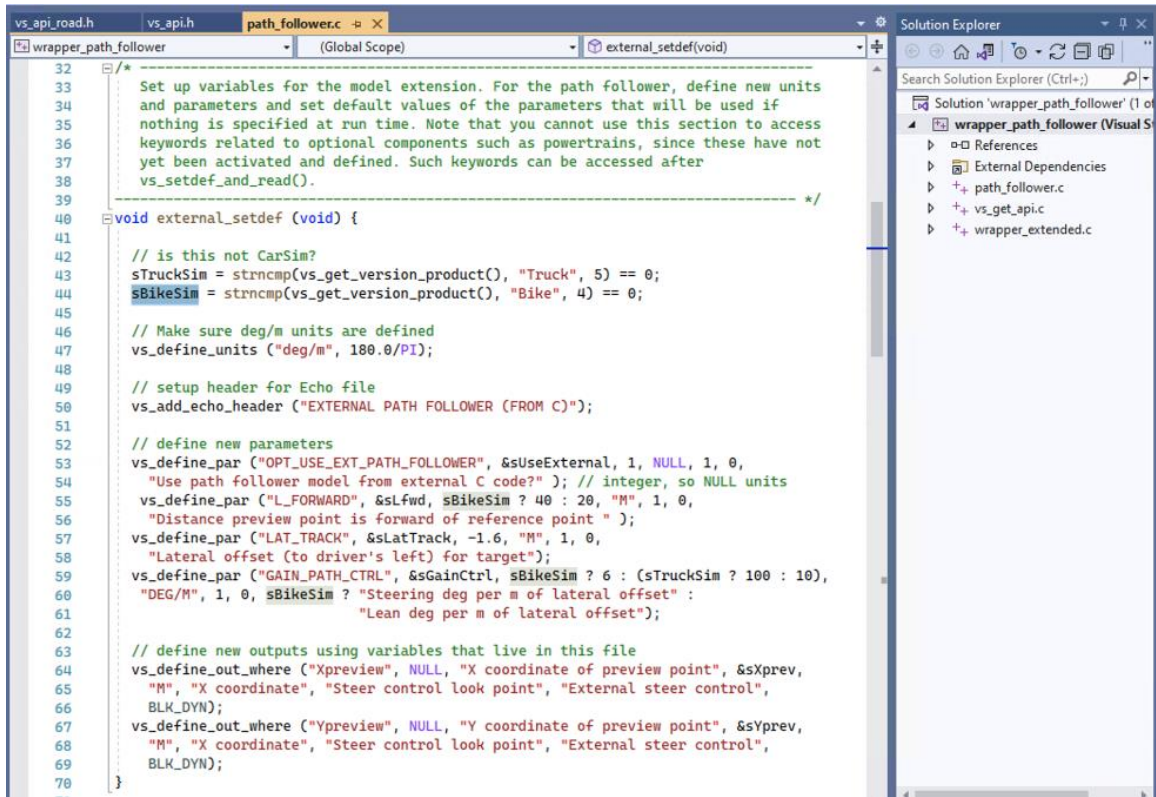


Figure 13. Example setdef callback function.

The argument *keyword* is a string in ALL CAPS that is used to identify the parameter in subsequent API function calls and in regular input files for the solvers. Both *keyword* and *desc* must remain constant from the time this function is invoked until the completion of the run.

The argument *par* is a pointer to a variable that is declared in your external code. The keyword will be installed in the internal database, so users can specify values in input Parsfiles and view the value used in a run in the echo file generated when the solver runs.

The argument *units* should be the uppercase keyword for the units for the new parameter if it is a `vs_real`. Alternatively, specify `NULL` to indicate that *par* is a pointer to an integer.

The default value of the parameter is specified as *value*. If the parameter is an integer, *value* is coerced internally to an integer value. If the parameter has units, the value should be specified using those units.

The argument *visible* indicates whether the parameter should normally be shown in the Echo file; specify 0 to hide it, 1 to show it.

The argument *init* indicates whether the VS Math Model should perform an initialization if the parameter is changed during a simulation via a VS Event. If 0, no initialization is needed.

The argument *desc* is a string that will be written into echo files to describe the parameter.

Lines 56 – 63 (Figure 13, page 59) show how this function is used to define four new parameters, using pointers to external variables `sLfwd`, `sLatTrack`, and `sGainStr` that were declared in lines 23, 24 and 25 (Figure 12, page 55), and the integer parameter `sUseExternal` (line 29).

These functions are similar in concept to the VS Command `DEFINE_PARAMETER`.

vs_define_sv

vs_define_sv_ode

Declaration:

```
int vs_define_sv(char *keyword, vs_real *real, vs_real value,
                 char *units, char *desc);

int vs_define_sv_ode(char *keyword, vs_real *real, vs_real value,
                    char *units, char *desc, vs_real *deriv);
```

Return:

This returns the ID of the new variable if successful; it returns -2 and generates an error if the keyword is already associated with an existing variable. ID numbers start at one and increase with each variable added. Thus a positive return value is also the total number of state variables that are now installed.

This function installs properties associated with a state variable into the internal database of the VS Math Model and is intended for use in the calling program or in a `setdef` callback function.

The argument *keyword* is an uppercase string used to identify the variable in subsequent API function calls and in regular input files for the solvers. To be consistent with native state variables, the keyword should begin with “SV_”, e.g., “SV_MY_NEW_VAR”. The keyword will be installed in the internal database, so users can specify an initial value in input Parsfiles and view the current value in the echo file generated when the solver runs.

The argument *real* is a pointer to a variable that is declared in your external code. The argument *value* is assigned to the variable. (This is done to avoid an extra line of code to assign the initial value.)

The argument *units* should be the uppercase keyword for the units for the new variable.

The argument *desc* is a string that will be written into echo files to describe the variable.

The new variable can be updated either with explicit calculations that you provide in your external code, or by integrating the derivative of the new variable. If the variable is calculated using numerical integration of its derivative, then you need to use the function `vs_set_sym_attribute` to set a pointer to the derivative, whose value you will calculate in your external code.

Calculations for this variable or its derivative should be made using internal dynamical units.

The strings *keyword* and *desc* must remain constant from the time this function is invoked until the completion of the run.

The function `vs_define_sv_ode` can be used if you also have a derivative variable. The pointer to this variable is the argument *deriv*.

vs_define_units

Declaration:

```
void vs_define_units (char *desc, vs_real gain);
```

This function adds units to the internal database of the VS Math Model. It is intended for use in a calling program or in a `setdef` callback function.

The argument *desc* is a mixed-case string. This is copied internally and converted to ALL CAPS to define a keyword that can be used later to identify the units. The argument *gain* defines the scale factor used to convert from internal units (e.g., meters) to user units (e.g., millimeters). For example, if the user units are millimeters, the gain is 1000.0. If the keyword is already in place, then this function returns without modifying the database. For details about the VS Math Model unit systems, see the *VS Math Models Manual*.

This function is similar in function to the VS Command `DEFINE_UNITS`.

An example application of this function is shown in line 50 of the listing in Figure 13 (page 59) to install units “deg/m” for the example steering controller.

vs_get_numeric_value

vs_get_int_value

Declaration:

```
vs_real vs_get_numeric_value(char *expression);  
int vs_get_int_value(char *expression);
```

Return:

Numerical value obtained by evaluating the string *expression*.

This function uses the internal VS symbolic expression capabilities to interpret the string *expression* using current symbols and VS functions. All symbols for parameters and variables are replaced with their current values and any functions are applied.

This function may modify the argument *expression*; therefore, the argument must be a string variable. (An error is likely to occur if a constant string is used.)

This function might be used to obtain a numerical value for a variable defined in external code (a variable not installed in the VS Math Model) that might be expressed as a symbolic expression of parameters and variables within the VS Math Model.

The function `vs_get_int_value` is used when you want an integer value of the expression being evaluated.

vs_get_sym_attribute

Declaration:

```
int vs_get_sym_attribute(int id, vs_sym_attr_type type,  
void **att);
```

Return:

If the function proceeded successfully, it returns a value of 0. It returns -1 and generates an error message if there is a problem, such as the ID number being out of range.

This function gets an attribute of an internal symbolic structure.

The argument *id* identifies the element of the internal array that is being accessed. The ID is returned from other functions that introduce new variables, such as `vs_define_imp_where` and `vs_define_out_where`. The argument *type* is one of the values defined in the header file `vs_deftypes.h`. The major types are listed and described in Table 12 on page 72. (Attributes identified in the **Get/Set** column as either Get or Both can be used with this function.)

A pointer to a pointer *att* is set by the function to allow access to the attribute.

This function is used extensively in the machine-generated C code from VS Lisp. It is not commonly used for external code but might be helpful for some advanced applications such as live animation. If used, it would be applied in the calling program or in a `setdef` callback function.

vs_get_var_id

Declaration:

```
int vs_get_var_id(char *keyword, vs_sym_attr_type *type);
```

Return:

ID number for element in internal array that is associated with a variable identified with *keyword*.

The function also provides the type of variable with the argument *type*, which will be given a value from the enum list `vs_sym_attr_type` declared in `vs_deftypes.h`.

If the variable was defined using an API function (`vs_define_imp_where`, `vs_define_out_where`, etc.), then the ID returned by this function is the same as the value returned when the new variable was defined. Therefore, this function is not really needed for new variables; it is more efficient to use the value returned from the `vs_define_xxx` function when the new variable was defined.

The main reason for using this function is to obtain the ID for a built-in variable or model parameter that is already in the VS Math Model. The ID can then be used to modify its properties using `vs_set_sym_attribute`. Resetting properties of built-in variables should be done only by advanced users who understand the implications.

vs_get_var_ptr

Declaration:

```
vs_real *vs_get_var_ptr(char *keyword);
```

Return:

Pointer to `vs_real` variable used in the solver program. If the keyword is not recognized or is not associated with a variable of type `vs_real`, the function returns `NULL`.

This function gets a pointer to one of the thousands of named parameters and variables in the VS Math Model.

The variable is identified by a keyword that must be ALL CAPS and identical to the keyword used to identify the variable in an echo file produced by the program (see the *VS Math Models Manual*

for details about echo files). Any variable appearing in the echo file with units is a `vs_real`. (Integer variables do not have units.)

The pointer remains valid throughout the running of the model, from initialization, to running, to termination. The pointer can be used either to obtain the value of the variable, or to set a new variable. Of course, setting the value is not recommended unless you know what you are doing.

This function is intended for use in the calling program or in a `setdef` callback function. However, keywords associated with optional model components (Speed Controller, Powertrain, etc.) cannot ever be accessed from within a `setdef` callback function. These special keywords can be accessed after the `vs_setdef_and_read` API function has been explicitly called by your program.

Note	Beginning in VehicleSim 2020.1, parameters and variables for the model are added when the <code>MODEL_LAYOUT</code> command is applied from the Parsfile. In older versions, these parameters and variables were defined when reading the vehicle code given in the Simfile. Consequently, much of what was previously accessed in <code>external_setdef</code> must now be accessed in <code>external_calc</code> , after <code>vs_setdef_and_read</code> has been explicitly called by your program.
-------------	--

For example, `vs_get_var_ptr` is used in the listing shown in Figure 10 (page 45) to provide access to existing variables from the VS Math Model, identified with keywords `XCM_TM`, `YCG_TM`, `YAW`, and others. In each case, an external variable points to the internal variable (`*sXcg`, `*sYcg`, etc.).

`vs_get_var_ptr2`

Declaration:

```
vs_real *vs_get_var_ptr2(const char *keyword,  
                        vs_keyword_type *type, vs_bool allowErrors);
```

Return:

Pointer to `vs_real` variable used in the solver program. If the *keyword* argument is not recognized or is not associated with a variable of type `vs_real`, the function returns `NULL`.

This function returns a pointer to one of the named parameters or variables in the VS Math Model. It operates identically to `vs_get_var_ptr` except that this function has the option to return (via output parameter) the type of keyword that was found and allows the caller to specify whether or not failing to find the keyword results in an error code being set internally in the solver.

The argument *keyword* must be ALL CAPS and identical to the keyword used to identify the variable in an echo file produced by the program (see the *VS Math Models Manual* for details about echo files). Any variable appearing in the echo file with units is a `vs_real`. (Integer variables do not have units.)

The argument *type* is an optional output parameter. This argument may be set to `NULL`. If not set to `NULL`, the variable specified will be populated with the type of variable that was found. If the variable is not found (i.e., the return value is `NULL`), the variable that this argument points to will not be modified by this call.

If the call is successful, the returned pointer remains valid throughout the running of the model, from initialization, to running, to termination. The pointer can be used either to obtain the value of the variable, or to set the variable to a new value.

Using the function to set a variable to an inappropriate value or changing a variable at an inappropriate time may result in undefined behavior (including program crashes). Use this capability with caution.

`vs_get_var_ptr_int`

Declaration:

```
int *vs_get_var_ptr_int(char *keyword);
```

Return:

Pointer to an `int` variable used in the solver program. If the keyword is not recognized or is not associated with a variable of type `int`, the function returns `NULL`.

This function gets a pointer to a named integer parameter in the VS Math Model.

The variable is identified by *keyword* that should be ALL CAPS and identical to the keyword used to identify the variable in an echo file produced by the program (see the *VS Math Models Manual* for details about echo files). Any variable appearing in the echo file without units is an `int`. (Floating-point variables always have units.)

The pointer remains valid throughout the running of the model, from initialization, to running, to termination. The pointer can be used either to obtain the value of the variable, or to set a new variable. Of course, setting the value is not recommended unless you know what you are doing.

This function is intended for use in the calling program or in a `setdef` callback function. However, many keywords associated with the model layout and its components (Speed Controller, Powertrain, etc.) cannot ever be accessed from within a `setdef` callback function. These keywords can be accessed after the `vs_setdef_and_read` API function has been explicitly called by your program.

`vs_get_var_ptr_int2`

Declaration:

```
int *vs_get_var_ptr_int2(const char *keyword,  
                          vs_keyword_type* type, vs_bool allowErrors);
```

Return:

Pointer to an `int` variable used in the solver program. If the *keyword* argument is not recognized or is not associated with a variable of type `int`, the function returns `NULL`.

This function returns a pointer to a named integer parameter in the VS Math Model. It operates identically to `vs_get_var_ptr_int` except that this function has the option to return (via output parameter) the type of keyword that was found, and it allows the caller to specify whether failing to find the keyword results in an error code being set internally in the solver.

The *keyword* argument must be ALL CAPS and identical to the keyword used to identify the variable in an echo file produced by the program (see the *VS Math Models Manual* for details about echo files). Any variable appearing in the echo file without units is an `int`. (Floating-point variables always have units.)

The argument *type* is an optional output parameter. This argument may be set to `NULL`. If not set to `NULL`, the variable associated with this pointer will be set to the `vs_keyword_type` property of the variable that was found. If the variable is not found (i.e., the return value is `NULL`), the variable that *type* points to will not be modified.

If a pointer is returned, it remains valid throughout the running of the model, from initialization and running to termination. The pointer can be used either to obtain the value of the variable, or to set a new variable.

Using the function to set a variable to an inappropriate value or changing a variable at an inappropriate time may result in undefined behavior (including program crashes). Use this capability with caution.

vs_get_var_output_gain

Declaration:

```
vs_real vs_get_var_output_gain(const char *keyword);
```

Return:

The gain that is applied to the specified variable when it is output (e.g. to VS or ERD simulation results data), or 1.0 if the specified variable cannot be found, or it does not have an output gain. The gain is due to the user units assigned to the variable.

The *keyword* argument must be ALL CAPS and identical to the keyword used to identify the variable in an echo file produced by the program (see the *VS Math Models Manual* for details about echo files).

vs_have_keyword_in_database

Declaration:

```
int vs_have_keyword_in_database(char *keyword);
```

Return:

1 if *keyword* is in the database; 0 if it is not.

This function is used to determine whether a keyword is in the VS Math Model database.

The string *keyword* is case sensitive and must be ALL CAPS.

This function can be used to determine features of the VS Math Model. For example, the keyword "XO_2" is in the database for CarSim and TruckSim models that include a trailer; it is not in the database if the model does not include a trailer.

vs_get_import_id

Declaration:

```
int vs_get_import_id(const char* keyword);
```

Return:

The ID of the Import with the specified name, or zero if an Import with the specified name does not exist.

vs_get_import_count

Declaration:

```
int vs_get_import_count();
```

Return:

The total number of Imports defined in the VS Math Model.

vs_get_active_import_count

Declaration:

```
int vs_get_active_import_count();
```

Return:

The number of Imports currently activated in the VS Math Model, which is the size of the Import Array.

vs_get_import_active_index

Declaration:

```
int vs_get_import_active_index(int id);
```

Return:

The index in the Import Array for the Import with the specified ID, or a negative value if the specified Import is not currently active.

The *id* argument must be ≥ 0 and $< \text{vs_get_import_count}()$.

vs_get_active_import_id

Declaration:

```
int vs_get_active_import_id(int index);
```

Return:

The Import ID of the specified element of the Import Array.

The *index* argument must be ≥ 0 and $< \text{vs_get_active_import_count}()$.

vs_get_import_keyword

Declaration:

```
const char *vs_get_import_keyword(int id);
```

Return:

The keyword/name of the specified Import.

The *id* argument must be ≥ 0 and $< \text{vs_get_import_count}()$.

vs_get_active_import_keyword

Declaration:

```
const char *vs_get_active_import_keyword(int index);
```

Return:

The keyword/name of the specified Import.

This function behaves like `vs_get_import_keyword`, except that it accepts an index into the Import Array instead of an Import ID.

The *index* argument must be ≥ 0 and $< \text{vs_get_active_import_count}()$.

See `vs_` for additional information.

vs_import_type

Declaration:

```
int vs_import_type(int id);
```

Return:

A value corresponding to the `vs_import_option` enum in `vs_deftypes.h`.

This function is used to determine the current Import Mode (type) of the specified Import Variable. Refer to `vs_import_option` enum in `vs_deftypes.h` for the meaning of each possible return value. Note that a return value of 0 indicates that the Import variable's mode is "Ignore", meaning that the Import variable is not active/visible.

The *id* argument must be ≥ 0 and $< \text{vs_get_import_count}()$.

vs_get_active_import_mode

Declaration:

```
vs_import_option vs_get_active_import_mode(int index);
```

Return:

A value corresponding to the `vs_import_option` enum in `vs_deftypes.h`.

This function behaves like `vs_import_type`, except that it accepts an index into the Import Array instead of an Import ID.

The *index* argument must be ≥ 0 and $< \text{vs_get_active_import_count}()$.

See `vs_import_type` for additional information.

vs_get_import_units

Declaration:

```
const char *vs_get_import_units(int id);
```

Return:

A string containing the name of the units of the specified Import. If the Import does not have units, this function may return an empty string, or NULL.

The *id* argument must be ≥ 0 and $< \text{vs_get_import_count}()$.

vs_get_active_import_units

Declaration:

```
const char *vs_get_active_import_units(int index);
```

Return:

A string containing the name of the units of the specified Import. If the Import does not have units, this function may return an empty string, or NULL.

This function behaves like `vs_get_import_units`, except that it accepts an index into the Import Array instead of an Import ID.

The *index* argument must be ≥ 0 and $< \text{vs_get_active_import_count}()$.

See `vs_get_import_units` for additional information.

vs_get_import_value

Declaration:

```
vs_real vs_get_import_value(int id);
```

Return:

The current value that the specified Import is set to.

The *id* argument must be ≥ 0 and $< \text{vs_get_import_count}()$.

vs_get_active_import_value

Declaration:

```
vs_real vs_get_active_import_value(int index);
```

Return:

The current value that the specified Import is set to.

This function behaves like `vs_get_import_value`, except that it accepts an index into the Import Array instead of an Import ID.

The *index* argument must be ≥ 0 and $< \text{vs_get_active_import_count}()$.

See `vs_get_import_value` for additional information.

vs_get_import_scale

Declaration:

```
vs_real vs_get_import_scale(int id);
```

Return:

The scale factor that converts from internal units to user units for the specified Import.

vs_import_result

Declaration:

```
vs_real vs_import_result(int id, vs_real native);
```

Return:

The result of combining an imported variable with a native variable.

This function is used to combine an Import variable with an internal (native) value, using a mode selected the user at runtime.

From the end-user point of view, all Import variables are optional and are disabled unless activated at runtime. In many cases, an internal native variable is used in the math model if the Import is not active. When the Import is activated, several modes can be set to specify how the imported variable is combined with the native variable. The modes are `add`, `vs_add`, `replace`, `vs_replace`, `multiply`, `vs_multiply`, and `ignore`. (See the *VS Math Models Manual* for details about the modes available when activating Import variables at runtime.)

The argument *id* is the ID associated with the Import. If the Import was installed with the installation function `vs_define_imp_where`, then this is the value returned when the Import variable was defined.

Note If the ID of the Import variable is not known, it can be obtained with the function `vs_get_var_id`.

The argument *native* is the value that is combined with the imported variable based on the user-selected mode (`add`, `multiply`, etc.)

If the native value is 0.0, then this function is not necessary. Instead, use the Import variable directly.

vs_get_active_import_result

Declaration:

```
vs_real vs_get_active_import_result(int index, vs_real native);
```

Return:

The result of combining an imported variable with a native variable.

This function behaves like `vs_import_result`, except that it accepts an index into the Import Array instead of an Import ID.

The *index* argument must be ≥ 0 and $< \text{vs_get_active_import_count}()$.

See `vs_import_result` for additional information.

vs_install_import_callback

Declarations:

```
void vs_import_callback_t(int id, vs_real* currentvalue, vs_real
    varToImpScale, vs_real rawValue, void* userData)

vs_bool vs_install_import_callback(int id, vs_import_callback_t*
    callbackFunc, void* userData)
```

Return:

True if the callback was installed, or false if the function was already registered.

Installs a callback function which will be called when the specified import is processed. If the import is active, this function will be applied after the base/internal value has been modified by the import. Any number of import callbacks may be registered for each import.

The `vs_import_callback_t` function prototype describes the function which the user implements to receive the callback.

Each registered function will be called in the order it was installed. When called, the callback function can modify the value pointed to by the *currentValue* parameter to change the value that solver will use. This value is in internal units (not Import units).

The *varToImpScale* parameter is the scale factor that is used to convert from internal units to Import units. Divide a value in Import units by this value to compute the corresponding value to apply to *currentValue*.

The *rawValue* parameter holds that original value (in internal units) of the variable before any modifications were made to it by an activated import value from the import array and/or other callbacks.

vs_remove_import_callback

Declarations:

```
void vs_import_callback_t(int id, vs_real* currentvalue, vs_real
    varToImpScale, vs_real rawValue, void* userData)

vs_bool vs_remove_import_callback( int id,
    vs_import_callback_t* callbackFunc)
```

Return:

True if the callback was previously installed and has been removed, false otherwise.

Removes a previously installed callback.

vs_install_symbolic_func

Declarations:

```
void vs_install_symbolic_func (char *name, void *func, int nargs);
```

Use `vs_install_symbolic_func` to install a function name and associated function pointer into the internal database, so it will be recognized and applied at run-time in response to inputs read from Parsfiles.

The argument *name* is the string used to identify the function in a calculator expression, and should be ALL CAPS. It must remain constant from the time `vs_install_symbolic_func` is called until the simulation is complete. The argument *func* is a pointer to the associated function. The function must have 1 to 4 arguments, all with type `vs_real`. The argument *narg* specifies how many arguments should be provided to the installed function. (The number must be between 1 and 4, inclusive.)

This function is intended for use in the calling program or in a `setdef` callback function.

vs_install_keyword_alias

Declaration:

```
int vs_import_result (char *existing, char *alias);
```

Return:

0 if the new keyword *alias* is installed successfully; -1 if keyword *existing* was not recognized, -2 if keyword *alias* has already been installed, +1 if there is a memory error.

This function installs an alternate keyword for a parameter, variable, or part of a configurable function into the internal database of the VS Math Model. If successful, the new keyword *alias* can be used with the same effect as the keyword *existing* in Parsfiles and API functions that use a keyword to identify an internal variable.

This function is used to provide compatibility with older versions of VehicleSim products when keywords have been changed for new versions.

vs_put_on_free_list

Declaration:

```
void vs_put_on_free_list (void *thing);
```

The VS Math Model maintains an internal list of pointers to structures that have been allocated memory. All items on this list are freed when a simulation terminates. The function `vs_put_on_free_list` will add the pointer *thing* to the internal list. The pointer *thing* should be a pointer to an array or other structure that was just allocated memory using `malloc` or `calloc`.

vs_set_stop_run

Declaration:

```
void vs_set_stop_run (vs_real if_gt_0, const char *format, ...);
```

If the argument *if_gt_0* is greater than zero, then this function will set the internal variable of the solver to indicate the run should be stopped (this is the same internal variable used to determine the value returned by the function *vs_stop_run*). If provided with a format string and optional arguments, it will also write a line into the log file. The *format* argument and any additional arguments are the same as the corresponding arguments for the ANSI C function *fprintf*.

If the argument *if_gt_0* is not greater than zero, then this function does nothing.

The optional arguments after *format* are not declared in the header file used with MATLAB; in this case, *format* is treated as a simple string.

vs_set_sym_attribute

Declaration:

```
int vs_set_sym_attribute(int id, vs_sym_attr_type type,  
void *att);
```

Return:

If the function proceeded successfully, it returns a value of 0. It returns -1 and generates an error message if there is a problem, such as the ID number being out of range.

The argument *id* identifies the element of the internal array that is being modified. The ID is returned from other functions that introduce new variables, such as *vs_define_out_where* and *vs_define_sv*. The argument *type* is one of the values defined in the header file *vs_deftypes.h*. The major types are listed and described in Table 12. Attributes identified in the **Get/Set** column as either Set or Both can be used with this function. A pointer is passed for the attribute.

Table 12. VS symbolic structure attributes that can be set and gotten.

Attribute	Type	Get/Set	Description
ANI_REAL	double *	Get	Pointer to output variable scaled to user units
ANI_SHORT_NAME	string	Get	Short plot label (up to 8 characters, no spaces)
IMP_DESC	string	Both	Description of Import variable
IMP_KEYWORD	string	Both	ALL CAPS keyword for Import variable
IMP_REAL	double *	Both	Pointer to Import variable
ISYM_DIMENSIONS	int	Set	Number of dimensions for array of parameters
ISYM_INDEX_ _n <i>n</i> = 1, 2, ... 5	int *	Set	Pointer to integer parameter used as one of the indices in a parameter array
ISYM_LIMIT_ _n <i>n</i> = 1, 2, ... 5	int	Set	Limit for one of the indices in a parameter array (indexing goes from 1 to this limit)
ISYM_REAL	double *	Set	Pointer to array of parameters
ISYM_INTEGER	int *	Set	Pointer to array of parameters

Table 12. VS symbolic structure attributes that can be set and gotten.

Attribute	Type	Get/Set	Description
OUTVAR_BODY_NAME	string	Both	Label to identify variable in overlay plots
OUTVAR_GEN_NAME	string	Both	Generic name used for Y axis of overlay plots
OUTVAR_LONG_NAME	string	Both	Detailed plot label (up to 64 characters)
OUTVAR_REAL	double *	Both	Pointer to output variable
OUTVAR_SHORT_NAME	string	Both	Short plot label (up to 8 characters, no spaces)
PAR_DESC	string	Both	Description of Import variable
PAR_INTEGER	int *	Both	Pointer to an integer parameter
PAR_KEYWORD	string	Both	ALL CAPS keyword for parameter
PAR_REAL	double *	Both	Pointer to real (double) parameter
PAR_VISIBLE	int	Set	Visibility of parameter in echo file
SV_DERIVATIVE	double *	Set	Pointer to state variable derivative
SV_DESC	string	Both	Description of state variable
SV_KEYWORD	string	Both	ALL CAPS keyword for state variable

vs_set_sym_int

Declaration:

```
int vs_set_sym_int(int id, vs_sym_attr_type type, int value);
```

Return:

If the function proceeded successfully, it returns a value of 0. It returns -1 and generates an error message if there is a problem, such as the ID number being out of range.

Use this function to set an integer value that is part of an internal symbolic structure. It is intended for use in the calling program or in a `setdef` callback function. However, symbolic structures associated with optional model components (Speed Controller, Powertrain, etc.) cannot ever be accessed from within a `setdef` callback function. These symbolic structures can be accessed after the `vs_setdef_and_read` API function has been explicitly called by your program.

The argument *id* identifies the internal symbolic structure. The ID is returned from other functions that introduce new variables, such as `vs_define_par`. The argument *type* is one of the values defined in the header file `vs_deftypes.h`.

The main value for *type* that might be used in custom C code is `PAR_VISIBLE`. If the `PAR_VISIBLE` attribute is zero, information about the parameter is not written into the echo file; otherwise, it is written. Further, if the value is negative, the parameter is protected during a run and cannot be changed when processing inputs for an Event.

vs_string_copy_internal

Declaration:

```
char *vs_string_copy_internal (char **target, char *source);
```

Use this function to make a copy of a string that will persist in memory until the memory is freed at the end of the run. The string `source` will be copied to `*target`, and a pointer is stored internally to allow automatic freeing of the memory.

For example, this function is useful for installing keywords that are created dynamically using formatted print statements such as `sprintf` (Listing 2).

In this example, a string is created in `buffer` and then duplicated to memory allocated for `keyword`. The string will persist in memory until the run terminates.

Listing 2. Example code showing use of `vs_string_copy_internal`.

```
static char * my_example (char *root, int n) {
    char buffer[200], *keyword;

    sprintf (buffer, "%s(%d)", root, n);
    vs_string_copy_internal (&keyword, buffer);
    return keyword;
}
```

Interact with Simulation Files

The Echo files and Log files are generated automatically, and keywords added for variables using API functions are automatically recognized when the VS Math Model scans input Parsfiles. There may be some cases where additional information is needed involving these files.

Parameter Groups in Echo Files

Echo files are organized into sections that list related parameters, e.g., all parameters related to payloads. New parameters added with the `vs_define_par` command are automatically associated with the current group in the Echo file, identified with an ID.

Table 13 lists some functions for managing parameter groups within Echo files.

Table 13. Functions for managing groups in Echo files.

Function	Description
<code>vs_add_new_par_group</code>	Add a new section to an Echo file
<code>vs_insert_echo_blank_line</code>	Insert a blank line into an Echo file
<code>vs_insert_echo_comment</code>	Insert a comment string into an Echo file
<code>vs_get_active_par_group</code>	Get the ID of the currently active Echo parameter group
<code>vs_set_active_par_group</code>	Set the currently active Echo parameter group

`vs_add_new_par_group`

Declaration:

```
int vs_add_new_par_group(char *title);
```

Return:

ID to new parameter group created for the Echo file.

This function adds a new parameter group section to the Echo file using the specified *title*. It returns the ID to the new section and sets the scope such that all new parameters created with the `vs_define_par` command are automatically placed in the new section until the scope is changed.

vs_insert_echo_blank_line

Declaration:

```
int vs_insert_echo_blank_line(void);
```

Return:

ID to blank line, handled the same as the ID returned for a new parameter.

This function adds a blank line into the Echo file. The returned ID can be used to make the blank line visible or hidden.

vs_insert_echo_comment

Declaration:

```
int vs_get_active_par_group(const char *comment);
```

Return:

ID to comment, handled the same as the ID returned for a new parameter.

This function adds a comment into the Echo file, wrapped if necessary using the same rules as used for other comments in the file. The returned ID can be used to make the comment visible or hidden.

vs_get_active_par_group

Declaration:

```
int vs_get_active_par_group(void);
```

Return:

ID to currently active parameter group.

This function gives the ID for the currently active parameter group.

vs_set_active_par_group

Declaration:

```
void vs_set_active_par_group(int id);
```

This function restores a parameter group. The existing group with the specified *id* becomes the currently active parameter group.

Miscellaneous Functions Involving Files

Table 14 lists some functions for adding custom content to an Echo file or Log file, and for scanning in input Parsfile. These are not needed if all information read and written involves parameters and variables added with API functions such as `vs_define_par`. However, if some of the parameters or

variables of interest are handled 100% with external code, then low-level access might be needed to show them in the Echo or scan the Parsfile to read information.

Table 14. Functions involving simulation files.

Function	Description
<code>vs_read_next_line</code>	Read the next line of text from an input Parsfile
<code>vs_write_f_to_echo_file</code>	Write an echo file entry for a floating-point parameter
<code>vs_write_header_to_echo_file</code>	Write a string into an echo file as a header
<code>vs_write_i_to_echo_file</code>	Write an echo file entry for an integer parameter
<code>vs_write_to_echo_file</code>	Write a string into an echo file
<code>vs_write_to_logfile</code>	Write a string into a log file

`vs_read_next_line`

Declaration:

```
void vs_read_next_line (char *buffer, int n);
```

This function is intended for use in a `scan` callback function, in the event that data used for external code covers multiple lines of text from the input file.

It reads the next line of the currently open Parsfile, up to *n* characters. The result is put into *buffer*.

`vs_write_f_to_echo_file`

Declaration:

```
void vs_write_f_to_echo_file (char *key, vs_real value,
                             char *doc);
```

This function is intended for use in an `echo` callback function to add information about an external floating-point variable to an echo file. It applies the same format that is used for all the parameters that are native to the VS Math Model.

The argument *key* is the keyword, *value* is the numerical value, and *doc* is the description that follows. To match the syntax used for built-in parameters, the documentation should begin with a comment character '!'.

`vs_write_header_to_echo_file`

Declaration:

```
void vs_write_header_to_echo_file (char *string);
```

This function is intended for use in an `echo` callback function to add a heading to identify a section involving external data. It applies the same format that is used for other headers written in the echo file.

The argument *string* is the text that will be in the header. The convention for VS headers is that they are less than 85 characters and are all caps. (There is no built-in limit to the length of the argument *string*; this is just a matter of convention.)

vs_write_i_to_echo_file

Declaration:

```
void vs_write_i_to_echo_file (char *key, int value, char *doc);
```

This function is intended for use in an `echo` callback function to add information about an external integer variable to an echo file. It applies the same format that is used for all the parameters that are native to the VS Math Model.

The argument *key* is the keyword, *value* is the numerical value, and *doc* is the description that follows. To match the syntax used for built-in parameters, the documentation should begin with a comment character '!'.

vs_write_to_echo_file

Declaration:

```
void vs_write_to_echo_file (char *format, ...);
```

This function is intended for use in an `echo` callback function to add a line of text to the file. The *format* argument and any additional arguments are the same as the corresponding arguments for the ANSI C function `fprintf`.

The optional arguments after *format* are not declared in the header file used with MATLAB; in this case, *format* is treated as a simple string.

vs_write_to_logfile

Declaration:

```
void vs_write_to_logfile (int level, char *format, ...);
```

This function will write text into the log file being generated as a run proceeds. The argument *level* is compared to the system parameter `OPT_LOG_VERBOSE`, which a user sets to 0 (log file should only show error messages), 1 (log file also documents events), or 2 (log file also documents reading of data files). For example, if *level* is 1, the specified information is written when `OPT_LOG_VERBOSE` is set to either 1 or 0.

The *format* argument and any additional arguments are the same as the corresponding arguments for the ANSI C function `fprintf`.

The optional arguments after *format* are not declared in the header file used with MATLAB; in this case, *format* is treated as a simple string.

vs_v_write_to_logfile

Declaration:

```
void vs_v_write_to_logfile (int level, const char *format,  
                           va_list args);
```

Alternate log function that uses a *va_list* of arguments instead of a variable number of parameters. Otherwise, this function operates identically to `vs_write_to_logfile()`. See ANSI C documentation for the `vprintf()` function for additional information on how to use the *va_list* parameter.

Define and Access Configurable Table Functions

VS Math Models often use configurable functions to define potentially nonlinear relationships between one or two independent variables and a dependent variable that is calculated by a method specified at runtime. Most of the methods involve interpolation and extrapolation using tables of data.

Existing functions can be applied using symbolic equations provided with VS Commands, or in external code using the API function `vs_table_calc`. New functions can also be defined at runtime using the VS Command `DEFINE_TABLE` or the API function `vs_define_table`.

Table 15 lists functions in the VS API that can be used to interact with configurable functions.

Table 15. Functions used to define and access configurable table functions.

Function	Description
<code>vs_define_table</code>	Define a new configurable table function.
<code>vs_set_units_table</code>	Set units for table dependent and independent variables.
<code>vs_table_calc</code>	Calculated value obtained from table function.
<code>vs_table_index</code>	ID of table function group associated with a root keyword.
<code>vs_table_ninst</code>	Number of instances available for table function.
<code>vs_table_ntab</code>	Number of datasets available for table function.

Internally, all configurable table functions are stored in groups with related keywords. Each function has integer indices that are used to identify the internal data to be used with the API functions.

`vs_define_table`

Declaration:

```
int vs_define_table(char *root_keyword, int ntab, int ninst);
```

Return:

ID associated with a new function group (if the new group is defined). If the function fails a value of -1 is returned, and an error message is generated.

This function creates a new group of configurable function datasets and associated keywords based on *root_keyword*. For example, if *root_keyword* is `MY_FUNC`, keywords are installed such as `MY_FUNC_TABLE`, `MY_FUNC_COEFFICIENT`, `MY_FUNC_GAIN`, and others described in the *VS Math Models Manual*.

The argument *root_keyword* must be ALL CAPS. The string is copied internally and therefore does not have to be static.

The new table function supports *ntab* datasets. If *ntab* is greater than one, a new parameter is installed with a keyword created by putting a prefix `I` on *root_keyword* (e.g., `IMY_FUNC`) to determine which dataset is active when reading data from a Parsfile. Also, keywords are installed with suffixes for each dataset, e.g., `MY_FUNC_TABLE(1)`, `MY_FUNC_TABLE(2)`, `MY_FUNC_GAIN(1)`, etc.

The new function supports *ninst* instances, as described below in the discussion of `vs_table_calc`.

This function is used internally by the VS Command `DEFINE_TABLE`.

`vs_set_units_table`

Declaration:

```
void vs_set_units_table (const char *table, const char *units,
                        const int which);
```

This function sets the units for a table group using three string arguments. *table* is the keyword for the function (it should end with `_TABLE` or `_CARPET`), *units* is a string for the units to be applied, and *which* is 0, 1, 2, to indicate whether units apply for the calculate output of the table (*which* = 0), the row variable (1), or the column variable (2).

`vs_table_calc`

Declaration:

```
vs_real vs_table_calc(int ID, vs_real xcol, vs_real x, int itab,
                    int instance);
```

Return:

Calculated value of function identified with *ID* and *itab*, given values of *x* and possibly *xcol*.

This function applies any of the configurable table functions that are built into a VS Math Model, or any that were added using the VS Command `DEFINE_TABLE` or the API function `vs_define_table`.

Internally, data for all of the configurable table functions are stored in an array. The specific group of functions is identified with *ID* and the dataset in that group is identified with *itab*. The argument *itab* should have a value between 1 and *ntab* (inclusive), where *ntab* can be obtained using the API function `vs_table_ntab` described below.

The *ID* associated with a group of functions is returned if the group is created using `vs_define_table`. Otherwise, it is obtained as a function of the root keyword using the API function `vs_table_index`, described below.

Depending on the data provided, the function might return a constant value, or use a coefficient multiplied by *x*, or apply one of many other calculation methods using *x*, or use a calculation method involving both *xcol* and *x*. The many available built-in methods are described in the *VS Math Models Manual*.

In some cases, you might want to perform multiple calculations using the same dataset. If the configurable function is set to use one of the tabular interpolation methods with a large number of data points, then the calculation is more efficient if the internal location in the table is known approximately. VS configurable functions can “remember” multiple locations in the tabular dataset, where each location is associated with an *instance* number. The argument *instance* should have a

value between 1 and *ninst* (inclusive), where *ninst* can be obtained using the API function `vs_table_ntab` described below.

vs_table_index

Declaration:

```
int vs_table_index(char *root_keyword);
```

Return:

ID of the group of table functions associated with *root_keyword* (if successful); -1 if *root_keyword* is not associated with a group of table functions.

The argument *root_keyword* must be ALL CAPS.

The ID returned by this function is needed to use the VS API functions `vs_table_calc`, `vs_table_ninst`, and `vs_table_ntab`.

vs_table_ninst

Declaration:

```
int vs_table_ninst(int ID);
```

Return:

Number of instances that are available for the configurable function identified by *ID*. If *ID* is not valid, then this function returns -1 and an error is generated.

vs_table_ntab

Declaration:

```
int vs_table_ntab(int ID);
```

Return:

Number of datasets in the group that can be accessed with the configurable function identified by *ID*. If *ID* is not valid, then this function returns -1 and an error is generated.

8. Model Arrays, Save and Restore Options

A VS Math Model performs a run by performing calculations at closely spaced intervals of time, using equations that define a math model. The current state of the model is defined by the simulation time, plus all parameters and Configurable Function datasets, plus a set of independent variables called *state variables*. Traditionally, the state variables for a multibody vehicle model have been defined exclusively with ordinary differential equations (ODE's). They are contained in an array called the *differential state variables* and are calculated each timestep using numerical integration as described in the VS Tech Memo: *Numerical Integration Methods in VS Math Models*.

New differential state variables can be added to the model using the API function `vs_define_sv_ode` (see Table 11 on page 53). Differential variables and equations can also be added with the VS Commands `DEFINE_VARIABLE` and `EQ_DIFFERENTIAL`.

Additional variables are used to keep track of conditions such as friction force. They are also needed to define the state of the model and are therefore considered state variables even though they do not have associated differential equations. They are called *extra state variables* in this document and can be added to the model with the API function `vs_define_sv`, or with the VS Commands `DEFINE_VARIABLE` and an appropriate `EQ_` command (`EQ_FULL_STEP`, `EQ_SAVE`, etc.).

Manage Model Arrays

Table 16 lists functions in the VS API that can be used to access the individual arrays of state variables.

```
vs_copy_all_state_vars_from_array  
vs_copy_all_state_vars_to_array  
vs_copy_differential_state_vars_from_array  
vs_copy_differential_state_vars_to_array  
vs_copy_extra_state_vars_from_array  
vs_copy_extra_state_vars_to_array
```

Declarations:

```
void vs_copy_all_state_vars_from_array (vs_real *array);  
void vs_copy_all_state_vars_to_array (vs_real *array);  
void vs_copy_differential_state_vars_from_array (vs_real *array);  
void vs_copy_differential_state_vars_to_array (vs_real *array);  
void vs_copy_extra_state_vars_from_array (vs_real *array);  
void vs_copy_extra_state_vars_to_array (vs_real *array);
```

These functions either copy values from internal memory to an externally defined array (`vs_copy_..._state_vars_to_array`), or they copy values from the external array into internal memory (`vs_copy_..._state_vars_from_array`).

Table 16. Functions used to manage model arrays.

Function	Description
<code>vs_copy_all_state_vars_from_array</code>	Set array of all state variables.
<code>vs_copy_all_state_vars_to_array</code>	Get array of all state variables.
<code>vs_copy_differential_state_vars_from_array</code>	Set array of differential state variables.
<code>vs_copy_differential_state_vars_to_array</code>	Get array of differential state variables.
<code>vs_copy_extra_state_vars_from_array</code>	Set array of extra state variables.
<code>vs_copy_extra_state_vars_to_array</code>	Get array of extra state variables.
<code>vs_get_export_names</code>	Get an array of the names of all active Export variables.
<code>vs_get_import_names</code>	Get an array of the names of all active Import variables.
<code>vs_get_state_q</code>	Get the array of Qs (state variables calculated with ODEs)
<code>vs_n_derivatives</code>	Number of derivatives.
<code>vs_n_extra_state_variables</code>	Number of extra state variables.

- The differential state variables are those calculated by the VS Math Model using numerical integration; the number of differential state variables is obtained with the function `vs_n_derivatives`.
- The extra state variables are calculated directly; the number of these variables is obtained with the function `vs_n_extra_state_variables`.
- The complete set of all state variables is the union of the above two arrays.

Note The array of differential state variables was originally named Q, following a convention in some textbooks on multibody dynamics.

`vs_get_export_names`

`vs_get_import_names`

Declarations:

```
int vs_get_export_names(char **names);
int vs_get_import_names(char **names);
```

Return:

The number of active Import or Export variables. The names are copied into an array of strings provided as the argument *names*.

These functions provide the names of the variables currently activated for Export or Import.

Built-in Export variables all have names that begin with "EXP_"; these four characters are removed from the names that are returned. For example, an Export variable named "EXP_YAW" would produce the name "YAW" in the `names` array sent to `vs_get_export_names`.

Built-in Import variables all have names that begin with "IMP_"; these four characters are removed from the names that are returned. For example, an Import variable named "IMP_FX_L1" would produce the name "FX_L1" in the `names` array sent to `vs_get_import_names`.

vs_get_state_q

Declaration:

```
void vs_get_state_q) (vs_real *array);
```

This function obtains a pointer to the internal array of state variables that are calculated with differential equations. The number of variables can be obtained with `vs_n_derivatives()`. This array includes all state variables that are built-in plus all that were added with VS Commands plus all that were added by using the API function `vs_define_sv` to define the variable and then using the API function `vs_set_sym_attribute` to set a pointer to the derivative (page 60).

vs_n_derivatives

vs_n_extra_state_variables

Declarations:

```
int vs_n_derivatives (void);  
int vs_n_extra_state_variables (void);
```

Return:

`vs_n_derivatives` returns the number of differential state variables.
`vs_n_extra_state_variables` returns the number of extra state variables.

These two functions can be used to determine the dimensions needed to create arrays for copying the state of the VS Math Model.

Save and Restore the Model State

For conventional simulations made with a VS Math Model, the model arrays are managed completely by the VS Math Model, and there is no need to be concerned with the organization of the arrays or their contents. However, there may be unconventional applications where access is needed for these arrays. One example is an option to back up in time and restore the state of the model to a previously recorded state. This capability is provided to support control optimizations in which the controller can back up and try different settings if the vehicle response is not satisfactory.

Table 17 lists the main functions used for saving and restoring of the state of the math model.

Saving a state is straightforward. However, restoring a state can sometimes be more complicated because it is necessary to reset the simulation time along with the internal arrays managed by the VS Math Model. The method for resetting the model depends on the runtime environment:

Table 17. Functions used to save and restore the model state.

Function	Description
<code>vs_free_saved_states</code>	Free memory used to save states.
<code>vs_get_request_to_restore</code>	See if there has been a request to restore a state.
<code>vs_get_request_to_save</code>	See if there has been a request to save a state.
<code>vs_get_saved_state_time</code>	Get the time of a saved state that can be restored.
<code>vs_restore_state</code>	Restore a saved state.
<code>vs_save_state</code>	Request that current state of the model be saved.
<code>vs_set_request_to_restore</code>	Request that a previously saved state be restored.
<code>vs_start_save_timer</code>	Start a timer to automatically save at a time interval.
<code>vs_stop_save_timer</code>	Stop the timer for automatically saving.

1. Restoring the state can be handled automatically if the VS Math Model is being run using the `vs_run` function or if it is run from a C/C++ program that uses `vs_integrate` to step through the run. (In these cases, the simulation time is automatically reset.) In these cases, the restoration can be triggered with the VS Command `RESTORE_STATE` or the API function `vs_set_request_to_restore`.
2. If the VS Math Model is being run from MATLAB, VB, or any program using `vs_integrate_io`, then restoration requires that the wrapper program check each timestep to see if a restart was requested, using the function `vs_get_request_to_restore`. If this function returns +1, then the function `vs_restore_state` should be used to restore the state of the internal matrices, and the returned value should be used to reset the simulation time.
3. Restoring the state while under the control of another simulation workspace, such as Simulink or LabVIEW, is not directly supported because the simulation clock cannot be controlled from the VS Math Model.

The capability to restore a past state also requires that the number of state variables be unchanged between the time a state was saved and the time it is restored. If the `vs_define_sv` function is used after a state is saved, then that state cannot be restored because the array sizes are not consistent. The same restriction applies for variables added with the VS Commands `ADD_VARIABLE` and `ADD_OUTPUT`. On the other hand, it is OK to use `vs_define_par` or the VS Command `ADD_PARAMETER` after a state is saved, because parameters are not affected by a restore operation.

`vs_free_saved_states`

Declaration:

```
void vs_free_saved_states (void);
```

This function frees memory used by `vs_save_state` to save the state of the model at one or more instances. It is applied automatically at the end of a run.

This function corresponds directly to the VS Command `RESET_STATES`.

vs_get_request_to_restore

Declaration:

```
int vs_get_request_to_restore (void);
```

Return:

−1 means a state was restored since the previous timestep, 0 means no restore was requested this timestep, +1 means a restore has been requested and will be made before the next timestep.

The VS Math Model has an internal variable that is given a value of 0, -1, or +1 to indicate the status of a request to restore a previous state. Such a request can be made three ways:

1. The VS Command `RESTORE_STATE` could be encountered when reading a Parsfile as the result of an Event being triggered.
2. The `vs_set_request_to_restore` function could be applied from within a `C_calc` callback function (callback functions were described in Chapter 6).
3. The `vs_set_request_to_restore` function could be applied from a wrapper program written in any language that supports the VS API.

If the model is run from a program written in C/C++, then this function might not be needed.

The function `vs_get_request_to_restore` is provided for use in external code that must support the restore action by also restoring the simulation time and/or externally defined model parts. In these cases, the status of the request should be checked each timestep: a status of −1 means the restore action has taken place in the VS Math Model, a value of +1 means that a restart is about to take place, and a value of 0 means no restart actions are associated with the current timestep.

At the end of each timestep, the status of the request is updated. If the request is −1 (restored previous timestep), it is changed to 0. If it is +1, then the `vs_restore_state` function automatically changes it from +1 to −1.

vs_get_request_to_save

Declaration:

```
int vs_get_request_to_restore (void);
```

Return:

−1 means a state was saved within the last timestep, 0 means no save actions have been performed or requested this timestep, +1 means a save has been requested and will be made before the next timestep.

Saving the state of the VS Math Model is always handled internally, so this function is not needed unless external code must perform save operations when the VS model is saved. For example, if an externally defined model has its own state variables that are not installed in the VS model, then it might be necessary to save them to properly support a restore operation.

vs_get_saved_state_time

Declaration:

```
vs_real vs_get_saved_state_time (vs_real time);
```

Return:

The last time that a state that was saved at or before *time*.

This returned value is the time that would be restored if a request were made via `vs_set_request_to_restore` using *time*.

vs_restore_state

Declaration:

```
vs_real vs_restore_state (void);
```

Return:

Simulation time when the state was saved.

This function restores the most recently saved state of the math model at or before the time specified with the `vs_set_request_to_restore` function or the VS Command `RESTORE_STATE`. If the model was not saved at or before the requested time, then an error is generated and the run stops.

This function is applied automatically if the run is made with the API function `vs_run`, or with custom code that uses `vs_integrate` to update the state variables each timestep. In other cases (e.g., using `vs_integrate_io` to update the state variables), the restore operation must be applied from external code in order to reset the simulation time *t*. The external code should check the value returned by `vs_get_request_to_restore` to see if the restore action should be applied (it should be applied if the returned value is +1).

The function `vs_restore_state` goes through the saved states to find the most recent one that was saved at or before the requested simulation time, and then restores all state variables and the Import array. Other properties of the model (parameters, tables, user-supplied VS Command equations, etc.) are not modified. If any parameters or other properties were changed after the state was saved, then the simulation will proceed starting from the saved time, position, orientation, etc., but will continue using equations of motion based on the current properties.

If changes have been made since the state was saved (these would be change that affect the number of derivative state variables, extra state variables, or the Import array) then the restore operation will fail and an error message will be logged.

This function sets the internal request variable to -1.

When this function goes through the internal list of saved states, it erases all of those that were saved at time more recent than the requested restore time. For example, if the requested restore is for *T* = 15.6, then all saved states made after *T* = 15.6 are automatically cleared from memory.

vs_save_state

Declaration:

```
void vs_save_state (void);
```

This function sets an internal variable to a value of +1, indicating that a request is made to save the state of the model.

The save operation is made at the end of the timestep. The VS Math Model allocates memory and copies the values of all state variables, elements of the Import array (if active), and the current simulation time. The internal request variable is changed from +1 to -1 to indicate that the save was made. (The value of the internal variable can be obtained with the function `vs_get_request_to_save`.)

All of the saved states are cleared using the function `vs_free_saved_states`.

This function corresponds directly to the VS Command `SAVE_STATE`.

vs_set_request_to_restore

Declaration:

```
void vs_set_request_to_restore (vs_real time);
```

This function sets an internal variable to a value of +1, indicating that a request is made to restore the state of the model. It sets another internal variable to *time*, indicating the past simulation time to which the model will be restored.

The restoration is done at the end of the timestep with the function `vs_restore_state`.

This function corresponds directly to the VS Command `RESTORE_STATE`.

vs_start_save_timer

Declaration:

```
void vs_start_save_timer (vs_real time_interval);
```

This function starts an internal timer to request a save at the specified *time_interval*. If this function is applied at the start of a run (when the simulation time *T* is equal to the parameter `TSTART`), the first save will be applied at $t = t_{start} + time_interval$. However, if this function is applied after the run has started, then the first save will occur at the current time.

This function corresponds directly to the VS Command `START_SAVE_TIMER`.

The timer remains in effect until stopped with the API function `vs_stop_save_timer` or the VS Command `STOP_SAVE_TIMER`.

vs_stop_save_timer

Declaration:

```
void vs_save_state (vs_real time_interval);
```

This function stops the internal timer that was created with the function `vs_start_save_timer` or the VS Command `START_SAVE_TIMER`.

This function corresponds directly to the VS Command `STOP_SAVE_TIMER`.

9. Steps in the Simulation Process

A VS simulation run normally occurs in five major steps:

1. Load the selected VS Solver library into memory.
2. Initialize.
3. Run the simulated test.
4. Terminate the run.
5. Unload the VS Solver.

The following subsections describe the behavior of the VS Math Model after the VS Solver has been loaded into memory, up until the run is terminated and the solver can be unloaded. It includes all the steps taken by the VS Math Model with built-in features, plus equations added at runtime with VS Commands, plus potential communication with external software such as Simulink using Import and Export arrays, plus external callback functions that were described in Chapter 6.

Stages Each Timestep

Each timestep, equations might be applied in four sequential stages. These stages are also used to initialize the model.

State stage

All state variables that were obtained with ordinary differential equations (ODEs) have values calculated for the new timestep. The purpose of this stage is to ensure that all variables that provide information about the vehicle location and environment are available for subsequent calculations. There are three sets of calculations to achieve this:

1. User-defined equations are applied:
 - a. Apply callback functions of type `calc` with the location `VS_EXT_EQ_STATE`
 - b. Apply equations added with the `EQ_STATE` VS Command.
2. The VS Math Model calculates a few important vehicle variables that were not calculated from ODEs, such as Station along a path and forward speed. It also calculates locations and speed of all moving objects and resulting detections of ADAS range and tracking sensors, if installed.

Variables that are essential for defining the environment are also calculated (wind, vehicle station, vehicle forward speed, locations and speeds of target objects, and ADAS sensor detections). None of the calculated motions of the vehicle are available yet, nor are any forces or moments.

Controls stage

Control inputs for a vehicle model are calculated, such as steering and speed control. None of the multibody vehicle calculations have been made yet. This stage has two sets of equations:

1. User-defined equations are applied:
 - a. Apply callback functions of type `calc` with the location `VS_EXT_EQ_IN`

- b. Apply equations added with the `EQ_IN` VS Command.
2. The VS Math Model applies the built-in driver models for steering and speed control.

Kinematics stage

This stage includes the portion of the multibody vehicle model that involves kinematical equations that determine locations of tire contact points, suspension motions, and any vehicle variables with internal units of meters, radians, meters/second, or radian/second. This stage has two sets of equations:

1. User-defined equations are applied:
 - a. Apply callback functions of type `calc` with the location `VS_EXT_EQ_KIN`
 - b. Apply equations added with the `EQ_KIN` VS Command.
2. The VS Math Model applies the built-in kinematical equations for the multibody vehicle model.

Dynamics stage

This stage applies all remaining equations for the VS Math Model:

1. User-defined equations are applied before the built-in dynamic equations of the vehicle model:
 - a. Apply callback functions of type `calc` with the location `VS_EXT_EQ_DYN`
 - b. Apply equations added with the `EQ_DYN` VS Command.
2. The VS Math Model applies the built-in dynamics equations and combines equations from multiple modules in the vehicle to obtain the remaining derivatives, which are mainly accelerations. It also calculates all built-in output variables that have not been calculated yet for the timestep.
3. User-defined equations are applied after all the built-in equations:
 - a. Apply callback functions of type `calc` with the location `VS_EXT_EQ_OUT`
 - b. Apply equations added with the `EQ_OUT` VS Command.
 - c. Apply equations added with the `EQ_DIFFERENTIAL` VS Command

Save stage

After all calculations have been made, some values can be saved for possible use in the next timestep:

1. The VS Math Model saves some internal variables that are needed in the next timestep.
2. User-defined equations are applied after all the built-in Save equations:
 - a. Apply callback functions of type `calc` with the location `VS_EXT_EQ_SAVE`
 - b. Apply equations added with the `EQ_SAVE` VS Command.

Exceptions

All four stages exist in every single VS Solver. However, the stages are not used in some solvers that are not for full VS vehicle models. For example, the VS Solver in SuspensionSim only uses the Kinematics stage. Other tools, such as the tire testers, do not use any of the stages; those libraries are used as servers that are queried by a wrapper program to exercise a tire model.

Initialize

Before a simulation run can be started, the solver module allocates memory, sets default values, reads input files, and prepares for the run. If using Import and Export arrays, the external wrapper (e.g., VS S-Function in Simulink) uses the function `vs_read_configuration`, followed by `vs_copy_io`. Otherwise, the initialization can be handled with `vs_run`. If more control over the simulation process is needed, then the functions `vs_setdef_and_read` and `vs_initialize` can be used to perform the two steps of the initialization, respectively.

Figure 14 shows a schematic timeline for the initialization steps that are described below. In this figure, actions always taken by the VS Math Model using built-in code are indicated with blue blocks above the timeline; equations added with VS Commands are indicated with tan blocks above the timeline; actions involving communication with external software are shown with white blocks; and calls to external callback functions are indicated by the type (`setdef`, `scan`, `echo`, or `calc`) and location (if applicable) below the timeline. Some of the built-in actions may be skipped, depending on user settings. These are shown with gray blocks.

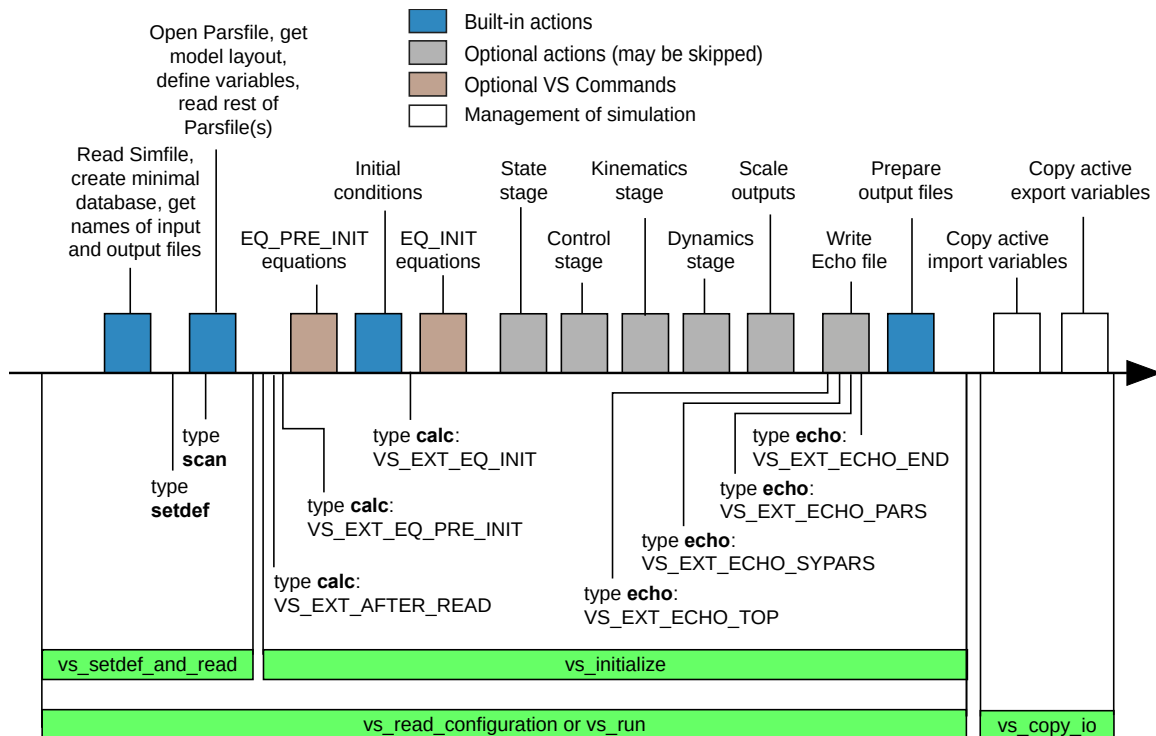


Figure 14. Schematic timeline for the VS Math Model initialization.

1. The solver starts the simulation by allocating memory, creating a minimal internal database, and reading input files that cause most of the model variables to be defined and

given values. This work is done by the function `vs_setdef_and_read`, which performs the following actions:

- a. Read a Simfile to obtain names of input and output files; allocate memory, create an internal database, define some core system variables, and determine how many variables will be imported and exported.
 - b. Apply callback functions of type `setdef`.
 - c. Begin reading the input Parsfile and obtain the model layout. From that information, activate modules based on the layout, extend the database with model parameters and variables, and continue to read from input Parsfiles. Any line from the input Parsfile that has a keyword not recognized by the solver will be passed to the callback functions of type `scan` (if any such functions were installed) for possible reading of values for variables used in external code. After the Parsfiles have been read, determine how many variables will be imported and exported, and allocate the Import and Export arrays accordingly.
2. The remaining initialization work is managed by the function `vs_initialize`, which first makes initialization calculations that depend on the values read in step 1. These include the following steps:
 - a. Apply callback functions of type `calc` with location `VS_EXT_AFTER_READ`.
 - b. Lock the math model and create arrays for ODEs.
 - c. Apply callback functions of type `calc` with location `VS_EXT_EQ_PRE_INIT`.
 - d. Apply equations provided at runtime with the VS Command `EQ_PRE_INIT`.
 - e. Perform built-in initialization calculations.
 - f. Apply callback functions of type `calc` with the location `VS_EXT_EQ_INIT`.
 - g. Apply equations provided at runtime with the VS Command `EQ_INIT`.
3. Optionally apply the model equations used each timestep, to prepare some parameters with calculated values that will appear in the Echo file, and output variables that might be used by external software such as Simulink. This step can be disabled with the parameter `OPT_SKIP_INIT_DYN`.
 - a. Apply all equations from the State stage.
 - b. Apply all equations from the Control stage.
 - c. Apply all equations from the Kinematics stage.
 - d. Apply all equations from the Dynamics stage.
 - e. Scale: convert all potential outputs from dynamical units to user units.
4. Write an Echo file that lists all the model parameters, Configurable Functions, and VS Commands. The values are labeled and converted from internal dynamical units to user units. If provided, any callback functions of type `echo` also write information into the echo file. This step is skipped here if the parameter `OPT_DELAY_INIT` is set to 1.
5. Prepare to record output variables during the simulation run:

- a. Write the text ERD or VS header file with information about the variables to be recorded (names, units, etc.).
 - b. If values will be written to file during the run, then create the binary file for writing. Otherwise, allocate memory for a buffer in which to store the values as the run progresses. (The option to store results in memory is selected with the parameter `OPT_BUFFER_WRITE`).
6. Copy import and export values to and from arrays shared with an external wrapper via the function `vs_copy_io`. This provides current values of export variables to the wrapper for use in the first timestep. It also clears default values for import variables that were set by the `IMPORT` command to receive initial values from the wrapper.

At the end of step 4, the model is still not fully initialized. A few more steps are taken during the first set of operations at a major step, as described in the next subsection.

Run

The simulation is made by incrementing time by a small step and performing calculations to update the state variables and output variables for each new time. This is done automatically by the function `vs_run`. Several functions are available in the VS API for integrating the equations over a timestep, with the main options being the functions `vs_integrate` and `vs_integrate_io`.

Operations at the Full Step (Baseline Sequence)

Figure 15 shows a schematic timeline for the main steps of the specified timestep when `vs_integrate` is used or when `vs_integrate_io` is used with the parameter `OPT_IO_SYNC_FM` set to 0. In the figure, ΔT is `TSTEP` for the Euler and AB-2 methods; it is `TSTEP/2` for the AM and RK methods.

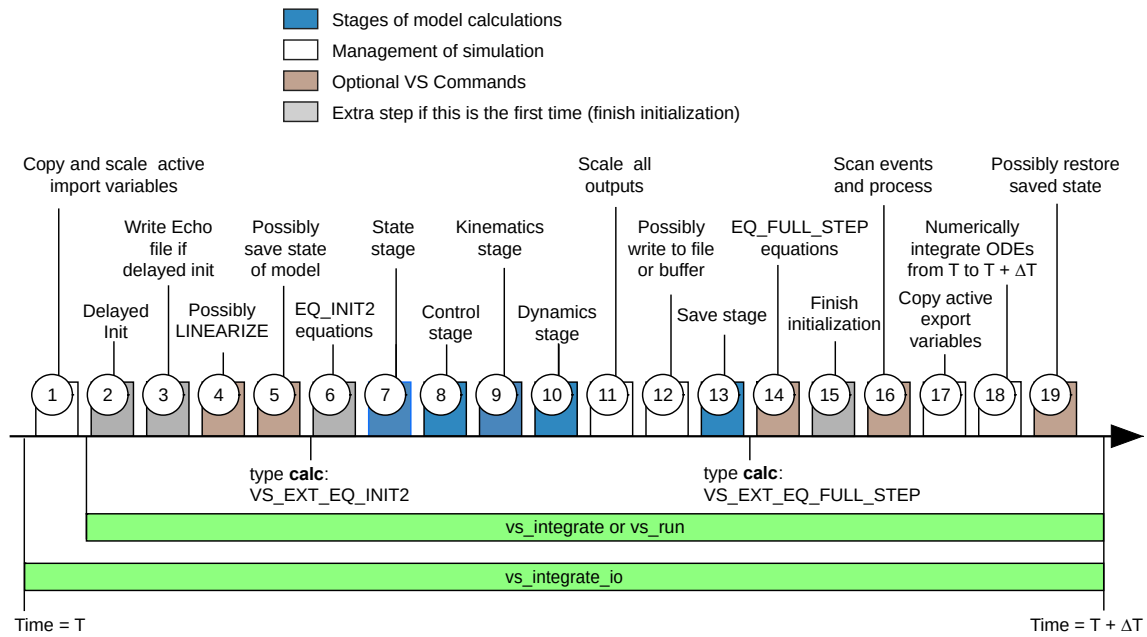


Figure 15. Schematic timeline for the main timestep, using the conventional sequence.

The following actions are performed at each timestep `TSTEP`.

- ① If there is an external workspace such as Simulink, the Import variables are copied from that workspace and converted from user units to internal dynamical units. This step is done with `vs_integrate_io` but not with `vs_run` or `vs_integrate`.
- ② If the parameter `OPT_DELAY_INIT` was set to 1, a delayed initialization is performed in the first timestep to include effects of the initial Import variables received from external software.
- ③ If the parameter `OPT_DELAY_INIT` was set to 1, the Echo file is written in the first timestep.
- ④ If requested via the command `LINEARIZE`, then perform the linearization calculations and generate an output M-file.
- ⑤ Check for requests to save the current state of the entire VS Math Model. (These requests are generated from the use of the VS Commands `SAVE_STATE` and `START_SAVE_TIMER` and the API functions `vs_save_state` and `vs_start_save_timer`.) If there is a request, then allocate memory and save the state together with the current time.
- ⑥ If this is the first timestep, process `INIT2` calculations:
 - a. Apply callback functions of type `calc` with the location `VS_EXT_EQ_INIT2`.
 - b. Equations provided at runtime with the VS Command `EQ_INIT2` are applied.
 - c. Set an internal flag indicating that the initialization is complete.
- ⑦ Apply the State stage of the equations, just after calculations from the callback `VS_EXT_EQ_STATE` and VS Command `EQ_STATE`.
- ⑧ Apply the Control stage of the equations, just after calculations from the callback `VS_EXT_EQ_IN` and VS Command `EQ_IN`.
- ⑨ Apply the Kinematics stage of the equations, just after calculations from the callback `VS_EXT_EQ_KIN` and VS Command `EQ_KIN`.
- ⑩ Apply the Dynamics stage of the equations, just after calculations from callback `VS_EXT_EQ_DYN` and VS Command `EQ_DYN`, but before and callback `VS_EXT_EQ_OUT`, and the VS Commands `EQ_OUT` and `EQ_DIFFERENTIAL`.
- ⑪ Scale all potential outputs from internal SI units to user units.
- ⑫ Determine if the output values should be saved for this step. If so, write output variables to file or save in memory for later writing.
- ⑬ If the run is not finishing, perform the Save stage, including equations added with the VS Command `EQ_SAVE` and callback `VS_EXT_EQ_SAVE`.
- ⑭ Process equations provided with the VS Command `EQ_FULL_STEP` and callback `VS_EXT_EQ_FULL_STEP`.
- ⑮ If this is the first timestep, finish the initialization by writing outputs to file or saving in a buffer for later writing, and initialize counters for VS Events and writing to file.

- ①⑥ Scan a list of potential VS Events to see if any Events have occurred. If an Event occurred, either terminate the run or obtain new data and continue:
 - a. If no Parsfile was specified when the Event was defined, then terminate the run.
 - b. If a Parsfile name was associated with the Event, then the program goes back to step 1b to read the new Parsfile. It then continues from there.
- ①⑦ If running with `vs_integrate_io` and there is an array of outputs shared with the software, copy scaled output variables into the shared array.
- ①⑧ Numerically integrate the derivatives of the state variables. If the integration method is Euler or AB-2, the values are updated to time $T+TSTEP$; for the AM and RK methods they are updated to time $T + TSTEP/2$. (For details about the available numerical integration methods, please see the tech memo *Numerical Integration Methods in VS Math Models*.)
- ①⑨ Check for requests to restore a previous state, generated with the VS Command `RESTORE_STATE`. If there is a request and the state was saved near the requested time, then restore the state variables and the simulation time T that was associated with the saved state.

Operations at the Half Step

The VS Math Models support several numerical integration methods, as described the *VS Math Models Manual*. The method is set at runtime with the parameter `OPT_INT_METHOD`. Four of the methods require breaking the integration into two steps: the first goes from T to $T+TSTEP/2$, and the second continues to $T+TSTEP$. Figure 16 shows the timeline for the half-step calculations.

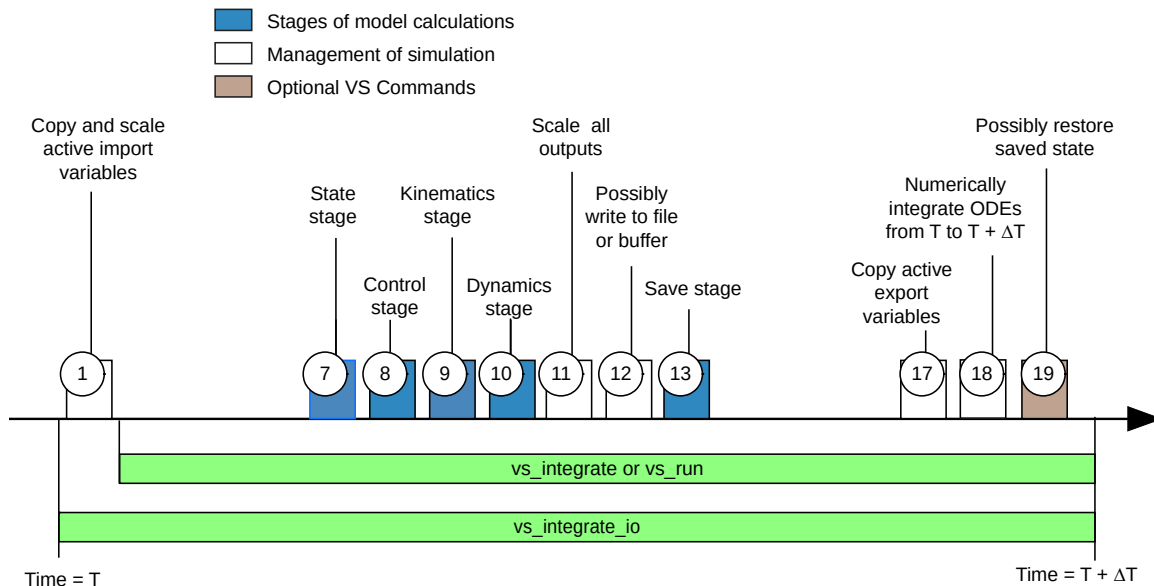


Figure 16. Schematic timeline for actions taken at the half-step.

In comparing this to Figure 15, many of the steps applied at the major step are skipped at the half step. Referencing the numbered items in the previous subsection, the deleted steps are:

- If the parameter `OPT_IO_UPDATE` is zero, skip steps ① (import) and ①⑦ (export).

- Always skip the steps not shown in Figure 16.

If the integration method is Euler or AB-2, there are no operations at the half step.

Operations at the Full Step (Synchronized: `OPT_IO_SYNC_FM = 1`)

An alternative timing sequence is used internally when the system parameter `OPT_IO_SYNC_FM` is set to 1. In this case, forces and moments calculated externally are synchronized with the rest of the model by using a different calculation sequence (Figure 17).

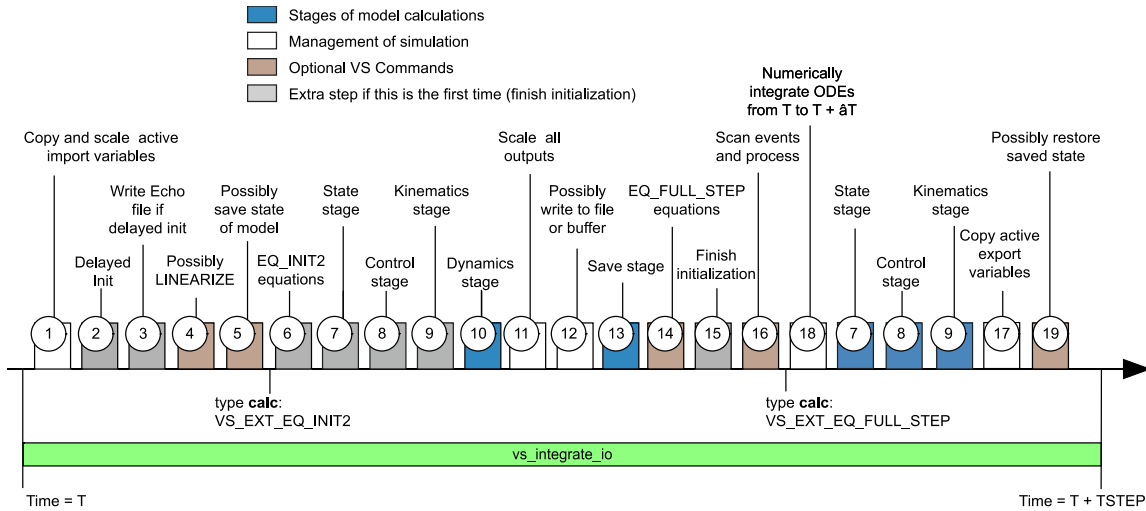


Figure 17. Schematic timeline for the main timestep, using the “synchronized” sequence.

With this sequence, the stages State (7), Control (8), and Kinematics (9) are shown twice. If this is the first timestep, they are applied at the start of the timestep immediately after the import of external variables as the final step in the initialization. After that first step, they are not applied at the start, because the values of the kinematical variables were done the previous step. Instead, after the import of external variables (1) and optional Linearize (4) and Save options (5), the calculations resume with the Dynamics stage (10). Going from this stage to the scanning for Events (16), the sequence of actions is the same as described earlier (Figure 15, page 93). However, in this version, the next step is to calculate all ODE state variables for the next timestep (18). After the state variables are known for time $T + TSTEP$, stages State, Control, and Kinematics are applied. ((7), (8), and (9)). Scaled copies of the activated outputs are then put into the Export array (17).

With this sequence, the variables calculated in the State, Control, and Kinematic stages apply for time $T + TSTEP$, while all other variables (forces, accelerations, miscellaneous outputs) have values calculated for time T . The next timestep, forces and moments imported from the external model will be synchronized with the new time $T + TSTEP$.

Operations with a Series of Wrappers

CarSim, TruckSim, and BikeSim support up to four Simulink S-Function wrappers that can link to a VS Math Model at different stages of the calculations each timestep. The four S-Functions available in the Simulink library are each tied to one of the four stages for the calculations in the VS Math Model. They are `vs_state`, `vs_ctl`, `vs_kin`, and `vs_dyn`. Each works with its own Import

and Export array. If all four are used in the same simulation, there are eight arrays available to exchange information between the Simulink model and the VS Math Model.

When running in Simulink with multiple VS V-Functions for the same VS Math Model, each wrapper takes control in series, and uses `vs_integrate_io` to allow the VS Math Model to perform part of the model calculations. The logic in `vs_integrate_io` ensures that all the operations shown for the full step (Figure 15, page 93) are completed by the end of the timestep.

There are three possible cases for the portion of the timeline used by each wrapper:

1. The first wrapper runs through the numbered actions ① to ⑥. If this is **vs_ctl** it also runs through the Control stage ⑧. If **vs_kin** is the first wrapper, it also runs both the Control ⑧ and Kinematics ⑨ stages. It then scales potential exports ⑪, copies scaled values from the model to the export array ⑰, and exits the VS Math Model.
2. If there are three or four wrappers, then the wrappers that are neither first or last copy and scale import variables from the Import array managed by the wrapper ①. If any previous stage(s) have not been run, the wrapper does so. It then runs the stage associated with the wrapper (Control ⑧ or Kinematics ⑨), scales potential exports ⑪, copies values from the model to the export array ⑰, and exits the VS Math Model.
3. The last wrapper prepares the imports ①. If any previous stage(s) have not been run, the wrapper does so. It then runs the stage associated with the wrapper. If this is not **vs_dyn**, it runs all remaining stage up to Dynamics ⑩. It then performs the remaining actions for the timestep (⑪ - ⑱) and exits the VS Math Model.

Terminate

When the run ends, the solver program cleans up and shuts down. This is handled either by `vs_run`, `vs_terminate`, or `vs_terminate_run`.

1. Apply callback functions of type `calc` with the location `VS_EXT_EQ_END` to finalize any statistics or clean up in the external code.
2. Process equations provided with the VS Command `EQ_END`.
3. Set parameter values for the possibility that a future run will continue from where this one stopped. For example, set the start time to the current time, and turn off some initialization options to allow the VS Math Model state variables to be left intact.
4. Finish the writing of outputs:
 - a. If variables were saved in memory (e.g., for RT operation), write to file.
 - b. Close the file where output variables were written.
5. Optionally write an End Parsfile that can be read as input to continue the run later.
6. Free memory that was explicitly allocated during the initialization and) the run.

Appendix: Legacy API Functions

The VS API was introduced with CarSim 7.0 in 2007. It has been extended many times with new product releases. Improvements have made some of the original functions and techniques unnecessary or obsolete.

Alternative Legacy Functions

This subsection describes API functions that have been replaced with newer versions that are more efficient to use. However, these legacy functions still exist and older code that uses them will probably still function.

Defining New Variables

Table 18 lists legacy functions for defining new variables. Most of these only set a few properties of the new variable; newer `vs_define_` functions are more convenient to use because they set more properties of the internal structure made for the new variable.

Table 18. Alternative legacy functions.

Function	Description
<code>vs_define_imp</code>	Define a new Import variable (older versions with fewer arguments)
<code>vs_define_import</code>	
<code>vs_define_indexed_parameter_array</code>	Support the <code>vs_define_ipar_nd</code> functions
<code>vs_define_out</code>	Define a new output variable (older versions with fewer arguments)
<code>vs_define_output</code>	
<code>vs_define_out_with_shortcode</code>	
<code>vs_define_parameter</code> <code>vs_define_parameter_int</code>	Define a new model parameter
<code>vs_define_variable</code>	Define a new state variable
<code>vs_set_units</code>	Set the units associated with a variable or parameter

`vs_define_imp`

Declaration:

```
int vs_define_imp(char *keyword, vs_real *real, char *units,  
                 int native, char *component, char *desc);
```

Return:

ID of the new Import if successful; it returns -2 and generates an error if the keyword is already associated with an existing variable. ID numbers start at one and increase with each Import. Thus, the return value is also the total number of Imports that are now installed.

`vs_define_imp_where` was introduced in 2023 and is preferred over `vs_define_imp` because it provides all information about the new Import variable with one call (page 55). It also drops the `*real` argument, which has not been used for a decade in VS Solvers. If this function is used, specify `NULL` for the `*real` argument. All other arguments have the same definitions as with `vs_define_imp_where`.

`vs_define_import`

Declaration:

```
int vs_define_import(char *keyword, char *desc, vs_real *real,
                    char *units);
```

Return:

ID of the new Import if successful; it returns -2 and generates an error if the keyword is already associated with an existing variable.

`vs_define_imp` was introduced in 2014 as an alternative for `vs_define_import`, and `vs_define_imp_where` was introduced in 2023 as an alternative for `vs_define_imp`. The function `vs_define_imp_where` provides all information about the new Import variable with one call (page 55). The legacy functions require additional calls to specify other internal properties of the symbolic structure. The arguments *keyword*, *desc*, and *units* have the same definitions as with `vs_define_imp_where`. The argument `*real` is no longer supported; if using this function, specify `NULL` for that argument.

`vs_define_indexed_parameter_array`

Declaration:

```
int vs_define_indexed_parameter_array(char *keyword);
```

Return:

The integer ID needed for other VS functions such as `vs_set_sym_attribute` and `vs_set_sym_int`.

This legacy function `vs_define_indexed_parameter_array` was used prior to 2014, along with multiple calls to `vs_set_sym_attribute` and `vs_set_sym_int`. Newer `vs_define_ipar_nd` functions are now recommended (`vs_define_ipar_1d`, `vs_define_ipar_2d`, etc., page 56).

`vs_define_out`

`vs_define_out_with_shortcode`

Declaration:

```
int vs_define_out (char *shortcode, char *longname,
                  vs_real *real, *units, char *genname, char *bodyname,
                  char *component));

int vs_define_out_with_shortcode (char *shortcode, char *erlname,
                                  char *longname, vs_real *real, char *units,
                                  char *genname, char *bodyname, char *component));
```

Return:

ID of the new output if successful; it returns -2 and generates an error if the keyword is already associated with an existing variable. ID numbers start at one and increase with each variable added. Thus, the return value is also the total number of outputs that are now installed.

`vs_define_out_where` was introduced in 2023 and is preferred over `vs_define_out` and `vs_define_out_with_shortcode` because it provides all information about the new output variable with one call (page 57).

The function `vs_define_out_with_shortcode` has all the arguments of the newer function except for the last one (*where*, the stages in the calculations where the output is calculated). The function `vs_define_out` has all but two of the same arguments.

`vs_define_output`

Declaration:

```
int vs_define_output(char *shortcode, char *longname,
                    vs_real *real, char *units);
```

Return:

ID needed for other VS functions for the new output if successful; this returns -2 and generates an error if the keyword is already associated with an existing variable.

`vs_define_out` was introduced in 2014 and was the preferred function for adding output variables until 2023, when `vs_define_out_where` was introduced.

The arguments *shortcode*, *longname*, *real*, and *units* have the same definitions as with `vs_define_out_where`.

`vs_define_parameter`

Declaration:

```
int vs_define_parameter(char *keyword, char *desc, vs_real *real,
                       char *units);
```

Return:

ID of the new parameter if successful; it returns -2 and generate an error if the keyword is already associated with an existing variable. ID numbers start at one and increase with each variable added. Thus, the return value is also the total number of parameters that are now installed.

`vs_define_par` was introduced in 2014 and is the preferred function to introduce new parameters because it provides all information for most parameters with one call (page 58). The legacy `vs_define_parameter` function requires additional calls to set other internal properties of the symbolic structure.

vs_define_parameter_int

Declaration:

```
int vs_define_parameter_int(char *keyword, char *desc, int *par)
```

Return:

ID of the new parameter if successful; it returns -2 and generates an error if the keyword is already associated with an existing variable.

This legacy function adds an integer parameter variable to the math model. The newer function `vs_define_par` performs the same function if the *units* string is set to NULL (page 58).

vs_define_variable

Declaration:

```
int vs_define_variable(char *keyword, char *desc, vs_real *real);
```

Return:

This returns the ID of the new variable if successful; it returns -2 and generates an error if the keyword is already associated with an existing variable.

`vs_define_sv` was introduced in 2014 and is the preferred function because it provides more information about the new variable with one call (page 60). The legacy `vs_define_variable` function requires additional calls to set the units and default value for the state variable.

vs_set_units

Declaration:

```
void vs_set_units (char *var_keyword, char *units_keyword);
```

Use this function to set the units associated with a floating-point (`vs_real`) internal variable, such as an Import or output variable.

Both keywords must be ALL CAPS. The argument *var_keyword* identifies the variable and the argument *units_keyword* identifies the units. If *var_keyword* does not identify an internal variable, then this function does nothing. If *units_keyword* does not identify valid installed units, an error message is generated.

This legacy function is similar in function to the VS runtime command `SET_UNITS`. However, new variables introduced with functions such as `vs_define_par` and `vs_define_out_where` include the keyword for the units, so this function is seldom needed.

Running the Simulation

Table 19 lists legacy functions that used to help manage the running of a simulation.

Table 19. Alternative legacy functions for running a simulation.

Function	Description
<code>vs_add_echo_header</code>	Add a header to the Echo file.
<code>vs_copy_import_vars</code>	Copy Import variables from an array to the VS Math Model.
<code>vs_free_all</code>	Free memory allocated when the solver started.
<code>vs_integrate_IO</code> <code>vs_integrate_io_2</code>	Advance one timestep and transfer i/o data to arrays; same as <code>vs_integrate_io</code> .
<code>vs_opt_all_write</code>	Same as the <code>OPT_ALL_WRITE</code> VS Command.
<code>vs_set_opt_error_dialog</code>	Same as the <code>OPT_ERROR_DIALOG</code> VS Command.
<code>vs_scale_import_vars</code>	Convert Import variables from user to internal units.
<code>vs_set_sym_real</code>	Set a floating-point value for an internal variable.

`vs_add_echo_header`

Declaration:

```
void vs_add_echo_header (const char *text);
```

Add a header to the Echo file. This has been replaced by the function `vs_add_new_par_group`, which supports more control of the Echo file layout.

`vs_copy_import_vars`

Declaration:

```
void vs_copy_import_vars (vs_real imports[]);
```

Copy Import variables into the VS Math Model from the array provided as an argument. The contents of the array `imports` are left intact by this function.

This function is no longer needed in custom wrappers because it is applied automatically as needed by `vs_copy_io` and `vs_integrate_io`.

`vs_free_all`

Declaration:

```
void vs_free_all (void);
```

Free memory that was allocated by the VS Math Model from the time it was loaded. This function is automatically applied by the `vs_run`, `vs_terminate`, and `vs_terminate_run` functions and is usually not needed in external code.

`vs_integrate_IO`

Declaration:

```
int vs_integrate_IO(vs_real t,vs_real imports[],exports[]);
```

Return:

Zero if the run should continue; non-zero if the run should stop.

`vs_integrate_IO` is identical to `vs_integrate_io` and is provided for backward compatibility with the first version of the VS API.

`vs_integrate_io_2`

Declaration:

```
int vs_integrate_io_2(vs_real t, vs_real imports[],
                     vs_real exports[], (*external_calc) (vs_real, int));
```

Return:

Zero if the run should continue; non-zero if the run should stop.

This function is like `vs_integrate_io` with the addition of a callback function argument.

With the introduction of `vs_install_calc_function` in 2009, external calculation functions can be installed and applied properly with `vs_integrate_io` so the function `vs_integrate_io_2` is no longer necessary; it remains in the VS API to provide backward compatibility.

`vs_opt_all_write`

Declaration:

```
void vs_opt_all_write (int arg);
```

This function performs the same function as the VS Command `OPT_ALL_WRITE`. The same behavior can be obtained with the `vs_statement` function.

`vs_set_opt_error_dialog`

Declaration:

```
void vs_set_opt_error_dialog (int value);
```

This function performs the same function as the VS Command `OPT_ERROR_DIALOG`, which specifies whether to show popup messages during errors (0 = Hide, 1 = Show). The same behavior can be obtained with the `vs_statement` function.

`vs_scale_import_vars`

Declaration:

```
void vs_scale_import_vars (void);
```

If you use the legacy function `vs_copy_import_vars` to import variables into the VS Math Model, and the variables in the array were scaled in user units, then you should convert the variables in the solver to internal dynamical units using `vs_scale_import_vars`.

Conversion of units is done automatically with the `vs_copy_io` and `vs_integrate_io` functions; `vs_scale_import_vars` should not be used when those functions are used to copy variables.

`vs_set_sym_real`

Declaration:

```
int vs_set_sym_real(int id, vs_sym_attr_type type, vs_real
value);
```

Return:

If the function proceeded successfully, it returns a value of 0. It returns -1 and generates an error message if there is a problem, such as the ID number being out of range.

This legacy function sets a value to a floating-point variable in the math model. The functions `vs_define_xxx` set default values, so `vs_set_sym_real` is rarely used.

`vs_scale_import_vars`

Declaration:

```
void vs_scale_import_vars (void);
```

If you use the function `vs_copy_import_vars` to import variables into the VS Math Model, and the variables in the array were scaled in user units, then you should convert the variables in the solver to internal dynamical units using `vs_scale_import_vars`.

Note Conversion of units is done automatically with the <code>vs_copy_io</code> and <code>vs_integrate_io</code> functions; <code>vs_scale_import_vars</code> should not be used when those functions are used to copy variables.

Mechanical Simulation

755 Phoenix Drive, Ann Arbor MI, 48108, USA

Phone: 734 668-2930 • Fax: 734 668-2877 • Email: info@carsim.com

carsim.com